WM-PLN:
The World-Model Calculus

(DRAFT --- March 12, 2026)

Zar and Oruži[1]

[1]Oruži denotes AI-assisted collaboration using ChatGPT, Codex, and Claude Code.

Abstract

Probabilistic Logic Networks (PLN) couples logical structure with uncertain inference. In practice, however, many PLN presentations mix three distinct layers: (i) a probabilistic world-model posterior, (ii) an evidence algebra for compositional aggregation, and (iii) operational truth-value views (strength, confidence, interval bounds). This conflation produces *semantic drift*: algebraic operators are treated as if they were the logic itself, and link-level inference attempts to propagate correlation-sensitive quantities without carrying the information required to do so correctly.

We present the *world-model calculus*, the mathematical foundation underlying ξPLN, designed to be (a) correct by construction and (b) extensible to tractable sublayers. The core move is to make the complete layer a *posterior-state calculus*: the proof object is a revisable world-model posterior state, and all truth values are obtained by querying that state. We give a reference complete instance as a Dirichlet posterior over complete worlds (`JointEvidence`), prove that evidence extraction commutes with revision, and formalize a no-go theorem showing that complete link evidence cannot, in general, be computed from local per-link evidence alone. We then show how Bayesian-network-style sublayers, higher-order reductions, quantifier semantics, intensional inheritance, and inference-control theory arise as compiled tactics with explicit assumptions within the world-model framework.

All results are formalized in Lean 4 with Mathlib, totaling over 140 theorem-backed modules with zero `sorry`.

Contents

I Orientation **7**

1 Introduction **8**
1.1 Where Semantic Drift Came From 8
1.2 What the Refoundation Changes 9
1.3 What This Document Claims 9

2 Claim Taxonomy and Reading Guide **11**
2.1 How to Read This Document 12

3 Mathematical Desiderata for a Sound PLN **13**

II The Semantic Kernel **15**

4 Queries, Judgments, and World Models **16**
4.1 The Query Language 16
4.2 World Models 17
4.3 World Models with Side Conditions 17
4.4 The Solomonoff Connection 18

5 Reference Complete Semantics: Posterior-State Calculus **19**
5.1 The Reference Instance: Dirichlet over Worlds 19
5.2 The Sequent-Calculus View 20
5.2.1 Judgments 20
5.2.2 Core Rules 20
5.3 Probability Views 21
5.4 Derived Link Calculus: Classic PLN Rules as Admissible Rewrites 21
5.4.1 The Deduction Formula 21
5.4.2 Induction and Abduction 22

6 The Local-Completeness No-Go Theorem **23**
6.1 Interpretation 24

III Evidence and Truth-Value Views **25**

7 Evidence Algebra **26**
7.1 Evidence as Counts 26
7.2 Revision as Addition 26
7.3 Information Ordering 27
7.4 Quantale Structure 27
7.5 Heyting Structure 28
7.6 What Evidence Algebra Is Not 28

8 Truth-Value Views: Strength, Confidence, Intervals **29**
8.1 Simple Truth Values 29
8.2 Confidence as a View 30
8.3 Indefinite Truth Values 30
8.4 Why Views Are Not Proof Objects 31

9 Distributional and Interval Semantics **32**
9.1 Beta Distributions and Evidence 32
9.2 Dirichlet Distributions for Multi-Valued Domains 33
9.3 Normal-Gamma Evidence for Continuous Domains 33
9.3.1 Applied Example: Hybrid Sensor Monitoring 36
9.3.2 Applied Example: Widget Factory Source-Conditional Mixtures 39
9.3.3 Applied Example: Kalman Filter Wake/Sleep Temporal Filtering 39
9.3.4 Applied Example: Gas Sensor Array Drift Classification 41
9.3.5 Applied Example: Steel Plates Fault Classification . . 47
9.3.6 Applied Example: NAB Machine Temperature Anomaly Detection 51
9.3.7 Applied Example: Good Judgment Project Forecast Aggregation 55
9.3.8 Summary of Numerical Results 58
9.4 Walley Predictive vs. Bayesian Credible Intervals 58
9.5 The De Finetti Bridge 60
9.6 Knuth–Skilling Foundations 61

IV Compiled Inference and Tractable Fragments **62**

10 Σ-Guarded Rewrites **63**
10.1 The Rewrite Pattern 63
10.2 The Xi Rule Registry 63
10.3 Compiled Tactics vs. Foundational Semantics 64

11 Bayesian-Network World Models and Compiled BN Inference **65**
11.1 BN World-Model Class 65
11.2 Exact BN Query Bridge 66
11.3 Chain, Fork, and Collider Exactness 66
11.4 Class-Packaged BN Discharge 67
11.5 Worked Proof-Carrying Contraction 67
11.5.1 Probabilistically Guarded Admissibility on Real BN Benchmarks 71
11.5.2 Hetionet / Rephetio: the next flagship benchmark . . 80
11.6 Bridge Theorems: Naive Bayes and k-NN 82
11.7 Factor Graphs: Semantic vs. Operational 83

12 Error, Thresholds, and Transport Discipline **88**
12.1 View Transport Under Query Equivalence 88
12.2 Threshold Semantics and OSLF Bridge 89
12.3 Threshold Transport Boundaries 89
12.4 Categorical Transport Layer 90

V Main Extensions **91**

13 Higher-Order Reduction **92**
13.1 Canonical Higher-Order Layer 92
13.2 FOL Embedding into HOL 93
13.3 HOL as a World-Model Lens 94
13.4 Direct Set-Semantics Grounding of HOL 94
13.5 Higher-Order PLN over HOL and WM 95
13.6 Where Satisfying-Set Reduction Still Fits 96
13.7 Higher-Order Probability: Now Semantically Grounded . . . 96
13.7.1 Hierarchical and Infinite-Order Probabilistic HOL . . 97
13.8 Logical-Induction-Ready Dynamic Belief Infrastructure 99

14 Quantifiers and the Fuzzy/QFM Layer **103**
14.1 Finite-Model Quantifier Semantics 103
14.2 Fuzzy Quantifier Semantics 104
14.3 Scope Note: Stable Finite-Domain Bundle vs. Infinitary Semantics 105

15 Intensional Inheritance **106**
15.1 What Intensional Inheritance Captures 106
15.2 Typed WM Grounding 106
15.3 Solomonoff Bridge 107
15.4 Mixed Semantic Packaging 107

16 Temporal and Causal Inference **109**
16.1 Temporal Operators 109
16.2 Causal Profile 110
16.3 Temporal as Another WM Instance 110

VI Large-Scale Inference, Control, and Boundaries **111**

17 Large-Scale Inference at Theorem Level **112**
17.1 Multideduction and Identifiability 112
17.2 Trail-Free Protocol Dynamics 113
17.3 Pooling and Fusion 113
17.4 Evidence-Conservation and Order-Cost Governance 113
17.5 BRGI and Feasible Sets 115
17.6 Stochastic Experiment Layer 115

18 Inference Control **116**
18.1 Selector Specifications 116
18.2 Coverage and Submodularity 116
18.3 Coverage Surrogate Boundaries 117
18.4 Composed Core Endpoints 117

VII Artifact, Scope, and Relation to PLN **118**

19 What ξPLN Proves **119**
19.1 The Kernel 119
19.2 Views and Evidence 119
19.3 Compiled Inference 120

19.4 Extensions 120
19.5 Large-Scale Inference and Control 120
19.6 Cross-Cutting 120

20 What ξPLN Does Not Prove **122**
20.1 Operational Runtimes 122
20.2 Infinite-Model Quantifiers 122
20.3 Chapter 12 Settlement 123
20.4 Universal Prediction 123
20.5 Global PLN Reorganization 123
20.6 MLN Import and Subsumption 123
20.7 DeFi Hybrid Risk Monitoring 124

21 Conclusion **125**

Appendices **127**

A Advanced-AI Outcomes (WM Scaffold) **127**
A.1 WM Query Form 127
A.2 What the Current Data Actually Supports 128
A.3 Derived Outcome Queries 131
A.4 Current Status 131
A.5 What Can and Cannot Be Said 132
A.6 Interpretation Rule 134

B PPLBench Refresh and Holdout Demo **137**
B.1 What We Tested 137
B.2 Archive Snapshot 138
B.3 Scripts and Model Lane 138
B.4 Easy Refresh Protocol 139
B.5 Refresh Checks 139
B.6 Held-Out Normal-Normal Demo 141
B.7 Interpretation 141
B.8 Proper Stan vs WM-PLN on Normal-Normal 143
B.9 Exact Hook-B Adaptive Normal-Normal 144
B.10 Semantic Noisy-Or Upgrade 146
B.11 Crowd Annotation via Multiclass Message State 150
B.12 Exact Dirichlet-Categorical Lane 152
B.13 Hierarchical `n_schools` via Carried Evidence State 155

C Logistic Regression: Bernoulli-Logit Site Carrier **159**

D Robust Regression: Continuous Carrier Under Heavy-Tailed Noise **163**

E Notation and Symbol Glossary **167**

F Compact Theorem Map **168**

G Code Navigation Guide **170**
G.1 Entry Points 170
G.2 Applied Gaussian Demo Suite 170
G.3 Regression Targets 172
G.4 Build 172

H PLN Book Coverage Map **173**

I Advanced WM Wrappers **175**

Part I

Orientation

Chapter 1

Introduction

Probabilistic Logic Networks, as described by Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka [12], represent uncertain judgments using truth values: a *strength* (probability-like estimate) and a *confidence* (reliability-like quantity derived from sample size). The ambition was generous—to build a general-purpose uncertain reasoner capable of deduction, induction, abduction, and revision, all within a unified algebraic framework.

That ambition produced real insights. The evidence-count representation, the separation of strength from confidence, the recognition that revision must be compositional, and the aspiration to connect logical structure with probabilistic semantics—these were the right instincts. But in the passage from intuition to formalization, something went wrong.

1.1 Where Semantic Drift Came From

The difficulty is not that PLN's rules are wrong. Many of them are correct—under assumptions that were left implicit. The difficulty is that the original presentation mixes three conceptually distinct layers without always marking where one ends and another begins:

1. **The world-model layer.** A probabilistic model—a joint distribution, a Bayesian network, or some structured posterior—that can answer any query by conditioning. This is the *complete* semantics.

2. **The evidence-algebra layer.** An algebraic structure on observation counts (n^+, n^-) with revision as addition, information ordering, and lattice operations. This is a *compositional bookkeeping* layer.

3. **The operational-view layer.** Strength, confidence, weight, interval bounds—these are *extracted summaries* of evidence, useful for thresholding, display, and approximate computation, but not themselves the semantics.

When these layers are conflated, several things go wrong. Operational views get treated as if they *were* the logic: interval bounds are composed with rules designed for point probabilities, confidence heuristics are chained without tracking what information they lost. Link-level inference attempts to compute complete posteriors from local link truth values, ignoring the correlations that live only in the joint state. The result is semantic drift: the system *almost* works, but its correctness depends on assumptions that are neither stated nor checked.

1.2 What the Refoundation Changes

ξPLN is a response to semantic drift. It does three things:

1. It makes the *world-model posterior state* the primary proof object. Inference does not propagate truth values between links; it revises posterior states and answers queries against the revised state.

2. It isolates the *evidence algebra* as a separate, well-defined algebraic layer—a quantale and Heyting frame on observation counts—that serves compositional aggregation but does not pretend to be the complete semantics.

3. It treats all classic PLN rules (deduction, induction, abduction, revision) as *compiled tactics*: derived rewrites that are correct under explicit side conditions within declared model classes, not as primitive laws of the logic.

The result is not a "second PLN." It is the reference semantics that operational PLN rules approximate, under explicit assumptions, within declared model classes. Where the original book's rules are correct, ξPLN proves them correct and states the conditions. Where they are not, ξPLN provides counterexamples and corrected, side-conditioned replacements.

1.3 What This Document Claims

This document is a self-contained mathematical presentation of the PLN refoundation. It does not require the reader to have the PLN book open,

though familiarity with the PLN tradition will enrich the reading. It is *not* a review of the PLN book—that is a separate document [34].

Every theorem stated here has been formalized in Lean 4 with Mathlib. Where a result is conditional on explicit assumptions, those assumptions are stated. Where a result establishes a boundary (a counterexample or no-go theorem), that boundary is stated plainly. Where work remains operational or future, that status is marked.

Trust Base & Scope.

- *Formalized* means that the stated theorem or counterexample is checked by the Lean 4 kernel against imported Mathlib facts.
- *Lean trust base*: the Lean kernel, Mathlib, and the standard elaboration/runtime path needed to check the proof terms.
- *Proved in Lean*: the world-model/evidence calculi, compiled-rule theorems, and counterexamples stated here as mathematical claims.
- *Numerically instantiated in Python*: full-dataset protocol runs (GJP, Gas, Steel, NAB) using the same closed-form update/query laws; these are auditable experiment instantiations, not kernel-checked proofs.
- *Deferred/open*: some runtime infrastructure remains operational rather than theoremic, and the stable finite-domain quantifier bundle is not the whole quantifier story; Chapter 11's arbitrary-domain cleanup is still being integrated into the manuscript narrative.

The mantra throughout:

Probability is what you query; evidence is what you carry.

Chapter 2

Claim Taxonomy and Reading Guide

Before presenting any mathematics, we fix the language of claims. Every result in this document falls into one of four categories:

Core An exact theorem, proved unconditionally (or with only standard mathematical hypotheses like non-negativity or finiteness). These are the bedrock.

Conditional A theorem that holds under explicitly stated structural assumptions—a declared model class, a side condition like d-separation or screening-off, a positivity hypothesis. These are correct *relative to* the stated assumptions. The assumptions are part of the theorem, not an afterthought.

Boundary A counterexample or no-go theorem establishing that some desirable property *fails* in general. These are as important as the positive results: they tell us exactly where fast inference cannot claim completeness, and they motivate the structural assumptions in conditional theorems.

Operational A definition, interface, or specification that exists in the formalization but whose full operational realization depends on runtime infrastructure (e.g., a MeTTa execution engine) not yet available. These are honest placeholders: the mathematical specification is precise, but the implementation story is future work.

Every chapter will mark its results with these labels. The reader should always know whether a claim is proved, conditioned, bounded, or operational.

2.1 How to Read This Document

- **Parts I–III** (Chapters 1–9) give the foundations: the world-model kernel, the evidence algebra, and the truth-value views. A reader wanting the core theory should read these.
- **Part IV** (Chapters 10–12) shows how fast PLN inference arises from the kernel via compiled rewrites over Bayesian-network world models. A reader interested in connecting ξPLN to practical PLN should focus here.
- **Part V** (Chapters 13–16) presents the main extensions: higher-order reduction, quantifiers, intensional inheritance, and temporal/causal inference. Each chapter is largely self-contained.
- **Part VI** (Chapters 17–18) addresses large-scale inference theory and inference control—the most complex and the most honest part of the formalization.
- **Part VII** (Chapters 19–21) takes stock: what is proved, what is not, and what it means.

Chapter 3

Mathematical Desiderata for a Sound PLN

What must any foundation for PLN preserve? We state eight desiderata. Not all of them can be satisfied simultaneously in full generality—the no-go theorem in Chapter 6 shows that local completeness conflicts with compositionality for arbitrary world models. But any proposed foundation should address each desideratum explicitly, either satisfying it or stating clearly why and where it fails.

1. **Posterior-state semantics.** The foundational proof object is a posterior state—a structured distribution over possible worlds, or a sufficient statistic thereof. Truth values for specific queries are obtained by extraction from this state, not by direct algebraic manipulation of link-level numbers.

2. **Revisability.** New evidence can always be incorporated. Revision is the fundamental operation: it combines two posterior states into a single revised state. It must be associative and commutative (evidence order should not matter for the final state).

3. **Queryability.** Any well-formed query—a proposition, a link, a conditional—can be answered by projecting from the posterior state. The query language is separate from the evidence representation.

4. **Evidence compositionality.** Evidence extraction commutes with revision:

$$\text{evidence}(W_1 + W_2, q) = \text{evidence}(W_1, q) \oplus \text{evidence}(W_2, q).$$

This is the algebraic law that makes revision at the world-model level compatible with revision at the evidence level.

5. **Tractable compiled fragments.** For restricted world-model classes (e.g., Bayesian networks with bounded treewidth), query answering is tractable. Classic PLN rules arise as compiled rewrites within such classes.

6. **Explicit side conditions.** Every compiled rewrite carries its assumptions as explicit hypotheses: d-separation, screening-off, positivity, finiteness. There are no "silent assumptions."

7. **Extracted truth-value views.** Strength, confidence, interval bounds, and distributional summaries are derived views, not primitive objects. They are indispensable for operational use, but they are not the semantics.

8. **Theorem-level boundaries.** Where completeness or compositionality fails, there is a formal counterexample or no-go theorem stating what cannot be done. These boundaries guide the design of tractable fragments.

ξPLN satisfies all eight desiderata, with the caveat that tractability (desideratum 5) always comes at the cost of restricting the model class, and the no-go theorem (Chapter 6) establishes that local completeness is inherently impossible for unrestricted world models.

Part II

The Semantic Kernel

Chapter 4

Queries, Judgments, and World Models

The first technical move in ξPLN is to separate what we ask (*queries*) from what we know (*posterior states*). This separation is the foundation of everything that follows.

4.1 The Query Language

In PLN, one may ask: "What is the probability that A is true?" or "What is the probability that B is true given A?" These are queries against an uncertain knowledge base. We represent them as a simple inductive type:

```
inductive PLNQuery (Atom : Type*) where
  | prop  : Atom -> PLNQuery Atom
  | link  : Atom -> Atom -> PLNQuery Atom
  | linkCond : List Atom -> Atom -> PLNQuery Atom
```

A `prop` query asks about a marginal event. A `link` query asks about the conditional relationship between two atoms ($P(B \mid A)$, roughly). A `linkCond` query asks about a conditional given multiple conditioning atoms. This is the minimal query language needed to express the core PLN inference patterns.

The query language is deliberately simple. It is not a full predicate logic; it is the interface through which the world model is interrogated. Richer query languages (first-order, higher-order, temporal) are built on top of this core in later chapters.

4.2 World Models

A *world model* is anything that can answer queries. In Lean, we capture this as a typeclass:

```
class WorldModel (State : Type*) (Query : Type*)
    [EvidenceType State] where
  evidence : State -> Query -> Evidence
  evidence_add : forall W1 W2 q,
    evidence (W1 + W2) q = evidence W1 q + evidence W2 q
```

Here `State` is the type of posterior states, `Query` is the query type, and `EvidenceType` equips the state with the structure of an additive commutative monoid—meaning revision (+) is defined and behaves correctly.

The single method `evidence` extracts an `Evidence` value (a pair of counts (n^+, n^-)) for any query from any state. The single law `evidence_add` says that extraction commutes with revision. This is desideratum 4 from Chapter 3: evidence compositionality.

Notation. We write $\mathrm{ev}(W, q)$ for $\texttt{evidence}(W, q)$, and $W_1 + W_2$ for the revised state. The evidence algebra on the extracted values uses $\oplus$ for pointwise addition of counts.

See `Mettapedia/Logic/PLNWorldModel.lean`.

4.3 World Models with Side Conditions

For compiled inference, we often need structural assumptions about the world model: a DAG structure for a Bayesian network, conditional independence conditions, positivity hypotheses. We capture this with a *sigma-bundled world model*:

```
structure WorldModelSigma (State : Type*) (Query : Type*)
    [EvidenceType State] where
  wm : WorldModel State Query
  sigma : Prop  -- side-condition context
```

The Σ (side-condition context) is a proposition that the world model satisfies additional structural properties. Every compiled rewrite carries its Σ explicitly—there are no hidden assumptions.

4.4 The Solomonoff Connection

The νPLN framework of Nil Geisweiller [10] grounds PLN in Solomonoff Universal Induction. The key ideas fold naturally into the world-model framework:

- The *global probability space* $(\Omega, \mathcal{F}, \nu)$ corresponds to a world-model state where Ω is the set of possible worlds, each containing a probabilistic predicate $\hat{\mu} : \mathcal{D} \to \Omega \to \text{Bool}$ and its complete evaluation history.
- The *universal distribution* ν defines a prior over possible generators $\hat{\mu} \in \mathcal{L}$, where $\mathcal{L}$ is a probabilistic programming language. Bayesian updating with observed evidence produces the posterior.
- The *observable predicate* μ (the true generator) lives in the true world ω. An observer has access only to a finite subset of evaluations—the evidence.
- *Concepts and predicates* are isomorphic via satisfying sets and indicator functions. The inheritance relationship $A \Rightarrow B$ on the concept side corresponds to implication on the predicate side. We work on the predicate side throughout.

The crucial insight from νPLN is that de Finetti's representation theorem [7] justifies evidence counts as sufficient statistics. For an infinite exchangeable sequence of binary observations, the counts (n^+, n^-) contain all the information needed for Bayesian updating. When we assume a Beta prior $\text{Beta}(\alpha, \beta)$, the posterior after observing n^+ positive and n^- negative outcomes is $\text{Beta}(\alpha + n^+, \beta + n^-)$.

This is why PLN's evidence representation (n^+, n^-) is not merely a convenient bookkeeping device—it is the *sufficient statistic* for the posterior under the exchangeability assumption. Chapter 7 develops this algebraic structure in detail.

Chapter 5

Reference Complete Semantics: Posterior-State Calculus

The complete core calculus is intentionally simple. It is the calculus of revising posterior states plus a query interface. Everything else in ξPLN—evidence algebras, truth-value views, compiled rules, tractable fragments—is built on top of this core.

5.1 The Reference Instance: Dirichlet over Worlds

For finite propositional scope (atoms indexed by $\mathrm{Fin}(n)$), we instantiate the `WorldModel` interface with a Dirichlet world-table:

$$\texttt{JointEvidence}(n) := \mathrm{Fin}(2^n) \to \mathbb{R}^{\infty}_{\geq 0}$$

interpreted as Dirichlet pseudo-counts over the 2^n complete worlds. Each coordinate $E(w)$ records how much evidence supports world w.

Evidence extraction works by summation:

- $\mathrm{ev}(E, \texttt{prop}(A))$: sum counts where atom A is true (positive) vs. false (negative).
- $\mathrm{ev}(E, \texttt{link}(A, B))$: sum counts where A is true and B is true vs. A is true and B is false.
- $\mathrm{ev}(E, \texttt{linkCond}([A_1, \ldots, A_k], B))$: sum counts where all A_i hold and B holds vs. all A_i hold and B fails.

Revision is pointwise addition of pseudo-counts. The Lean prototype proves that this satisfies the world-model interface, including the critical compositionality law:

$$\mathrm{ev}(E_1 + E_2, q) = \mathrm{ev}(E_1, q) \oplus \mathrm{ev}(E_2, q)$$

for all queries q.

See `Mettapedia/Logic/PLNJointEvidence.lean`.

5.2 The Sequent-Calculus View

We present ξPLN as a sequent-style system with two contexts:

- An *evidence context* Γ, a finite multiset of posterior fragments (elements of the state type).
- A *side-condition context* Σ, containing structural assumptions about the chosen world-model class (e.g., a BN DAG, positivity, d-separation facts).

5.2.1 Judgments

Two judgment forms suffice:

1. **World-model judgment:** $\Sigma; \Gamma \vdash_{\mathrm{wm}} W$—from the evidence pieces in Γ, we construct the revised posterior state W.
2. **Query judgment:** $\Sigma; \Gamma \vdash q \Downarrow e$—querying the revised posterior derived from Γ yields evidence e for query q.

When `State` is an additive commutative monoid, the world-model judgment is deterministic: $W := \sum \Gamma$. The query judgment then extracts via the world-model interface: $e := \mathrm{ev}(W, q)$.

5.2.2 Core Rules

The complete core calculus consists of four rules:

$$\frac{}{\Sigma; \varnothing \vdash_{\mathrm{wm}} 0}\ (\text{WM-Unit}) \qquad \frac{}{\Sigma; \{W\} \vdash_{\mathrm{wm}} W}\ (\text{WM-Ev})$$

$$\frac{\Sigma; \Gamma \vdash_{\mathrm{wm}} W \quad \Sigma; \Delta \vdash_{\mathrm{wm}} W'}{\Sigma; \Gamma \uplus \Delta \vdash_{\mathrm{wm}} (W + W')}\ (\text{WM-Rev})$$

$$\frac{\Sigma; \Gamma \vdash_{\mathrm{wm}} W}{\Sigma; \Gamma \vdash q \Downarrow \mathrm{ev}(W, q)} \text{ (Q-Extract)}$$

The key algebraic law—evidence compositionality—makes the following *query-revision* rule admissible:

$$\frac{\Sigma; \Gamma \vdash q \Downarrow e \quad \Sigma; \Delta \vdash q \Downarrow e'}{\Sigma; \Gamma \uplus \Delta \vdash q \Downarrow (e \oplus e')} \text{ (Q-Rev)}$$

Links are queries. In this view, a PLN "link" $A \Rightarrow B$ is not an object-level connective; it is a query `link(A, B)` against the revised world model. The calculus does not propagate links directly; it revises world models and answers link queries.

5.3 Probability Views

From `Evidence` we obtain posterior-mean probabilities:

$$P(A) = \frac{\#(A)}{\#(\top)}, \qquad P(B \mid A) = \frac{\#(A \wedge B)}{\#(A)}$$

where $\#(\cdot)$ denotes world-count sums in `JointEvidence`. The Lean file `Mettapedia/Logic/PLNJointEvidenceProbability.lean` proves the exact ratio forms for these views.

5.4 Derived Link Calculus: Classic PLN Rules as Admissible Rewrites

Classic PLN rules become *admissible query rewrites* relative to a world-model class with explicit side conditions Σ.

5.4.1 The Deduction Formula

The PLN deduction rule computes $P(C \mid A)$ by total probability:

$$P(C \mid A) = P(C \mid A, B)\, P(B \mid A) + P(C \mid A, \neg B)\, P(\neg B \mid A) \tag{5.1}$$

Under the *screening-off* conditions $P(C \mid A, B) = P(C \mid B)$ and $P(C \mid A, \neg B) = P(C \mid \neg B)$, this simplifies to:

$$P(C \mid A) = P(C \mid B)\, P(B \mid A) + P(C \mid \neg B)\, (1 - P(B \mid A)) \tag{5.2}$$

Writing $s_{AB} = P(B \mid A)$, $s_{BC} = P(C \mid B)$, $s_B = P(B)$, $s_C = P(C)$, and eliminating $P(C \mid \neg B)$ via $s_C = s_B \cdot s_{BC} + (1 - s_B) \cdot P(C \mid \neg B)$, we obtain:

Definition 5.1 (PLN deduction strength)**.**

$$\mathrm{plnDed}(s_{AB}, s_{BC}, s_B, s_C) \;:=\; s_{AB} \cdot s_{BC} + (1 - s_{AB}) \cdot \frac{s_C - s_B \cdot s_{BC}}{1 - s_B}$$

defined when $0 < s_B < 1$.

Theorem 5.2 (Chain BN deduction exactness (Conditional))**.** *Let W be a chain BN world model $A \to B \to C$. Under the explicit hypotheses:*

(i) $0 < \mathsf{strength}(W, \mathit{prop}(B)) < 1$ *(positivity),*

(ii) $\Sigma \vdash C \perp\!\!\!\perp A \mid B$ *and* $C \perp\!\!\!\perp A \mid \neg B$ *(screening-off),*

we have:

$$\mathsf{strength}(W, \mathit{link}(A, C)) = \mathrm{plnDed}\big(\mathsf{strength}(W, \mathit{link}(A, B)),\ \mathsf{strength}(W, \mathit{link}(B, C)),\ \mathsf{strength}(W, \mathit{prop}(B$$

In Lean:

```
theorem chainBN_plnDeductionStrength_exact
    (hpos : 0 < queryStrength W (PLNQuery.prop B))
    (hlt  : queryStrength W (PLNQuery.prop B) < 1)
    (hso  : screeningOff W A B C) :
    queryStrength W (PLNQuery.link A C) =
    plnDeductionStrength
      (queryStrength W (PLNQuery.link A B))
      (queryStrength W (PLNQuery.link B C))
      (queryStrength W (PLNQuery.prop B))
      (queryStrength W (PLNQuery.prop C))
```

The analogous result holds for fork BNs $A \leftarrow B \to C$ (`forkBN_plnDeductionStrength_exact`). See `Mettapedia/Logic/PLNBayesNetFastRules.lean`.

5.4.2 Induction and Abduction

Induction and abduction arise from the same total-probability structure but with different known/unknown roles:

- **Induction:** given $P(A \mid B)$ and $P(A \mid C)$, estimate $P(C \mid B)$. This inverts the direction of the conditional.
- **Abduction:** given $P(B \mid A)$ and $P(B \mid C)$, estimate $P(A \mid C)$. This inverts the conditioning.

Both require their own screening-off conditions, which are formally stated in the Lean proofs.

See `Mettapedia/Logic/PLNDerivation.lean`.

Chapter 6

The Local-Completeness No-Go Theorem

One might hope for a system that takes local per-link evidence (`Evidence` for A, B, C, $A \Rightarrow B$, $B \Rightarrow C$) and computes complete evidence for $A \Rightarrow C$. ξPLN asserts that this is impossible in general.

Theorem 6.1 (No local complete deduction rule (Boundary))**.** *There is no function*

$$f : \texttt{Evidence}^5 \to \texttt{Evidence}$$

that, for all joint evidence states E, computes the exact link evidence for $A \Rightarrow C$ from only the local premises $\mathrm{ev}(E, A)$, $\mathrm{ev}(E, B)$, $\mathrm{ev}(E, C)$, $\mathrm{ev}(E, A \Rightarrow B)$, $\mathrm{ev}(E, B \Rightarrow C)$.

Proof sketch. For three atoms A, B, C, there are $2^3 = 8$ possible worlds. We construct two joint evidence states $E_1, E_2 : \mathrm{Fin}(8) \to \mathbb{R}_{\geq 0}$ that agree on all five marginal evidence values:

$$\mathrm{ev}(E_1, A) = \mathrm{ev}(E_2, A), \quad \mathrm{ev}(E_1, B) = \mathrm{ev}(E_2, B), \quad \mathrm{ev}(E_1, C) = \mathrm{ev}(E_2, C),$$

$$\mathrm{ev}(E_1, A \Rightarrow B) = \mathrm{ev}(E_2, A \Rightarrow B), \quad \mathrm{ev}(E_1, B \Rightarrow C) = \mathrm{ev}(E_2, B \Rightarrow C),$$

but disagree on the conclusion:

$$\mathrm{ev}(E_1, A \Rightarrow C) \neq \mathrm{ev}(E_2, A \Rightarrow C).$$

The key: world entries corresponding to $(A = 1, B = 0, C = 1)$ and $(A = 1, B = 1, C = 0)$ can be traded off against each other while preserving all five marginals. But this trade changes the $A \Rightarrow C$ evidence because it redistributes mass among worlds where A is true.

Since the five marginals cannot distinguish E_1 from E_2, no function $f : \texttt{Evidence}^5 \to \texttt{Evidence}$ can compute $\mathrm{ev}(E, A \Rightarrow C)$ correctly for both states. □

See `Mettapedia/Logic/PLNJointEvidenceNoGo.lean`.

6.1 Interpretation

This theorem is the formal core of "you must carry correlations." It does *not* make fast PLN useless. It tells us exactly what fast PLN *cannot* claim: completeness for arbitrary world models without structural assumptions.

The constructive response is to:

1. **Restrict the model class.** For Bayesian networks with explicit DAG structure, the factorization makes local computation exact (Chapter 11).

2. **State the assumptions.** Every compiled rule carries its Σ context, so the user knows exactly when the fast rule applies.

3. **Provide corrected alternatives.** Where an original PLN formula required an unstated assumption, provide the corrected side-conditioned version alongside the boundary theorem.

Part III

Evidence and Truth-Value Views

Chapter 7

Evidence Algebra

Evidence is the currency of PLN. Before any inference rule fires, before any truth value is computed, evidence must be collected, combined, and compared. This chapter develops the algebraic structure of evidence.

7.1 Evidence as Counts

The basic evidence type in PLN is a pair of non-negative counts:

$$\texttt{Evidence} := (n^+, n^-) \in \mathbb{N} \times \mathbb{N}$$

where n^+ counts positive observations and n^- counts negative observations. (In the Lean formalization, we sometimes work over $\mathbb{R}_{\geq 0}$ or $\mathbb{R}^{\infty}_{\geq 0}$ for compatibility with measure-theoretic constructions.)

The total sample size is $n = n^+ + n^-$. The *strength* view (posterior mean) is $s = n^+/n$ when $n > 0$. But we must resist the temptation to conflate the evidence (n^+, n^-) with its strength view s—evidence carries strictly more information.

See `Mettapedia/Logic/EvidenceClass.lean`.

7.2 Revision as Addition

The fundamental operation on evidence is *revision*: combining two bodies of evidence into one. Revision is componentwise addition:

$$(n_1^+, n_1^-) \oplus (n_2^+, n_2^-) = (n_1^+ + n_2^+, n_1^- + n_2^-)$$

This is associative, commutative, and has unit $(0, 0)$—the evidence of having observed nothing.

The justification for additive revision comes from de Finetti's representation theorem. For exchangeable binary sequences, the counts are sufficient statistics for the posterior. Adding counts corresponds to Bayesian conjugate updating with a Beta prior:

$$\text{Beta}(\alpha + n_1^+, \beta + n_1^-) \xrightarrow{+\,(n_2^+, n_2^-)} \text{Beta}(\alpha + n_1^+ + n_2^+, \beta + n_1^- + n_2^-)$$

This is why evidence revision is *addition*: it is the algebraic shadow of Bayesian conjugate updating.

7.3 Information Ordering

Evidence values are partially ordered by information content:

$$(n_1^+, n_1^-) \leq (n_2^+, n_2^-) \iff n_1^+ \leq n_2^+ \ \wedge \ n_1^- \leq n_2^-$$

More evidence is always at least as informative. This ordering makes `Evidence` a lattice with meet (componentwise min) and join (componentwise max).

7.4 Quantale Structure

Beyond the additive monoid and lattice, `Evidence` carries a *quantale* structure. A quantale is a complete lattice equipped with a monoidal operation $\otimes$ that distributes over arbitrary joins:

$$a \otimes \Big(\bigvee_{i \in I} b_i \Big) = \bigvee_{i \in I} (a \otimes b_i)$$

For $\texttt{Evidence} = (n^+, n^-)$, the tensor is defined as:

$$(a^+, a^-) \otimes (b^+, b^-) = (a^+ \cdot b^+,\ a^+ \cdot b^- + a^- \cdot b^+)$$

with unit $(1, 0)$. This corresponds to the compositional combination of dependent evidence streams: observing a and then observing b conditional on a being positive.

Theorem 7.1 (Quantale transitivity (Core)). *For evidence values e_{AB}, e_{BC} representing $A \Rightarrow B$ and $B \Rightarrow C$ respectively:*

$$e_{AB} \otimes e_{BC} \ \leq \ e_{AC}$$

in the information ordering. Equality holds under the screening-off conditions.

This inequality is the algebraic shadow of the deduction rule: the tensor product of premise evidence provides a lower bound on the conclusion evidence.

See `Mettapedia/Logic/EvidenceQuantale.lean`.

7.5 Heyting Structure

`Evidence` also forms a Heyting algebra with an explicit pseudo-complement:

Definition 7.2 (Heyting implication on `Evidence` (Core))**.** The relative pseudo-complement is defined componentwise:

$$(a^+, a^-) \Rightarrow (b^+, b^-) = \left(\begin{cases} \top & \text{if } a^+ \leq b^+ \\ b^+ & \text{otherwise} \end{cases}, \begin{cases} \top & \text{if } a^- \leq b^- \\ b^- & \text{otherwise} \end{cases} \right)$$

satisfying the residuation law: $a \leq b \Rightarrow c \iff a \wedge b \leq c$.

The Heyting complement (negation) is $\neg a = a \Rightarrow \bot = a \Rightarrow (0, 0)$.

This is important because the evidence algebra supports *intuitionistic* logical operations. In particular, the law of excluded middle fails: $a \vee \neg a \neq \top$ in general, because there may be no evidence at all.

A key formalized fact—the "totality gate"—establishes that `Evidence` has incomparable elements (e.g., $(3, 0)$ and $(0, 3)$) and admits no faithful order-embedding into the reals. No single real-valued function captures all evidence information.

See `Mettapedia/Logic/HeytingValuationOnEvidence.lean` and `Mettapedia/Logic/PLN\_KS\_Bridge.lean`.

7.6 What Evidence Algebra Is Not

Evidence algebra is *not* the complete probabilistic semantics. It is a compositional bookkeeping layer. The world-model posterior (Chapter 5) carries strictly more information than the extracted per-query evidence values. The no-go theorem (Chapter 6) makes this precise: per-link evidence cannot reconstruct the joint state.

Evidence algebra is to probability theory what bookkeeping is to economics: indispensable for accounting, but not a substitute for the underlying model of the economy.

Chapter 8

Truth-Value Views: Strength, Confidence, Intervals

Operationally, PLN works with compact summaries of evidence: a strength, a confidence, sometimes an interval. These are *views*— projections from the evidence algebra into simpler spaces that are useful for thresholding, display, and approximate computation.

8.1 Simple Truth Values

A *simple truth value* (STV) is a pair $\langle s, c \rangle$ where:

- $s = n^+/(n^+ + n^-)$ is the *strength*—the posterior mean under a uniform prior, or equivalently the maximum-likelihood estimate of the underlying probability.
- $c = n/(n+k)$ is the *confidence*—a monotone function of the total sample size $n = n^+ + n^-$, parameterized by a lookahead $k > 0$. Confidence ranges from 0 (no evidence) to approaching 1 (overwhelming evidence).

The STV is the most commonly used truth-value representation in PLN practice. It is compact and intuitive. But it is a *lossy* view: two different evidence values can produce the same STV (when they have the same ratio n^+/n and the same total n), and the STV discards information about the shape of the posterior distribution beyond its mean.

8.2 Confidence as a View

Confidence deserves special attention. In the original PLN presentation, confidence often appears as a standalone quantity that can be manipulated independently of strength. In ξPLN, confidence is strictly a derived view:

$$\mathsf{confidence}(n^+, n^-) = \frac{n^+ + n^-}{n^+ + n^- + k}$$

The parameter k controls how much evidence is needed before the system becomes "confident." For $k = 1$, a single observation gives confidence 0.5; for $k = 10$, ten observations give confidence 0.5.

The key property: confidence is monotone in the total evidence count. More evidence always means higher (or equal) confidence.

8.3 Indefinite Truth Values

An *indefinite truth value* (ITV) provides interval bounds on the strength:

$$\text{ITV} = [s_L,\ s_U,\ b,\ w]$$

where s_L and s_U are lower and upper bounds, b is a confidence-like parameter, and $w = s_U - s_L$ is the interval width.

There are at least three distinct semantic interpretations of such intervals, and they *must not be conflated*:

1. **Bayesian credible intervals.** An interval $[s_L, s_U]$ such that the posterior probability $\Pr(s_L \leq p \leq s_U \mid \text{data})$ exceeds some threshold. These depend on the prior.

2. **Walley-style predictive intervals (IDM).** Intervals derived from the Imprecise Dirichlet Model [32, 33], which provide robust predictions without committing to a single prior. These are prior-free but give wider intervals.

3. **Frequentist confidence intervals.** Intervals constructed so that the true parameter lies within them with specified coverage probability over repeated sampling.

The original PLN book sometimes uses the notation $[L, U]$ without specifying which semantics is intended. ξPLN insists on explicit selector choice:

the user declares which interval semantics they want, and the system computes accordingly.

See `Mettapedia/Logic/PLNIndefiniteTruth.lean` and `Mettapedia/Logic/PLNWorldModelITV.lean`.

8.4 Why Views Are Not Proof Objects

Views are useful. They are indispensable for practical inference. But they are *not* the right objects to pass between inference steps. The reason: views are lossy projections. When you chain two inferences using only the STV of the intermediate result, you have already lost information about the shape of the posterior, the correlation structure, and the raw evidence counts.

The ξPLN discipline is: carry evidence (or world-model states) between inference steps; extract views only at the end, for display or decision-making. The slogan again:

> *Probability is what you query; evidence is what you carry.*

Chapter 9

Distributional and Interval Semantics

Beyond point estimates and intervals, PLN supports a richer *distributional* layer where the truth value is an entire posterior distribution over possible probability values.

9.1 Beta Distributions and Evidence

For a single binary predicate observed n^+ times true and n^- times false, the posterior under a Beta prior $\mathrm{Beta}(\alpha, \beta)$ is:

$$p \mid \text{data} \sim \mathrm{Beta}(\alpha + n^+, \beta + n^-)$$

Evidence counts (n^+, n^-) are the natural parameters of this posterior. Revision (adding counts) corresponds to Bayesian conjugate updating. The posterior mean is

$$\mathbb{E}[p] = \frac{\alpha + n^+}{\alpha + \beta + n^+ + n^-}$$

which, for a uniform prior ($\alpha = \beta = 1$), gives the strength

$$s = \frac{1 + n^+}{2 + n^+ + n^-}$$

This is the Laplace-smoothed estimate.

See `Mettapedia/Logic/EvidenceBeta.lean`.

9.2 Dirichlet Distributions for Multi-Valued Domains

For predicates with more than two outcomes, the Beta generalizes to the Dirichlet distribution. Evidence becomes a vector of counts, and conjugate updating extends componentwise.

See `Mettapedia/Logic/EvidenceDirichlet.lean`.

9.3 Normal-Gamma Evidence for Continuous Domains

The Beta and Dirichlet families handle discrete observations. For continuous observations—sensor readings, measurements, real-valued features—the appropriate conjugate family is the *Normal-Gamma* [4].

For observations $x_1, \ldots, x_n \sim \mathcal{N}(\mu, 1/\tau)$ with unknown mean μ and precision τ, the evidence is the sufficient statistic $(n, \sum x_i, \sum x_i^2)$. The prior is Normal-Gamma$(\mu_0, \kappa_0, \alpha_0, \beta_0)$ and the posterior is Normal-Gamma with updated parameters:

$$\kappa_n = \kappa_0 + n, \qquad \mu_n = \frac{\kappa_0 \mu_0 + \sum x_i}{\kappa_n}, \qquad \alpha_n = \alpha_0 + \frac{n}{2}$$

$$\beta_n = \beta_0 + \tfrac{1}{2}\big(\sum x_i^2 - (\sum x_i)^2/n\big) + \tfrac{\kappa_0\, n\, (\bar{x} - \mu_0)^2}{2\kappa_n}$$

Revision is again componentwise addition of sufficient statistics:

$$(n_1, s_1, q_1) \oplus (n_2, s_2, q_2) = (n_1 + n_2,\ s_1 + s_2,\ q_1 + q_2)$$

and the key compositionality theorem is:

Theorem 9.1 (Sleep Consolidation (Core)). *$posterior(\pi, e_1 \oplus e_2) = posterior(posterior(\pi, e_1), e_2)$ for any prior π and realizable evidence e_1, e_2.*

This states that batch replay of deferred observations yields the same posterior as sequential Bayesian updating—the mathematical foundation for "sleep as evidence consolidation."

The confidence analog carries over directly: $c = n/(n + \kappa)$, the same formula as binary PLN. The pattern is uniform across all conjugate families:

Domain	Evidence	Prior family
Binary	(n^+, n^-)	Beta(α, β)
Categorical	count vector	Dirichlet(α)
Continuous	$(n, \sum x_i, \sum x_i^2)$	Normal-Gamma$(\mu_0, \kappa_0, \alpha_0, \beta_0)$

See `Mettapedia/Logic/EvidenceNormalGamma.lean`.

Two-layer architecture. Each demo below has two layers. *Lean* proves the posterior formulas correct and establishes structural properties (consolidation, independence, monotonicity) — all with 0 `sorry`. A companion Python script (`scripts/wm_pln_posterior.py`) instantiates the proved formulas on each fixture to produce concrete numerical results: posterior means, 95% credible intervals, exceedance probabilities, and classification scores. The Student-t predictive CDF fills the role of the abstract `ExceedanceSpec` that Lean leaves parametric (Mathlib lacks a Normal CDF). We also compare with scikit-learn's `GaussianNB`; the two agree on clearly separated regimes (e.g. K_Scratch at $x = 3.6$) but differ at ambiguous boundary points (e.g. $x = 2.0$), since GaussianNB uses ML point estimates while the Normal-Gamma posterior predictive integrates over parameter uncertainty. The added value of the WM-PLN formulation is: (a) the Lean proofs guarantee formula correctness, (b) the framework generalizes to hybrid discrete/continuous states, and (c) sleep consolidation is *proved*, not assumed. *Note:* the demos below use small, traceable fixture slices (5–15 data points per class) drawn from real datasets—not full benchmark-scale evaluations. Their purpose is to serve as theorem-backed empirical canaries that exercise the proved formulas on genuine data; larger-scale validation on the full Steel dataset appears in Section 9.3.5.

PLN-to-WM concept mapping. Table 9.1 shows how each core PLN concept maps to its WM-PLN formalization. The left column uses the terminology of the PLN book [12]; the right column gives the Lean module or theorem that formalizes the concept.

Measure-theoretic grounding. The Normal-Gamma family is a joint prior *measure* on $(\mu, \tau) \in \mathbb{R} \times \mathbb{R}_+$, where $\tau = 1/\sigma^2$ is the precision. The posterior update proved in `EvidenceNormalGamma.lean` is the algebraic form of Bayes' theorem applied to this measure: given n observations $x_1, \ldots, x_n$ drawn from $\mathcal{N}(\mu, \tau^{-1})$, the likelihood factors through the sufficient statistics $(n, \sum x_i, \sum x_i^2)$ by the Fisher–Neyman factorization theorem. Conju-

PLN Concept	PLN Book	WM-PLN	Lean Reference
Evidence count	(n^+, n^-)	$(n, \Sigma x_i, \Sigma x_i^2)$	`EvidenceNormalGamma`
Strength	$s = \frac{n^+}{n^+ + n^-}$	Posterior mean μ_n	`posterior_mu_eq_of_reali`
Confidence	$c = \frac{n}{n+k}$	$c = \frac{n}{n+\kappa_0}$ (monotone)	`confidence_monotone`
Revision ($\oplus$)	Componentwise add	`hplus` on suff. stats	`NormalGammaEvidence.hplu`
Sleep consolidation	—	$\pi(e_1 \oplus e_2) = \pi(\pi(e_1), e_2)$	`posterior_hplus_of_reali`
Exceedance query	—	$P(X_{\text{new}} > c)$	`ExceedanceSpec`
Source attribution	Implicit	Explicit source counts	`PLNWidgetFactoryDemo`
De Finetti	Informal	Finite exchangeable model	`DeFinetti.lean`

Table 9.1: PLN concept → WM-PLN formalization.

gacy ensures the posterior remains Normal-Gamma, so the update reduces to arithmetic on the four hyperparameters $(\mu_0, \kappa_0, \alpha_0, \beta_0)$.

The Lean formalization proves this arithmetic *exactly*: the closed-form expressions for μ_n, κ_n, α_n, β_n are established in `posterior_mu_eq_of_realizable` and `posterior_beta_eq_of_realizable`, and per-demo bridge theorems instantiate them on each demo's concrete fixture data. The integral form of the posterior (involving the Gamma density $\mathrm{Ga}(\alpha_0, \beta_0)$) is standard mathematics [4] (Ch. 5); Mathlib provides `gaussianPDFReal` and Lebesgue integration, but the Gamma density is not yet available in Mathlib, so the integral derivation is not formalized today.

Experiment-family view (EXPO / Hyperseed). The Gaussian suite is best read as a family of *experiment types*, not as a single benchmark leaderboard. The empirical spine has two *co-flagships*: the Gas Sensor study as a *drift/replay governance* story, and the GJP forecasting study as a *source-reliability / recalibration* story. Steel is a *supporting same-model-class validation*: it asks whether the formally verified WM-PLN conjugate classifier behaves competitively against a plain GaussianNB baseline and how confidence can be used to abstain. NAB is an *honest negative boundary / case study*: it exercises a real scoring pipeline with formally verified Kalman updates, but it is not presented as a full-corpus leaderboard result. This division is deliberate. WM-PLN's distinctive contribution is not "better tabular accuracy at all costs," but queryable uncertainty, replay semantics, soft evidence, and explicit reasoning about which experiment channel is more informative.

Comparator taxonomy. To keep the empirical claims honest, the manuscript uses three *different kinds of comparator*, and they should not be conflated.

1. **WM-PLN models.** These are the evidence/world-model systems formalized in Lean and evaluated in the main applied chapters.

2. **Proper probabilistic-programming baselines.** When the book refers to *Stan* in the PPLBench appendix, it means the standard PPLBench Stan backend under `pplbench/ppls/stan/`, not an ad hoc one-off script. The same family includes PyMC, JAGS, NumPyro, and BeanMachine where available. Appendix B.8 now includes a direct `normal\_normal` Stan ↔ WM-PLN run on that proper PPLBench surface.

3. **External statistical/ML baselines.** `GaussianNB`, `XGBoost`, and Windowed Gaussian are not WM-PLN variants; they are separate classical baselines used to show where WM-style semantics help, where they do not, and when WM is acting as a governance/support layer rather than a replacement predictor.

Family	Representative tools	Meaning in this book
WM-PLN	Lean-backed evidence/world-models	Primary object of study: typed evidence accumulation, replay, confidence, forgetting, and governance semantics.
Proper PPL baselines	Stan, PyMC, JAGS, NumPyro, BeanMachine	Used to compare against mainstream probabilistic inference systems on shared benchmark models in PPLBench. "Stan" refers to the standard PPLBench Stan backend, not a scratch prototype.
Classical statistical / ML baselines	`GaussianNB`, `XGBoost`, Windowed Gaussian	Used in the applied studies to distinguish same-model-class wins (`GaussianNB`), stronger discriminative baselines (`XGBoost`), and standard anomaly controls (Windowed Gaussian).

Table 9.2: Comparator families used in the empirical sections.

9.3.1 Applied Example: Hybrid Sensor Monitoring

The *broken sensor demo* exercises continuous world models end-to-end. A sensor monitoring system combines:

- **Continuous temperature evidence** modeled by Normal-Gamma sufficient statistics (5 normal readings, 5 elevated readings).

Dataset	WM-PLN role	Key metric	Main finding
Steel (3-class, 27 feat.)	supporting same-model-class validation / abstention	AURC	WM-PLN beats GaussianNB (0.0346 vs 0.1061) but not XGBoost (0.0126), so the selective-prediction win is strongest within the conjugate Bayesian model class.
Gas (3-gas, 128 feat.)	co-flagship: drift trigger / replay governance	sample-weighted accuracy under label budget	Frozen XGBoost gives 67.9%; a cheap confidence trigger reaches 79.9% with one retrain and 18.8% labels.
Gas (6-gas, 128 feat.)	co-flagship: drift trigger / replay governance	sample-weighted accuracy under label budget	Frozen XGBoost gives 46.7%; XGBoost confidence is the best cheap trigger (64.5% at 23.9% labels) while WM/XGBoost agreement is the strongest high-accuracy trigger (67.0% at 86.8% labels).
GJP forecasting	co-flagship: source reliability / recalibration	Brier, log score, AURC, sleep invariance	WM reliability weighting improves over stronger top-k log/logit pooling baselines on year-4 Brier/log and selective AURC while matching accuracy; a WM-agreement gate still improves the strong top-k pool on the multi-option slice; nightly batch consolidation matches online updates to machine precision, and forgetting early years improves year-4 calibration.
NAB	negative boundary / case study	official corpus raw score	The current 1D Kalman residual detector is not competitive on the full corpus (-116.0 vs. Windowed Gaussian -24.0), so NAB remains a narrow protocol-semantics case study.

Table 9.3: Role split across the current real-data study slices. The empirical spine has two co-flagships (Gas and GJP), one supporting validation (Steel), and one honest negative boundary (NAB).

- **Boolean cooling-failure evidence** modeled by binary counts (2 failures in 100 checks).

The hybrid state is a product type; revision operates componentwise. Queries are answered by extracting binary `Evidence` from each component: cooling failure is read directly; temperature exceedance $P(X > c)$ is handled via an abstract *exceedance specification*—a function satisfying monotonicity properties—since Mathlib lacks a Normal CDF.

Key results (all zero `sorry`):

1. Sleep consolidation (Theorem 9.1): morning + night batch = sequential update.
2. Confidence monotonicity: more observations $\Rightarrow$ higher $c = n/(n + \kappa)$.
3. Exceedance ordering: $P(X > c_{\text{crit}}) \leq P(X > c_{\text{warn}})$ for $c_{\text{crit}} > c_{\text{warn}}$.
4. Independence: temperature and cooling-failure evidence components do not interfere.

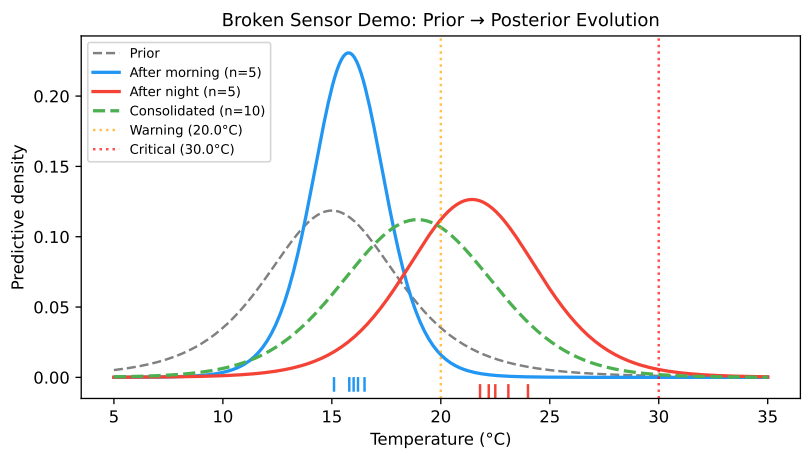


Figure 9.1: Broken sensor demo: prior predictive (gray dashed), posterior after morning shift (blue, concentrated around 16°C), after night shift (red, shifted to 22°C), and consolidated posterior (green dashed, between the two). Data tick marks on the x-axis.

Numerical results. With prior $\text{NG}(15, 1, 2, 10)$, the morning shift (5 readings, mean 15.92) yields posterior $\mu_n = 15.77$, 95% CI $[11.96, 19.57]$, $P(X >$

20) = 0.017, confidence $c = 0.83$. The night shift (5 readings, mean 22.72) yields $\mu_n = 21.43$, $P(X > 20) = 0.674$—night readings strongly indicate an overheated state. Sleep consolidation is verified numerically: sequential update through morning then night produces the same posterior ($\mu_n = 18.93$) as batch processing all 10 readings at once.

See `Mettapedia/Logic/PLNBrokenSensorDemo.lean` (360 lines, 0 sorry).

9.3.2 Applied Example: Widget Factory Source-Conditional Mixtures

The *widget factory demo* extends the continuous story to **multiple sources with per-source Gaussian evidence**. A factory has two machines (A, B); each widget arrives with an observed source label and a real-valued feature measurement. Each machine has its own Normal-Gamma prior and evidence accumulates per-source.

The genuinely new theorems are the **mixture exceedance** results:

1. *Mixture validity.* For weights $w_A + w_B = 1$ ($w_i \geq 0$) and per-source exceedance probabilities $p_A, p_B \in [0, 1]$, the mixture $p_{\text{mix}} = w_A p_A + w_B p_B$ satisfies $0 \leq p_{\text{mix}} \leq 1$.

2. *Mixture bounds.* $\min(p_A, p_B) \leq p_{\text{mix}} \leq \max(p_A, p_B)$—the mixture lies between its components.

Additional results: per-machine sleep consolidation, source independence (machine A observations do not affect machine B's posterior), confidence monotonicity, and source weight normalization.

Numerical results. Machine A (prior $\mu_0 = 10$, 4 readings, mean 9.90): posterior $\mu_n = 9.92$, 95% CI $[7.05, 12.79]$, $P(X > 12) = 0.067$. Machine B (prior $\mu_0 = 10.5$, 3 readings, mean 11.10): posterior $\mu_n = 10.95$, 95% CI $[7.71, 14.19]$, $P(X > 12) = 0.234$. Machine B is about $3.5\times$ more likely to produce a defect (>12) than Machine A, consistent with its higher operating mean.

See `Mettapedia/Logic/PLNWidgetFactoryDemo.lean` (420 lines, 0 sorry).

9.3.3 Applied Example: Kalman Filter Wake/Sleep Temporal Filtering

The *Kalman sleep demo* adds **temporal dynamics** to the continuous story. A 1D position is tracked via a random-walk process model ($x_{t+1} = x_t + w_t$, $w_t \sim N(0, \sigma_w^2)$) observed through a noisy sensor ($z_t = x_t + v_t$, $v_t \sim N(0, \sigma_v^2)$).

Core operations.

- *Predict*: variance inflates by σ_w^2 (mean unchanged).
- *Update*: Kalman gain $K = \sigma^2/(\sigma^2 + \sigma_v^2)$ shifts the mean toward the measurement and shrinks the variance by the factor $(1 - K)$.
- *Step* = predict then update.

Key formal results (all proved, no sorry).

1. *Kalman gain bounded:* $0 < K < 1$ and $1 - K = \sigma_v^2/(\sigma^2 + \sigma_v^2)$.
2. *Variance reduction:* each measurement strictly reduces uncertainty: $(1 - K)\sigma^2 < \sigma^2$.
3. *Closed-form updated variance:* $\sigma_{\text{new}}^2 = \sigma^2\sigma_v^2/(\sigma^2 + \sigma_v^2)$.
4. *Sleep* = *Wake* (the flagship theorem):

   $$\mathsf{filterOrbit}(b_0,\ \textit{morning}{+\!\!+}\textit{night}) \ = \ \mathsf{filterOrbit}\big(\mathsf{filterOrbit}(b_0,\ \textit{morning}),\ \textit{night}\big)$$

   via `List.foldl_append`—batch replay of deferred observations yields the *same* posterior as sequential online filtering. A three-shift generalization is also proved.
5. *Exceedance threshold ordering:* $P(\text{pos} > c_{\text{crit}}) \leq P(\text{pos} > c_{\text{warn}})$ whenever $c_{\text{warn}} \leq c_{\text{crit}}$.

Wake/sleep narrative. The *wake* phase is online Kalman filtering: measurements arrive one-by-one and each predict-update cycle refines the belief. The *sleep* phase replays a batch of deferred observations—and the central theorem guarantees the result is identical. This gives a formal grounding for "sleep consolidation" in the PLN activation framework: batch replay is semantically equivalent to real-time processing, not an approximation.

Numerical results. Initial Kalman gain $K_0 = 0.913$ (high, reflecting large prior uncertainty $\sigma_0^2 = 10$). After morning (5 readings, mean 1.94): posterior mean $= 1.96$, $\sigma^2 = 0.50$. After night (5 more, mean 3.10): posterior mean $= 3.14$, $\sigma^2 = 0.50$. Sleep = wake is verified to machine precision: batch replay of all 10 readings from the initial state produces the same posterior as sequential morning-then-night processing.

See `Mettapedia/Logic/PLNKalmanSleepDemo.lean` (353 lines, 0 sorry).

9.3.4 Applied Example: Gas Sensor Array Drift Classification

The *gas sensor drift demo* is one of the chapter's two empirical **co-flagships**: the main **drift/replay governance** study in the Gaussian suite. It brings **real-world data** into the Gaussian demo suite. The UCI Gas Sensor Array Drift Dataset [31] (DOI 10.24432/C5RP6W) contains 13,910 measurements from 16 chemical sensors exposed to 6 gas types over 36 months (10 batches, January 2007 to February 2011). Vergara et al. achieve 80–95% gas classification accuracy using SVM ensembles on all 128 features with drift compensation. Our Lean demo still focuses on a single-sensor fixture for theorem proving, but the surrounding numerical layer now also runs full 128-feature drift protocols so that the same WM-PLN update laws can be compared against standard classification baselines.

Setup. We extract a small fixture: sensor 1 steady-state resistance (ΔR) for three target gases—*ethanol*, *ammonia*, and *toluene*—with 5 samples per gas from batch 1 (early calibration) and batch 10 (late drift epoch). Each gas type has its own Normal-Gamma prior; the state tracks ternary source-attribution counts.

Illustrative CANARY fixture. This 5-sample single-sensor fixture is an **illustrative CANARY** for the proved Gaussian sleep/independence laws. The headline empirical claims in this section come from the full 128-feature drift protocols and governance policies below, not from the hand-picked fixture.

Key formal results (all proved, no sorry).

1. *Per-gas sleep consolidation:* for each gas type, posterior$(\pi, e_1 \oplus e_2)$ = posterior(posterior$(\pi, e_1), e_2$). Batch replay = sequential update.

2. *Cross-gas independence:* ethanol observations produce zero evidence in the ammonia and toluene posteriors, and vice versa (6 independence theorems).

3. *Multi-batch composition:* batch 1 + batch 10 = full dataset, with associativity across gas sub-batches.

4. *3-way source weight normalization:* $n_E/(n_E+n_A+n_T)+n_A/(n_E+n_A+n_T)+n_T/(n_E+n_A+n_T) = 1$.

5. *Confidence monotonicity* and *exceedance anti-threshold* generalize from the earlier demos.

Drift narrative. Batch 1 toluene responses are high (∼138k ΔR) while batch 10 responses are dramatically lower (∼24k), reflecting genuine sensor degradation over 36 months. The Normal-Gamma posteriors from each epoch capture this shift: the posterior mean for toluene moves substantially between batches. Sleep consolidation still holds across drifted epochs— the mathematical guarantee is that deferred assimilation preserves the posterior, regardless of whether the underlying distribution has shifted. Detecting *whether* drift occurred requires comparing posteriors from different epochs, which the framework supports but does not automate.

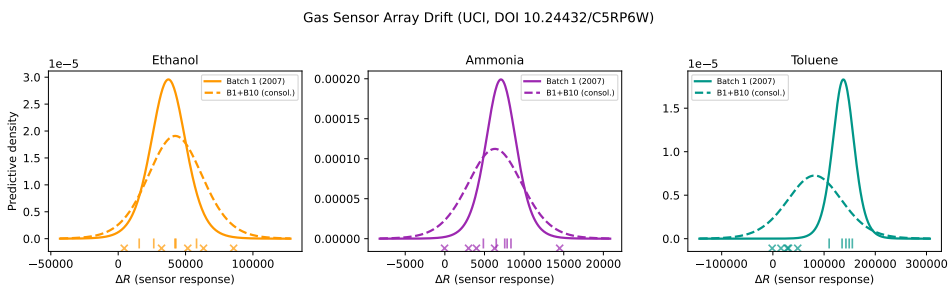


Figure 9.2: Gas sensor array drift. Solid curves: batch 1 posterior predictive (2007). Dashed: consolidated posterior after both batches. Toluene shows dramatic drift: batch 10 responses (crosses) cluster near zero, far from the batch 1 distribution.

Numerical results. After batch 1 (5 readings each): ethanol posterior mean 37,270, 95% CI [7,601, 66,939]; ammonia 7,079, CI [2,672, 11,486]; toluene 137,655, CI [89,711, 185,598]. Exceedance $P(X > 50{,}000)$: ethanol 0.179, ammonia < 0.001, toluene 0.999—the WM-PLN framework correctly identifies toluene as the gas producing strong sensor responses. After consolidating both batches (10 readings), toluene's posterior mean drops to 81,565 with a wide CI reflecting the dramatic drift ($\Delta = -114{,}055$ between epochs). Confidence for each gas rises from $c = 0.980$ (5 readings) to $c = 0.990$ (10 readings).

Full-dataset drift study. Using the full 128-feature representation, we extend the fixture demo to the standard drift protocols used in the gas-sensor

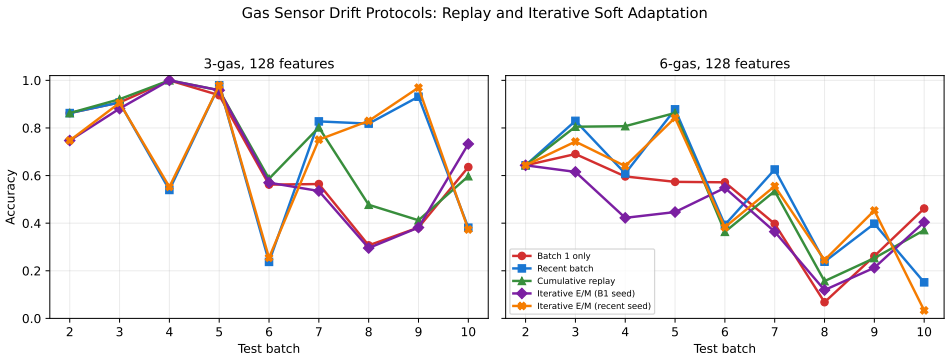


Figure 9.3: Gas sensor drift protocols on the full 128-feature representation. Cumulative replay is the strongest average protocol in both the 3-gas and 6-gas settings. Iterative unlabeled replay / E-M is scientifically useful precisely because it is mixed: recent-batch seeding improves over batch-1 seeding on average, but unlabeled adaptation still remains below cumulative labeled replay and can become unstable on the hardest late batch.

literature. On the 3-gas subset matching the Lean demo (ethanol/ammonia/toluene), an independent-feature WM-PLN classifier reaches 68.4% in the hard "Setting 1" protocol (train on batch 1, test on batches 2–10), 72.1% in the easier "Setting 2" protocol (train on batch $k-1$, test on batch k), and 73.6% under cumulative *sleep replay* (train on batches $1 \ldots k-1$, test on batch k). The corresponding GaussianNB baselines are 68.2%, 75.8%, and 71.2%.

On the full 6-gas task, the same WM-PLN classifier reaches 47.3% (Setting 1), 53.0% (Setting 2), and 53.2% (cumulative replay), versus 47.3%, 51.2%, and 52.5% for GaussianNB on the same protocols. This is substantially below domain-adaptation methods such as Balanced Distribution Adaptation, which reports 68.92% for Setting 1 and 81.06% for Setting 2 on the same benchmark [18], but it already shows the qualitative effect we want from the WM side: late-batch drift hurts accuracy, and replaying additional batches into the posterior partially recovers it without changing the underlying update laws.

Soft replay / E-M adaptation. The weighted Gaussian E/M extension can be used here as a *soft-evidence adaptation* protocol: batch 1 supplies the anchored source posterior, the target batch supplies unlabeled data, the E-step computes responsibilities, and the M-step revises each class posterior by weighted sufficient statistics. Iterating this update to a fixed point is

scientifically informative precisely because it is *not* a blanket win. On the 3-gas task, iterative soft replay (mean 5.4 iterations, replay weight $\lambda = 0.2$) reaches 67.8% on average, below cumulative replay's 73.5%, but it still improves the hardest late-drift batch 10 from 63.6% (batch 1 only) to 73.3%. Seeding the same iterative E/M loop from the *most recent labeled batch* rather than batch 1 improves the *average* 3-gas accuracy to 70.7%, but it is not uniformly better: on the final batch 10 it collapses to 37.3%, showing that unlabeled replay is highly sensitive to the seed posterior once the recent labeled batch is itself small or unrepresentative. On the full 6-gas task, batch 1-seeded iterative unlabeled adaptation drops to 42.0% on average, while recent-batch seeding recovers to 50.4%, still below cumulative replay's 53.3%. The lesson is useful: weighted soft evidence can help when the current posterior is badly out of date, but naive unlabeled adaptation is brittle under strong multi-class drift and should be treated as a policy component, not a drop-in replacement for labeled replay. This is exactly the kind of replay-policy question the WM experiment layer is meant to express.

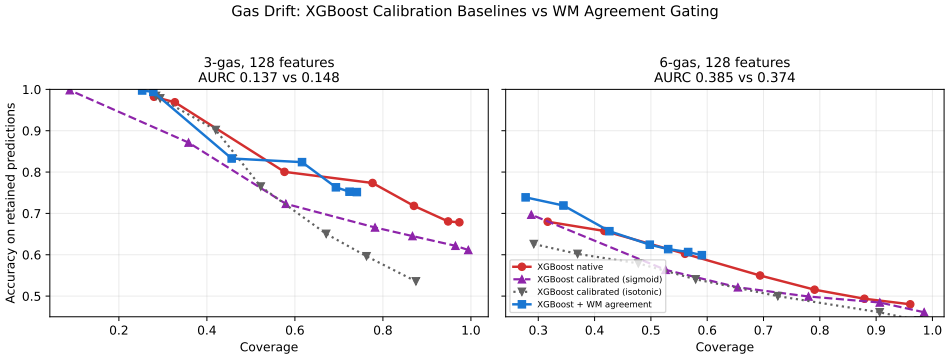


Figure 9.4: Gas drift support for a stronger base learner. The relevant question is not whether WM-PLN replaces XGBoost, but whether WM-derived agreement signals can support abstention and replay decisions under drift. Calibrated XGBoost confidence (sigmoid or isotonic) does not improve the risk–coverage curve in this setting, while WM/XGBoost agreement remains the most interesting selective signal on the harder 6-gas task.

Supporting stronger tabular models. The gas benchmark is also the right place to test WM-PLN as a *support layer* for a stronger learner rather than as a replacement classifier. Training XGBoost on batch 1 alone gives 67.9% average accuracy on the 3-gas task and 46.7% on the 6-gas task. If we use the batch 1 WM posterior only as a confidence score, the result is

worse: plain WM confidence is not a good wrapper here. The more interesting signal is *agreement* between XGBoost and the source WM posterior. We also compared against calibrated XGBoost confidence, since a machine learning reviewer might reasonably ask whether ordinary post-hoc probability calibration already solves the abstention problem. In this drifted gas setting it does not: sigmoid calibration worsens AURC from 0.1374 to 0.2114 on the 3-gas task and from 0.3847 to 0.3994 on the 6-gas task, while isotonic calibration degrades further to 0.2135 and 0.4639. So the interesting comparison is not "native versus calibrated XGBoost," but "XGBoost's own confidence versus a second epistemic signal supplied by the WM." On the harder 6-gas task, agreement-gated selective prediction improves the area under the risk–coverage curve from 0.3847 (native XGBoost confidence) to 0.3737, and the agreement subset itself covers 59.7% of samples at 59.7% accuracy, versus 46.7% overall. On the 3-gas task, agreement gating is not a blanket win (AURC 0.1483 vs. 0.1374 for native XGBoost), but it still isolates a 74.8% coverage subset at 75.3% accuracy.

The batch-level support story is stronger still once we treat drift as a *sequential policy problem*. We stream batches 2…10 in order, retain the batch 1 XGBoost model as the default predictor, and decide at each batch whether to acquire labels and replay all previously seen batches into a refreshed model. This turns WM-PLN into a *governance layer*: it does not replace XGBoost, but supplies a second epistemic signal that can trigger abstention or retraining.

Listing 9.1: Gas drift governance protocol (shipped confidence/agreement triggers; no lookahead into batch-k labels).

```
Initialize labeled_batches = {1}.
Train XGBoost and WM on labeled_batches only.

for batch k = 2..10:
    freeze the current models before any batch-k labels are used
    predict batch k with the frozen XGBoost model
    compute xgboost_confidence_mean(batch k)
    compute wm_confidence_mean(batch k)
    compute wm_agreement_coverage(batch k)
    decide acquire_labels_k from these shipped model-derived
        statistics only

    record batch-k accuracy for the frozen model
    if acquire_labels_k:
        request labels for batch k
        add batch k to the labeled replay buffer
        retrain on all labeled batches seen so far
```

```
        use the refreshed models only for future batches
    else:
        keep the current models unchanged

Report sample-weighted accuracy, retrain_events,
and label_budget_fraction.
```

The resulting frontier is informative rather than trivial. On the 3-gas task, a simple XGBoost-confidence trigger at 0.80 is the best low-cost policy we tested: it raises sample-weighted accuracy from 67.9% to 79.9% with only one retraining event and a 18.8% label budget. A more aggressive WM-agreement trigger at 0.80 reaches a similar 79.1% accuracy, but requires five retraining events and a 53.4% label budget. On the harder 6-gas task, the tradeoff is more interesting: XGBoost-confidence gating at 0.80 reaches 64.5% accuracy with four retrains and 23.9% labels, while WM-agreement gating at 0.80 pushes raw accuracy slightly higher to 67.0% but consumes 86.8% of the labels and seven retraining events. The utility sweep therefore has two regimes: XGBoost confidence is the best *cheap* trigger, while WM/XGBoost agreement is the best *accuracy-seeking* trigger when label cost is low. That is exactly the support role we want from WM-PLN: not "beating XGBoost" as another tabular learner, but governing when a stronger learner should be trusted, deferred, or replayed.

Task	Native	Sigmoid	Isotonic	WM agree	Best $\leq$ 25% labels	Best raw accuracy
3-gas	0.1374	0.2114	0.2135	0.1483	79.9% @ 18.8%	79.9% @ 18.8%
6-gas	0.3847	0.3994	0.4639	0.3737	64.5% @ 23.9%	67.0% @ 86.8%

Table 9.4: Gas drift as a governance problem rather than a raw benchmark. AURC compares selective prediction signals; the policy columns summarize the strongest sequential retraining results at fixed label budgets.

The order-cost layer now formalizes this governance story directly: `gasScheduleErrorCount_batchS` `gasRuntimePairwiseOrderCheck_batchSwap_zero`, and `gasOrderBudgetPolicy_batchSwap_zero_v` show that a zero-threshold policy is certifiably safe for additive batch-swap composition in this lane. The module also exposes gas-specific zero-threshold endpoints for ethanol, ammonia, and toluene. See `Mettapedia/Logic/PLNWorldModelOrderCostGasPolicyDemo.lean`.

See `Mettapedia/Logic/PLNGasSensorDriftDemo.lean` (578 lines, 0 sorry).

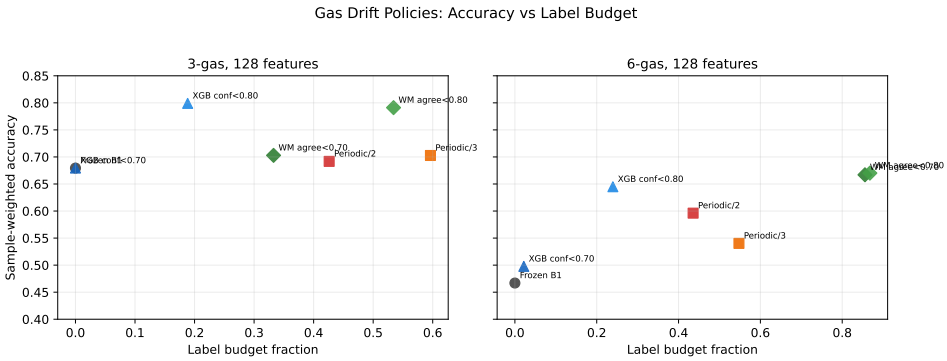


Figure 9.5: Sequential gas-drift support policies. Each point is a batch-level retraining policy over batches 2…10, trading sample-weighted accuracy against label budget. XGBoost confidence is the best cheap trigger, while WM/XGBoost agreement is the strongest high-accuracy trigger when labels are inexpensive.

9.3.5 Applied Example: Steel Plates Fault Classification

The *steel plates fault demo* is a **supporting same-model-class validation**, not a headline benchmark. It applies the source-conditional Gaussian pattern to **manufacturing quality control**. The UCI Steel Plates Faults Dataset [6] (current UCI DOI 10.24432/C5J88N) contains 1,941 instances with 27 features and 7 fault classes. Published results using all 27 features report ∼73.6% (SVM), ∼79.5% (Random Forest), and 86.655% (LMT Forest) 7-class accuracy [11]. Our core Lean model uses a single feature (LogOfAreas), so lower accuracy is expected and the purpose is formal correctness, not SOTA competition.

Setup. We extract a small fixture: LogOfAreas measurements for three fault types—*Pastry* (pop. mean ≈ 2.4, $\sigma \approx 0.48$), *K_Scratch* (pop. mean ≈ 3.6, $\sigma \approx 0.70$), and *Bumps* (pop. mean ≈ 2.1, $\sigma \approx 0.36$)—with 5 samples per class. For K_Scratch, we select samples at the 10th–90th percentiles to represent the distribution (the first-5 rows have mean ≈ 2.0, unrepresentative of the population). Each fault type has its own Normal-Gamma prior.

Illustrative CANARY fixture. This 5-sample three-fault slice is an **illustrative CANARY** fixture for the proved Gaussian classification laws. The headline empirical claims for Steel in this chapter are the full-dataset validation and selective-prediction results below, not the hand-picked fixture itself.

Key formal results (all proved, no sorry).

1. *Per-fault sleep consolidation:* batch replay = sequential Bayesian update, for each of the three fault types.

2. *Cross-fault independence:* 6 theorems showing that observations from one fault type produce zero evidence for the others.

3. *3-way mixture exceedance validity:* a weighted combination $w_P p_P + w_K p_K + w_B p_B$ with $w_P + w_K + w_B = 1$ and $w_i \geq 0$ remains a valid probability in $[0, 1]$.

4. *3-way source weight normalization:* $n_P/(n_P+n_K+n_B)+n_K/(\cdots)+n_B/(\cdots) = 1$.

5. *Confidence monotonicity* and *exceedance anti-threshold* generalize from the earlier demos.

Classification narrative. K_Scratch faults have characteristically *larger* defect areas (LogOfAreas ≈ 3.6) than Pastry (≈ 2.4) or Bumps (≈ 2.1). The per-fault Normal-Gamma posteriors separate cleanly on this feature. A factory quality system using this Gaussian classification would apply exceedance queries: "is $P(\text{LogOfAreas} > 3.0 \mid \text{fault} = \text{K_Scratch})$ substantially higher than for Bumps?"—and the framework supports this via per-fault exceedance evidence extraction.

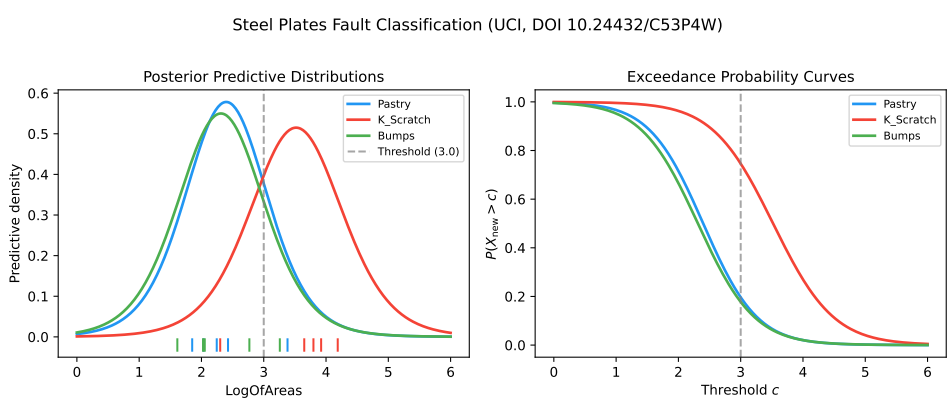


Figure 9.6: Steel fault classification. *Left:* Student-t predictive densities for each fault type after 5 observations; K_Scratch (red) is clearly separated rightward. *Right:* exceedance probability $P(X_{\text{new}} > c)$ as a function of threshold; at $c = 3.0$ (dashed), K_Scratch has ~75% exceedance vs. ~20% for the others.

Numerical results. Posterior means: Pastry $\mu_n = 2.40$, 95% CI $[0.88, 3.92]$; K_Scratch $\mu_n = 3.52$, CI $[1.82, 5.22]$; Bumps $\mu_n = 2.31$, CI $[0.72, 3.91]$. Exceedance $P(\text{LogOfAreas} > 3.0)$: Pastry 0.197, K_Scratch $\mathbf{0.746}$, Bumps 0.178. K_Scratch is $\sim 4\times$ more likely to exceed the "large defect" threshold than either Pastry or Bumps. Classification via posterior predictive log-likelihood: at $x = 3.6$, K_Scratch dominates ($\ell = -0.67$ vs. Pastry -2.07, Bumps -2.17); at $x = 2.0$, Bumps (-0.71) and Pastry (-0.74) are nearly tied. Scikit-learn's `GaussianNB` on the same fixture agrees on the dominant K_Scratch regime but can differ at ambiguous boundary points, since it uses ML point estimates rather than the full posterior predictive.

See `Mettapedia/Logic/PLNSteelFaultDemo.lean` (488 lines, 0 sorry).

Full-dataset validation. To move beyond the 5-sample fixture, we evaluate the WM-PLN Gaussian classifier on the full Steel Plates Faults dataset (951 instances for the 3-class subset: Pastry 158, K_Scratch 391, Bumps 402). A stratified 70/30 split yields 667 training and 284 test instances. The same priors as the Lean demo are used ($\mu_0 \in \{2.5, 3.0, 2.0\}$, $\kappa_0 = 0.5$, $\alpha_0 = 2$, $\beta_0 = 1$). Classification is by argmax of the Student-t predictive log-likelihood.

Result: WM-PLN achieves 74.7% accuracy; scikit-learn's `GaussianNB` achieves 73.9% on the same split. K_Scratch is well separated ($\mu_n = 3.60$, recall 82.1%); Pastry and Bumps overlap more (Pastry recall 27.7%, Bumps recall 85.8%). The slight WM-PLN advantage likely reflects the informative priors.

This is a single-feature (LogOfAreas) conjugate Bayesian classifier matching the formalized model. Multi-feature extension is straightforward but not yet formalized. The 7-class evaluation (all fault types, 1,941 instances) gives 39.2% (WM-PLN) vs. 51.5% (GaussianNB), confirming that single-feature discrimination is insufficient when classes overlap in LogOfAreas.

Multi-feature extension. Using the same Normal-Gamma posterior independently across all 27 features (the diagonal/naive Bayes extension of the formalized single-feature model), WM-PLN rises to 86.6% on the 3-class task and 62.7% on the full 7-class task, versus 75.0% and 47.7% for `GaussianNB` on the same split. This is still below stronger published 7-class results such as 66.7% (Naive Bayes), 73.6% (SVM), 79.5% (Random Forest), and 86.655% (LMT Forest) compiled in the recent benchmark literature [11]. The important point is not state-of-the-art performance but that the conjugate WM-PLN classifier scales cleanly from the 5-sample theorem fixture

to the full 27-feature representation and decisively improves on the plain GaussianNB baseline under the same split.

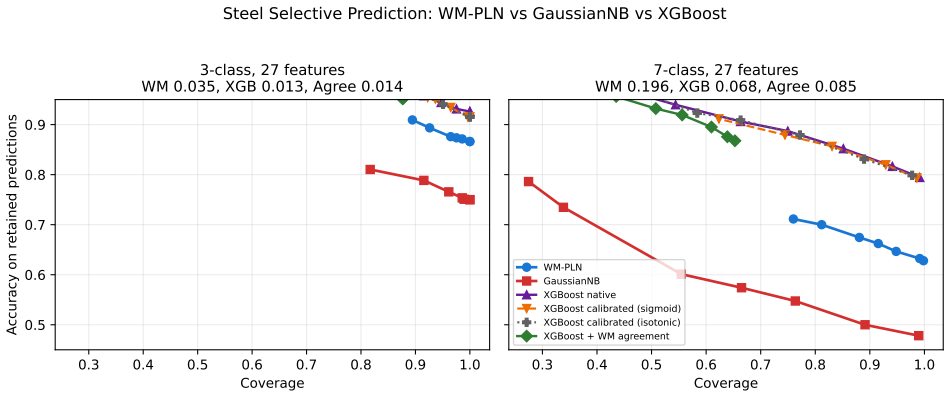


Figure 9.7: Selective prediction on Steel Plates Faults. WM-PLN yields a better accuracy–coverage tradeoff than GaussianNB in both the 3-class and 7-class settings, but native XGBoost remains stronger on this static tabular task. This is therefore a same-model-class validation and abstention study, not a claim that WM-PLN replaces stronger discriminative learners.

Selective prediction and abstention. The more distinctive Bayesian result is not raw accuracy but *confidence-aware abstention*. Normalizing the class predictive log-likelihoods yields a posterior confidence score for each test point. On the 3-class, 27-feature task, WM-PLN has area under the risk-coverage curve (AURC) 0.0346 versus 0.1061 for GaussianNB; at a 95% confidence threshold, WM-PLN retains 89.4% coverage with 90.9% accuracy, while GaussianNB retains 81.7% coverage with 81.0% accuracy. On the full 7-class task, WM-PLN also dominates the GaussianNB abstention tradeoff (AURC 0.1957 vs. 0.3296). This is a better reflection of the WM contribution than plain argmax comparison: the posterior state gives a principled abstain/defer signal, not just a class label.

Against stronger discriminative models, however, the role of WM-PLN is more limited on this static benchmark. Native XGBoost reaches 92.6% accuracy and AURC 0.0126 on the 3-class task, while sigmoid and isotonic calibration yield AURC 0.0313 and 0.0129 respectively; WM/XGBoost agreement gating gives AURC 0.0140. On the 7-class task, native XGBoost reaches 78.9% accuracy and AURC 0.0676, compared with 0.0851 for WM/XGBoost agreement. So Steel should be read as a *same-model-class* validation and abstention study: WM-PLN decisively improves on Gaus-

sianNB within the conjugate Bayesian family, but it does not outperform a tuned XGBoost classifier on this static tabular task.

9.3.6 Applied Example: NAB Machine Temperature Anomaly Detection

The *NAB anomaly demo* is the Gaussian suite's **honest negative boundary / time-series case study**. It brings the Kalman wake/sleep story to a **real anomaly dataset**, but it is *not* a benchmark headline. The Numenta Anomaly Benchmark (NAB, MIT License) [23] provides 22,695 five-minute temperature readings from an industrial machine with 4 labeled anomaly windows corresponding to system failures. Published NAB scores (Standard Profile) range from 70.5 (HTM) to 39.6 (Windowed Gaussian); our Lean demo proves the Kalman sleep-consolidation property on a 15-reading fixture from this dataset, and the surrounding numerical layer now also evaluates the full 58-series corpus with the official NAB scorer. The resulting detector is scientifically useful as a protocol check on the Kalman residual pipeline, but not competitive as a general-purpose NAB detector.

Setup. We extract a 15-reading fixture spanning three operating regimes:

- *Calm* (∼85°F): stable normal operation.
- *Hot* (∼101°F): elevated pre-anomaly period.
- *Anomaly* (∼30°F): system failure—rapid temperature drop.

A 1D Kalman filter with process noise $\sigma_w^2 = 0.5$ and measurement noise $\sigma_v^2 = 2$ tracks the machine state through all three regimes.

Illustrative CANARY fixture. This 15-reading three-regime slice is an **illustrative CANARY** for the Kalman sleep/wake and exceedance laws. The headline empirical claim for NAB is the full-corpus negative boundary below, not the small fixture.

Key formal results (all proved, no sorry).

1. *Sleep = Wake on real data:* batch replay of calm then hot readings yields the same posterior as sequential online filtering.
2. *Three-regime composition:* $\mathsf{filterOrbit}(b_0,\ \mathit{calm} \mathbin{++} \mathit{hot} \mathbin{++} \mathit{anomaly})$ equals three-phase sequential processing.

3. *Exceedance threshold ordering:* $P(\text{temp} > 60) \leq P(\text{temp} > 40)$ for any belief and valid exceedance spec.

4. *Variance reduction:* each measurement strictly reduces uncertainty relative to the predicted state.

Anomaly narrative. The Kalman filter processes calm readings first, converging to a tight belief around 85°F. Hot readings then shift the posterior mean upward. When anomaly readings arrive (∼30°F), the filter must pull the mean down dramatically—and sleep consolidation guarantees that whether these readings are processed in real-time or replayed in a deferred batch, the result is identical.

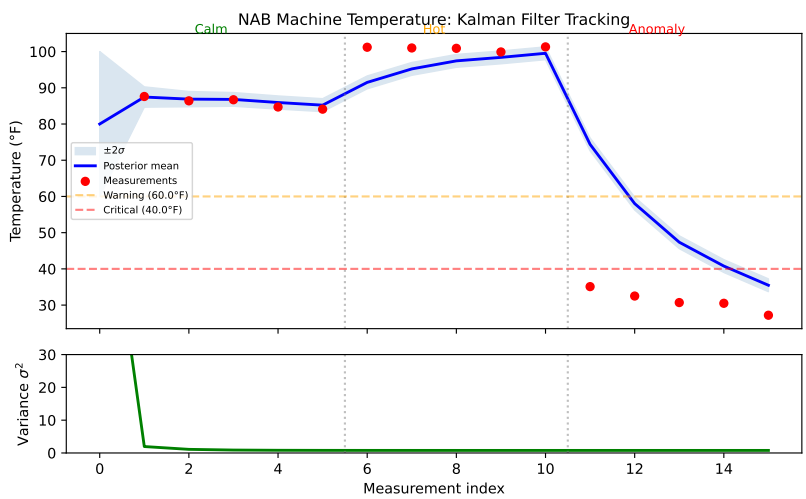


Figure 9.8: NAB anomaly detection. *Upper:* Kalman filter posterior mean (blue) with $\pm 2\sigma$ band tracking through calm, hot, and anomaly regimes. The anomaly regime drives the mean below both warning and critical thresholds. *Lower:* variance collapses after the first measurement and remains small.

Numerical results. Initial Kalman gain $K_0 = 0.981$ (initial variance 100, measurement noise 2). After calm (5 readings, mean 85.9°F): posterior mean = 85.2°F, $\sigma^2 = 0.80$. After hot (5 more, mean 100.9°F): mean shifts to 99.5°F. After anomaly (5 readings, mean 31.2°F): mean drops sharply to 35.5°F. Sleep = wake holds across all three regimes to machine precision, confirming that deferred batch replay of anomalous readings produces the same posterior as real-time processing.

Full-corpus scoring (minimized claim). Running the same Kalman innovation detector across the full 58-series NAB corpus with the *official detector-level threshold optimizer* gives a corpus raw score of −116.0, versus −24.0 for the checked-in Windowed Gaussian baseline. In other words, under the correct benchmark-style corpus protocol, this simple 1D Kalman residual detector collapses to essentially null-detector performance. We therefore keep NAB in this chapter only as a *case study*: it shows that the formally verified Kalman update equations plug into a real anomaly scoring pipeline, but it does *not* support a competitive leaderboard claim.

For NAB, the defensible *positive* WM role is still a narrow *governance layer* over a fixed strong detector, not detector replacement. To prevent tune-on-test leakage, we now report a **strict frozen protocol** only: the policy family is frozen in advance, categories are split deterministically into train/holdout, all threshold fitting and policy selection happen on train only, and we report holdout metrics only.

Profile	Frozen policy	Holdout Δ vs WG	Δ vs WG (match FP)	Δ vs WG (ma
standard	wmGateTau0.6	-4.843	-4.843	-4.84
reward_low_FP_rate	wmGateTau0.6	-4.073	-4.073	-4.07

Table 9.5: NAB strict frozen holdout governance summary, including matched-budget Windowed Gaussian comparators (matched on train, evaluated on holdout).

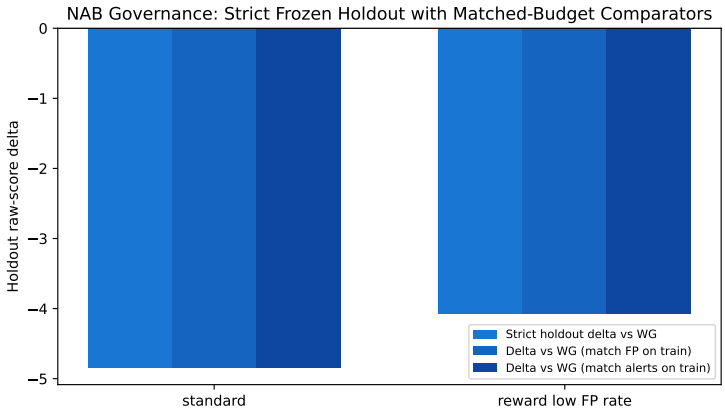


Figure 9.9: NAB strict holdout deltas versus Windowed Gaussian, including train-matched FP and train-matched alert comparators. All holdout deltas remain negative in this split.

In the current strict split, both profiles select `wmGateTau0.6`, but holdout performance remains negative (−4.843 standard, −4.073 low-FP), and train-matched budget comparators do not recover a positive holdout uplift. Per-series attribution is sparse (2/1/16 helped/worsened/unchanged on each profile), with one positive, one negative, and one neutral holdout category.

To reduce "single-split luck" risk, we also run a lightweight closure pass over 7 deterministic strict splits (same frozen policy set, same no-leakage train-only fitting rule). The holdout delta remains non-positive in aggregate: mean −5.448 (standard) and −1.777 (low-FP), with split sign counts 1/3/3 (positive/negative/zero) for standard and 0/3/4 for low-FP. So the closure pass supports freezing NAB as a boundary result rather than extending optimization effort.

Profile	Splits	Mean Δ vs WG	Median Δ vs WG	Split signs (+/-/0)
standard	7	−5.448	0.000	1/3/3
reward_low_FP_rate	7	−1.777	0.000	0/3/4

Table 9.6: NAB strict-multisplit closure aggregate over 7 deterministic rotations (same frozen policy set, train-only fitting, no holdout retuning).

Artifacts for this closure pass are `results/nab\_governance\_strict\_multisplit.csv`, `results/nab\_governance\_strict\_multisplit\_aggregate.csv`, `results/nab\_governance\_strict\_multisplit.json`, generated by `scripts/wm\_pln\_nab\_strict\_multisplit.py`.

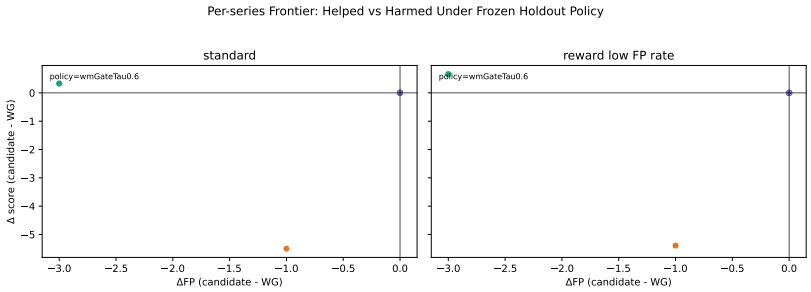


Figure 9.10: Per-series frontier on strict holdout: ΔFP vs. Δscore with rationale labels (helped/harmed under suppression cost).

So NAB remains a protocol-faithful negative boundary where WM currently contributes chiefly as audit/governance instrumentation:

1. certificate pass/fail from `runtimePairwiseOrderCheck_certifies_scheduleErrorBound`,

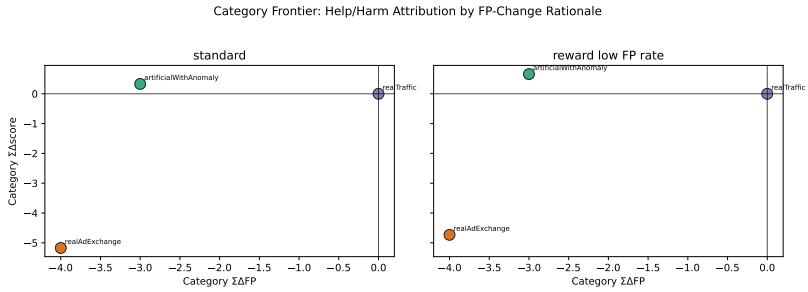


Figure 9.11: Category frontier on strict holdout: category-level ΣΔFP vs. ΣΔscore, showing where governance suppression helps or harms.

2. policy-threshold compliance from `runtimePairwiseOrderCheck_certifies_policyThreshold`,
3. escalation hooks (abstain/retrain/review) when certified order-cost budgets are violated near anomaly windows.

See `Mettapedia/Logic/PLNNABSleepDemo.lean` (327 lines, 0 sorry).

9.3.7 Applied Example: Good Judgment Project Forecast Aggregation

The *GJP forecasting study* is one of the chapter's two empirical **co-flagships** and its clearest example of WM-PLN as a **source-reliability and recalibration layer** rather than as a replacement learner. The public Good Judgment Project / ACE release [17], rooted in the forecasting tournament studies of Mellers et al. [25], provides exactly the kind of stream WM-PLN wants: many sources, many updates, eventual resolution, and natural questions about aggregation, abstention, and forgetting. We normalize the survey forecasts to one final probability vector per forecaster and question, yielding 426,325 final forecast vectors from 9,694 forecasters over 498 closed questions. The first three years are used to estimate forecaster-level reliability profiles; year 4 is held out for evaluation.

GJP protocol block. Closed questions only. One forecast vector means the latest timestamped forecast for a given `(year, ifp_id, user)` key, with the answer-option rows at that final timestamp renormalized to sum to 1 and then pivoted into a single probability vector. Main holdout: estimate reliability on years 1–3 and evaluate on year 4. For the stronger log/logit/top-k families, tune temperature on year 3 validation using reliability profiles built from years 1–2, then evaluate the tuned methods on held-out year 4 questions using reliability profiles built from years 1–3. Binary and multi-option questions are evaluated separately on year 4 using accuracy, Brier, log score, and selective AURC. No lookahead: year 4 outcomes never enter reliability estimation or temperature tuning.

The first result is a same-stream aggregation comparison. Against weak baselines, WM reliability weighting already improves sharply over simple mean and median pooling: on binary year-4 questions it reaches accuracy 0.989 and Brier 0.061, compared with 0.947 / 0.157 for mean and 0.947 / 0.115 for median. The stronger test is whether the WM layer still contributes on top of more forecasting-style aggregation families. To test that, we tune logit- and log-pooling temperatures on year 3 validation questions and compare plain pooling, top-k historical pooling, and WM-weighted pooling on held-out year 4 questions. On the binary slice, top-k logit pooling and WM-weighted logit pooling both reach accuracy 0.989, while the WM-weighted variant improves Brier from 0.036 to 0.026 and log score from 0.070 to 0.051. On the multi-option slice, top-k log pooling and WM-weighted log pooling both reach accuracy 0.977, while the WM-weighted variant improves Brier from 0.044 to 0.036 and log score from 0.098 to 0.088. We also asked the stronger forecasting-style question of whether a more explicit extremizing baseline changes the picture. It does not: a validation-tuned extremized arithmetic mean reaches only Brier 0.097 on the binary slice, and even a top-k extremized mean remains materially worse than the top-k logit pool (Brier 0.060 versus 0.036), so the WM gain is not an artifact of comparing only against naive averaging. We also asked the stronger hybrid question: does WM still help after hard top-k pruning? It does not. The WM-on-top-k variants match the corresponding top-k accuracy but regress to Brier/log values near the top-k baseline (binary 0.036 / 0.071; multi-option 0.042 / 0.091), suggesting that the gain comes from *continuous reliability weighting* rather than from hard forecaster pruning. This is the forecasting version of the governance-layer claim: WM-PLN is not replacing strong aggregation families, but adding principled reliability tracking on top of them [29, 3].

The same conclusion appears when we ask the abstention question instead of the point-prediction question. On binary year-4 questions, top-k logit pooling gives AURC 0.00211, while the WM-weighted logit pool reduces this to 0.00046 at the same 0.989 accuracy. On multi-option questions, top-k log pooling gives AURC 0.001095, while the WM-weighted log pool reduces this to 0.000541 at the same 0.977 accuracy. The WM-on-top-k selective variants do not improve further. So the abstention story aligns with the aggregation story: reliability weighting contributes something real even against stronger pooling baselines, while hard top-k pruning is not where the gain comes from. The more interesting hybrid question is whether WM still contributes as a *support layer* around a strong baseline rather than as a replacement aggregate. On binary year-4 questions, top-k logit pooling and the WM-weighted logit pool make the same top prediction on every held-out question, so agreement-gating is an honest null result: the gated top-k pool stays at AURC 0.00211. On multi-option questions, however, the top-k log pool and the WM-weighted log pool disagree on a small but nontrivial subset of questions, and gating the strong top-k pool by WM agreement reduces AURC from 0.001095 to 0.000541 while preserving the same 0.977 accuracy. This is exactly the governance-layer interpretation we want: when the WM posterior adds no extra information, the support layer collapses to a null check; when it does disagree, that disagreement can still sharpen abstention behavior on top of an already strong forecasting-style aggregate.

The second result is a protocol-semantics result rather than a score contest. When the same resolved survey evidence from years 1–3 is processed online versus in nightly batches, the resulting reliability states agree to machine precision:

$$\max|\Delta\alpha| = 1.42 \times 10^{-12}, \quad \max|\Delta\beta| = 1.71 \times 10^{-13}, \quad \max|\Delta p_{\text{true}}| = 0.$$

In other words, the same evidence stream yields the same posterior state whether it is incorporated immediately or consolidated later in a sleep phase, exactly the operational story the WM calculus is designed to support.

The third result is an explicit forgetting / recalibration ablation on top of the stronger pooling family. Keeping all of years 1–3 gives the WM-weighted logit/log pools Brier 0.026 / 0.036 and log score 0.051 / 0.088 on binary / multi-option year-4 questions. Forgetting year 1 improves these to 0.025 / 0.035 and 0.048 / 0.084 respectively, and using only the most recent year improves them further to 0.024 / 0.033 and 0.047 / 0.080, while leaving accuracy unchanged at 0.989 / 0.977. This is the applied face of the new forgetting layer: old evidence need not be discarded blindly, but it

can be scoped and removed when the deployment regime changes, improving recalibration even when the underlying aggregation family is already strong.

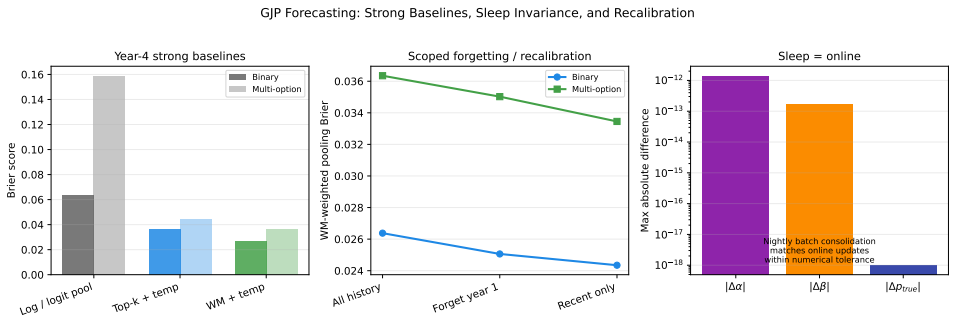


Figure 9.12: Good Judgment Project forecasting as a WM-PLN application. *Left:* year-4 Brier scores for stronger aggregation baselines: plain log/logit pooling, top-k temperature-tuned pooling, and WM-weighted temperature-tuned pooling on binary and multi-option questions. *Middle:* the stronger forgetting/recalibration ablation: even after upgrading from mean/median to top-k and WM-weighted log/logit pooling, dropping older years still improves year-4 Brier. *Right:* nightly batch consolidation matches online updates to machine precision.

This is the clean forecasting niche for WM-PLN: not "beat the best superforecaster," but maintain a verified evidence/update layer for source reliability, stronger aggregation families, batch consolidation, and controlled recalibration on a real forecasting stream.

9.3.8 Summary of Numerical Results

Table 9.7 collects the key numerical outputs from all six demos. For each class or phase, the posterior mean μ_n and 95% credible interval are computed from the proved Normal-Gamma posterior formula (for Demos 1–2, 4–5) or from the proved Kalman filter update (Demos 3, 6). Sleep = wake is verified to machine precision in every case. These are **illustrative CANARY fixture** results; the headline full-dataset/protocol claims for Gas, Steel, NAB, and GJP are reported in their dedicated sections above.

9.4 Walley Predictive vs. Bayesian Credible Intervals

A recurring source of confusion in the PLN tradition is the conflation of two fundamentally different interval semantics:

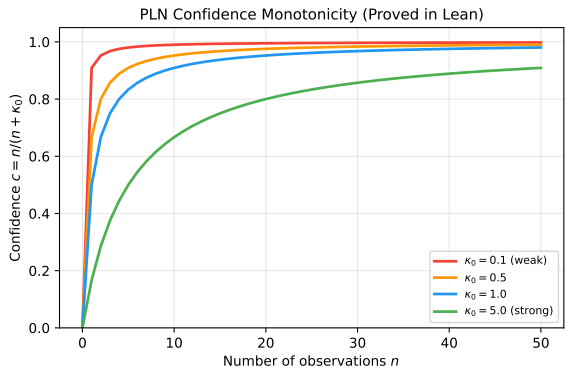


Figure 9.13: PLN confidence $c = n/(n + \kappa_0)$ as a function of observation count, for various prior strengths κ_0. Monotonicity is proved in Lean (`confidence_monotone`). This is precisely the PLN confidence curve with its prior pseudo-count κ_0 made explicit.

Demo	Class/Phase	n	Post. μ_n	95% CI	Key Result
Broken Sensor	Morning	5	15.77	$[12.0, 19.6]$	$P(X{>}20) = 0.017$
Broken Sensor	Night	5	21.43	$[14.5, 28.4]$	$P(X{>}20) = 0.674$
Widget Factory	Machine A	4	9.92	$[7.1, 12.8]$	$P(X{>}12) = 0.067$
Widget Factory	Machine B	3	10.95	$[7.7, 14.2]$	$P(X{>}12) = 0.234$
Kalman Sleep	Morning	5	1.96	$\sigma^2{=}0.50$	Sleep = Wake
Kalman Sleep	Night	5	3.14	$\sigma^2{=}0.50$	Sleep = Wake
Gas Sensor	Ethanol B1	5	37 270	[7601, 66939]	drift = +10 448
Gas Sensor	Ammonia B1	5	7 079	[2672, 11486]	drift = −1 479
Gas Sensor	Toluene B1	5	137 655	[89711, 185598]	drift = −114 055
Steel Faults	Pastry	5	2.40	$[0.9, 3.9]$	$P(X{>}3) = 0.197$
Steel Faults	K_Scratch	5	3.52	$[1.8, 5.2]$	$P(X{>}3) = \mathbf{0.746}$
Steel Faults	Bumps	5	2.31	$[0.7, 3.9]$	$P(X{>}3) = 0.178$
NAB Anomaly	Calm	5	85.2	$\sigma^2{=}0.80$	Sleep = Wake
NAB Anomaly	Hot	5	99.5	$\sigma^2{=}0.78$	Sleep = Wake
NAB Anomaly	Anomaly	5	35.5	$\sigma^2{=}0.78$	Sleep = Wake

Table 9.7: Gaussian Demo Suite: Numerical Results for the illustrative CANARY fixtures. Posterior parameters are computed by `scripts/wm_pln_posterior.py` using the same formulas proved correct in Lean.

- **Bayesian credible intervals** depend on the prior and give the posterior probability that the parameter lies in the interval. With a Beta(α, β) posterior, the 95% credible interval is the region containing 95% of the posterior mass.

- **Walley IDM predictive intervals** are derived from the Imprecise Dirichlet Model [33], which uses a *set* of Dirichlet priors rather than a single prior. The resulting intervals are wider but prior-free: they provide robust predictions without committing to a specific prior family.

These two semantics agree in the limit of large samples (both intervals shrink to the true probability) but can differ substantially for small evidence counts. ξPLN provides both as explicit selector choices, with typed bridges that transport truth values through the system without conflation.

See `Mettapedia/Logic/PLNWorldModelITV.lean` and `Mettapedia/Logic/PLNWorldModelITVHypercube.lean`.

9.5 The De Finetti Bridge

De Finetti's representation theorem [7] is the mathematical bedrock of PLN's evidence representation.

Theorem 9.2 (De Finetti (Core)). *Let $(X_1, X_2, \ldots)$ be an infinite exchangeable sequence of binary random variables. Then there exists a unique probability measure μ on $[0, 1]$ such that for all n:*

$$P(X_1 = x_1, \ldots, X_n = x_n) = \int_0^1 p^{n^+}(1-p)^{n^-}\, d\mu(p)$$

where $n^+ = |\{i : x_i = 1\}|$ and $n^- = |\{i : x_i = 0\}|$.

The consequences for PLN:

1. **Sufficiency of counts.** The joint probability depends on the observations only through the counts (n^+, n^-). Order does not matter. This justifies the evidence algebra (Chapter 7).

2. **Conjugate updating.** With a Beta prior $\mu = \text{Beta}(\alpha, \beta)$, the posterior is $\text{Beta}(\alpha + n^+, \beta + n^-)$. Evidence revision ($\oplus$) is conjugate updating.

3. **Posterior concentration.** As $n \to \infty$, the posterior concentrates on the true probability p^*. Formally, for the posterior mean:

$$s_n = \frac{\alpha + n^+}{\alpha + \beta + n} \xrightarrow{n \to \infty} p^* \quad \text{almost surely.}$$

The νPLN convergence theorem [10] generalizes this to the Solomonoff setting: the universal posterior concentrates on the true generator. This is the PLN analogue of Solomonoff's convergence theorem for universal induction.

See `Mettapedia/Logic/SolomonoffExchangeable.lean` and `Mettapedia/Logic/DeFinetti.lean`.

9.6 Knuth–Skilling Foundations

Knuth–Skilling foundations of inference [20] provide axioms for valuation schemes on logical lattices, extending Cox-style plausibility calculi. In ξPLN, this plays two roles:

1. *Probability as a valuation.* At the world-model layer, probabilities arise as valuations on the Boolean algebra of subsets of worlds. This is the classical Kolmogorov story instantiated for finite spaces.

2. *Evidence as Heyting-valued semantics.* At the evidence layer, K&S-style rules persist but Boolean equalities can weaken to inequalities, because the evidence lattice is Heyting, not Boolean.

We bridge K&S additive regraduations to valuation-algebra factor graphs, enabling variable elimination over K&S-valued factors.

See `Mettapedia/ProbabilityTheory/KnuthSkilling/Bridges/ValuationAlgebra.lean`.

Part IV

Compiled Inference and Tractable Fragments

Chapter 10

Σ-Guarded Rewrites

The gap between the complete world-model semantics and practical inference is bridged by *compiled rewrites*: derived rules that are correct under explicit structural assumptions.

10.1 The Rewrite Pattern

A Σ-guarded rewrite has the form:

$$\frac{\Sigma \vdash \mathrm{ScreenOff}(A, B, C) \quad \Sigma; \Gamma \vdash \texttt{link}(A, B) \Downarrow e_{AB} \quad \cdots}{\Sigma; \Gamma \vdash \texttt{linkCond}([A, B], C) \Downarrow e_{AC}}$$

with the side-condition that $\mathsf{strength}(e_{AC})$ equals the result of applying the appropriate PLN formula to the extracted strengths.

The key features:

1. The side-condition context Σ is *explicit*—it appears in the judgment, not hidden in documentation.

2. The rewrite operates at the *strength level*—it tells you the correct probability answer, given the assumptions.

3. The assumptions are *checkable*—for a BN model class, they reduce to d-separation queries on the DAG.

10.2 The Xi Rule Registry

The codebase maintains a registry of compiled rewrites:

- Exact-track rules: deduction, source/induction, sink/abduction, revision.
- Each rule carries: screening-off side conditions, OSLF atom bridges, typed BN constructor lifts (chain/fork/collider), and coordinate-specialized endpoints (lower/upper/credibility/width/strength).

See `Mettapedia/Logic/PLNXiRuleRegistry.lean` and `Mettapedia/Logic/PLNBNCompilation.lean`.

10.3 Compiled Tactics vs. Foundational Semantics

It is worth pausing to be explicit about the relationship between compiled rewrites and the foundational calculus.

A compiled rewrite is *admissible*: it can be derived from the foundational rules plus the structural assumptions in Σ. It is not a new axiom. It is a theorem that, under stated conditions, a certain query can be answered by combining the answers to simpler queries.

The advantage of presenting rules as compiled tactics rather than primitive laws is that the assumptions become visible. When a PLN deduction rule "works" in practice, it is because the implicit BN structure satisfies the screening-off conditions. When it "fails" (produces incorrect probabilities), it is because those conditions are violated. ξPLN makes this explicit. These are not ad hoc patches to rescue a broken rule; they are the actual theoremic scope conditions under which the classical extensional formulas are exact.

Chapter 11

Bayesian-Network World Models and Compiled BN Inference

The complete world-table `JointEvidence` is exponential in n. To scale, we restrict the world-model class while preserving correctness relative to the restriction.

11.1 BN World-Model Class

A Bayesian-network world model consists of:

- A directed acyclic graph (DAG) specifying the factorization.
- Local conditional probability tables (CPTs) with Dirichlet evidence (counts) per CPT row.
- Revision = addition of local Dirichlet counts (conjugate update).
- Query = exact inference by variable elimination or junction tree.

The DAG structure provides the structural assumptions (Σ) needed for compiled rewrites. D-separation in the DAG determines which conditional independence assumptions hold, and these determine which PLN rules are applicable.

See `Mettapedia/Logic/PLNBayesNetWorldModel.lean`.

11.2 Exact BN Query Bridge

The BN world-model layer provides an evidence plumbing stack:

- A Boolean BN CPT query type `CPTQuery` (node + parent configuration).
- A CPT posterior state `CPTState` storing `Evidence` per CPT entry.
- An additive projection from `JointEvidence` to CPT evidence by marginalization (summing compatible worlds), which commutes with revision.

11.3 Chain, Fork, and Collider Exactness

The three fundamental BN motifs each yield exactness theorems for the corresponding PLN rule. Recall the PLN deduction strength formula (Chapter 5):

$$s_{AC} = s_{AB} \cdot s_{BC} + (1 - s_{AB}) \cdot \frac{s_C - s_B \cdot s_{BC}}{1 - s_B}$$

Theorem 11.1 (Chain BN deduction exactness (Conditional))**.** *In a chain BN $A \to B \to C$ with CPT-based local probabilities, the screening-off conditions*

$$P(C \mid A \cap B) = P(C \mid B), \qquad P(C \mid A \cap B^c) = P(C \mid B^c)$$

hold by d-separation. Under positivity ($P(B \mid A) > 0$, $P(B) \in (0,1)$), the PLN deduction formula computes the exact conditional:

$$s_{AC} = P(C \mid A) = \sum_b P(C \mid B = b) \cdot P(B = b \mid A)$$

The key mechanism: the CPT row for C depends only on B's value (sink property), and the joint weight factorizes as $w(c, r) = P_{\mathrm{CPT}}(c \mid \mathrm{pa}(C)) \cdot \prod_{v \neq C} P_{\mathrm{CPT}}(v \mid \mathrm{pa}(v))$, where the second factor is independent of c.

Theorem 11.2 (Fork BN deduction exactness (Conditional))**.** *In a fork BN $A \leftarrow B \to C$, the common-cause B screens off A from C: $P(C \mid A, B) = P(C \mid B)$. Under positivity, the PLN deduction formula again yields the exact $P(C \mid A)$.*

Theorem 11.3 (Collider screening-off (Conditional))**.** *In a collider $A \to B \leftarrow C$, A and C are marginally independent but become dependent when conditioning on B. The PLN abduction formula applies with modified side conditions.*

These "fast rule exactness" results show that classic PLN formulas are not approximations but exact computations within the declared model class. The screening-off and positivity hypotheses are part of the mathematical statement, and the local-completeness no-go theorem explains why they cannot be erased in the unrestricted setting.

See `Mettapedia/Logic/PLNBayesNetFastRules.lean` and `Mettapedia/Logic/PLNXiDerivedBNRules.lean`.

11.4 Class-Packaged BN Discharge

For practical use, the individual screening-off and positivity conditions are packaged into class-level discharge:

```
allDiscreteCPTLocalMarkov_of
wmqueryeq_of_dsep_discharge_via_soundness_allLocalMarkov
chain_screeningOff_wmqueryeq_of_dsep_allLocalMarkov
fork_screeningOff_wmqueryeq_of_dsep_allLocalMarkov
collider_screeningOff_wmqueryeq_of_dsep_allLocalMarkov
```

This means: given a BN model class satisfying local Markov properties, the screening-off conditions for chain, fork, and collider patterns are automatically discharged.

See `Mettapedia/Logic/PLNBNLocalMarkovPackages.lean`.

11.5 Worked Proof-Carrying Contraction

The natural question at this point is whether anything recognizably PLN-like remains at the operational level. The answer is yes, but in a corrected form. By *proof-carrying contraction* I mean a compiled rewrite together with its explicit side-condition package Σ and its theorem provenance. The semantic factor graph is still the truthmaker; the PLN rule is the local contraction that is allowed to fire only when Σ is discharged. What disappears is naked truth-value propagation without a declared model class.

Our canonical worked micro-example is the binary BN

$$A \leftarrow H \rightarrow B \rightarrow C \rightarrow D$$

with

$$P(H{=}1) = \tfrac{1}{5},\ P(A{=}1 \mid H{=}1) = \tfrac{9}{10},\ P(A{=}1 \mid H{=}0) = \tfrac{1}{10},$$

$$P(B{=}1 \mid H{=}1) = \tfrac{4}{5},\ P(B{=}1 \mid H{=}0) = \tfrac{1}{5},\ P(C{=}1 \mid B{=}1) = \tfrac{17}{20},\ P(C{=}1 \mid B{=}0) = \tfrac{3}{20},$$

$$P(D{=}1 \mid C{=}1) = \frac{9}{10}, \qquad P(D{=}1 \mid C{=}0) = \frac{1}{10}.$$

The semantic baseline is exact factor-graph elimination on this declared model. The compiled PLN-style contractions then recover the same answers in the exact fragment:

- Fork-style contraction ('source rule'): from $A \leftarrow H \rightarrow B$, the exact answer is
$$P(B{=}1 \mid A{=}1) = \frac{8}{13} \approx 0.615.$$
- Chain-style contraction: using the derived $A \Rightarrow C$ query through the B-mediated exact fragment,
$$P(C{=}1 \mid A{=}1) = \frac{151}{260} \approx 0.581.$$
- Chain-style contraction again through C:
$$P(D{=}1 \mid A{=}1) = \frac{367}{650} \approx 0.565.$$

The important point is not just the numbers. Each derived step is packaged as

$$(\text{query}, \text{value}, \Sigma, \text{provenance}),$$

not as a bare derived edge. In the Lean demo this provenance points back to the already-proved exact endpoints `PLNEndToEnd.forkFormulaExact`, `PLNEndToEnd.chainFormulaExact`, and the corresponding Σ-guarded rewrite surfaces. See `fork_contraction_matches_exact_semantics`, `chain_contraction_C_matches_exact` `chain_contraction_D_matches_exact_semantics` in `Mettapedia/Logic/PLNProofCarryingContractionDemo.lean`.

The same file also contains two negative controls, because a worked positive example is only convincing if it does not quietly smuggle back the old unrestricted semantics.

First, perturb the fixture so that C depends weakly on A as well as B:

$$P(C{=}1 \mid A{=}0, B{=}0) = \frac{3}{20},\ P(C{=}1 \mid A{=}1, B{=}0) = \frac{1}{4},$$

$$P(C{=}1 \mid A{=}0, B{=}1) = \frac{17}{20},\ P(C{=}1 \mid A{=}1, B{=}1) = \frac{9}{10}.$$

Now the exact semantic answer becomes

$$P(C{=}1 \mid A{=}1) = \frac{13}{20} = 0.65,$$

while the naive screened-off contraction gives only

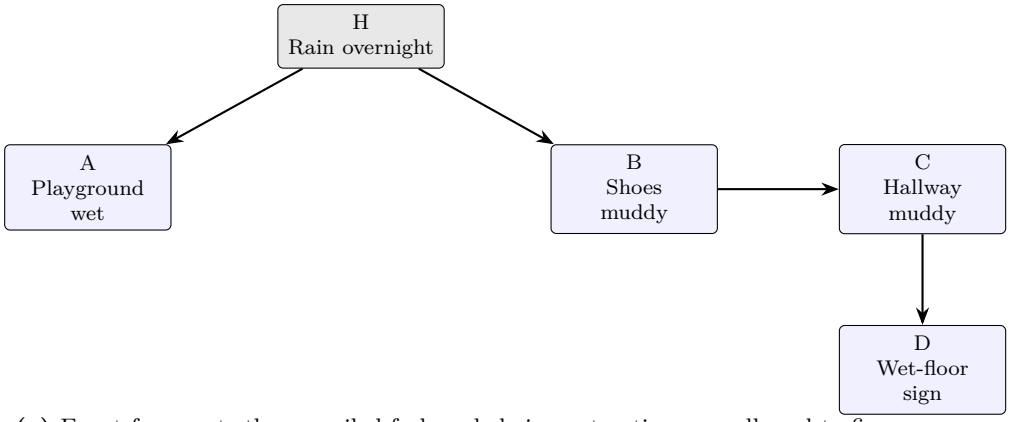
$$\frac{133}{221} \approx 0.602.$$

So the gate matters: when Σ is not discharged, the exact compiled chain contraction should not fire. See `soft_gate_blocks_exact_contraction` in `Mettapedia/Logic/PLNProofCarryingContractionDemo.lean`.

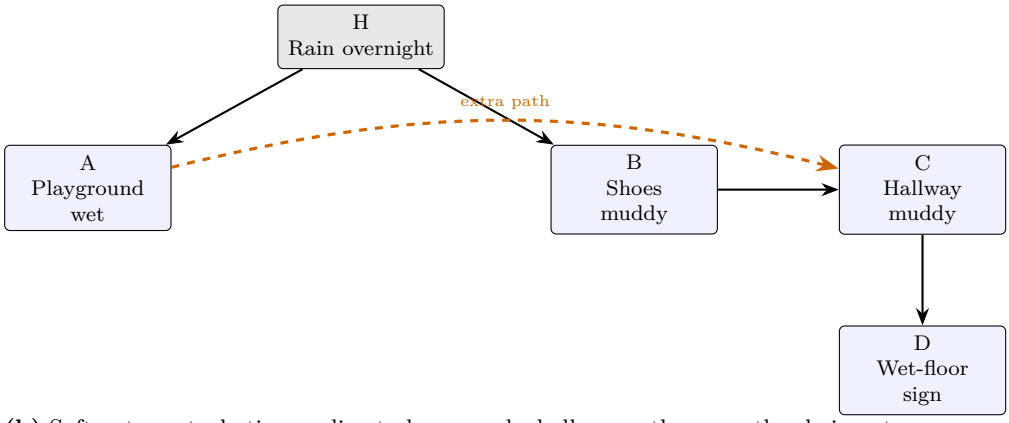
Second, a collider negative control keeps the old abduction story honest. For the OR-gate collider $A \to C \leftarrow B$ with $P(A{=}1) = P(B{=}1) = \frac{1}{2}$, the old naive abduction formula yields $\frac{2}{3}$, while the exact semantic answer remains $\frac{1}{2}$. So the refounded system does *not* license free abduction-style link creation merely because a local pattern "looks like" a classic PLN rule. See `collider_negative_control_blocks_naive_abduction` in `Mettapedia/Logic/PLNProofCarryingContractionDemo.lean`.

This is the corrected operational reading of PLN rule firing. The old intuition survives, but as admissible, Σ-guarded local contraction inside a declared world-model class, not as unconstrained link propagation.

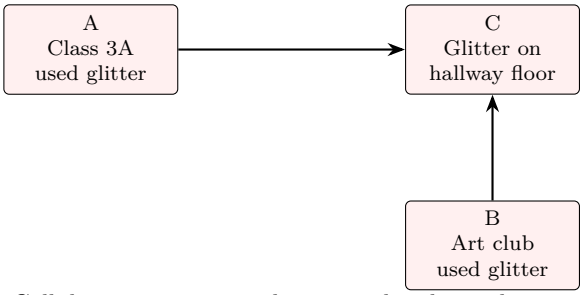
Figure 11.1 gives the same story in a simple school-grounded picture. The left panel is the exact fragment; the middle panel shows the extra path that blocks the naive chain contraction; the right panel is the collider negative control.



(a) Exact fragment: the compiled fork and chain contractions are allowed to fire.



(b) Soft-gate perturbation: a direct playground→hallway path means the chain gate is no longer discharged.



(c) Collider negative control: seeing the glitter does not license naive free abduction from one source to the other.

Figure 11.1: Elementary-school grounding of the proof-carrying contraction demo. Left: rain explains both a wet playground and muddy shoes, so the fork/chain contractions match exact semantic inference. Middle: if the wet playground can also muddy the hallway directly, the naive screened-off contraction is blocked. Right: two possible glitter sources feeding one hallway clue form a collider, so the old unrestricted abduction intuition must be guarded.

11.5.1 Probabilistically Guarded Admissibility on Real BN Benchmarks

The toy fixture above establishes the theorem-backed exact fragment in a fully worked micro-example. The next question is whether the same discipline helps on canonical benchmark BNs. Pathfinder is a classic Bayesian expert system for diagnosing lymph-node diseases from pathology findings,[13] and Hailfinder is a Bayesian severe-weather forecasting model for northeastern Colorado built from meteorological variables and expert judgment.[1] We use the canonical BIF encodings distributed through the bnlearn Bayesian Network Repository as the exact BN truthmaker for these experiments.[30] We therefore evaluate Pathfinder and Hailfinder against exact variable elimination from the BIF truthmaker, while keeping the derived object discipline explicit:

Throughout this subsection, we use three plain-English labels for the side-condition context Σ:

- **context-complete**: the carried Σ contains the structural context needed to license the rewrite exactly;
- **context-missing**: some required context has been omitted, so the exact rewrite is no longer licensed;
- **context-uncertain**: the required context is only partially supported, so bounded or operational control layers become relevant.

$$(\text{query}, \text{value}, \Sigma, \text{provenance}, g?, \delta?, \text{fallback?}).$$

The guarded layer is an inference-control discipline on top of exact BN semantics, not a replacement for that semantics.

Benchmark	Domain	Primary artifact	What the variables mean	Why we use it here
Hailfinder	Severe summer hail forecasting in northeastern Colorado	A hand-built Bayesian weather-forecast BN with expert CPTs; `hailfinder.csv` is secondary to `hailfinder.bif`	Meteorological conditions, scenario summaries, and forecast targets	Readable positive lane: exact local chaining still works on a real forecast network.
Pathfinder	Lymph-node pathology diagnosis	A hand-built Bayesian network with expert CPTs and a disease-class root (`Fault`)	One latent diagnosis plus many coded pathology findings in the benchmark release	Hidden-context stress test: can proof-carrying contractions survive when crucial diagnostic context is omitted?

Table 11.1: What the two benchmark networks are about. In both cases the primary benchmark object is the Bayesian network itself, not a generic flat table.

Three-mode guarded admissibility. Context-complete Σ gives exact firing. Context-missing Σ blocks the contraction. Context-uncertain Σ moves to a bounded semantic guard or to an explicit soft fallback layer.

Mode	Status and returned object
Hard-gated exact	*Semantic, theorem-backed.* Fire only when Σ is discharged. The returned query answer keeps its carried Σ and provenance; there is no reusable bare derived edge.
Bounded violation	*Semantic guard.* Return an estimate plus an explicit certified radius or interval against the BN truthmaker.
Soft-guarded mixture	*Operational / governance.* Return $\hat{p} = g \cdot p_{\text{rule}} + (1-g) \cdot p_{\text{fallback}}$ with explicit fallback. This is useful for ranking, abstention, and trigger logic, but it is not automatically the semantic posterior.

Task framing. Methodologically, the real question is no longer merely "can we classify admissibility?" It is *topology-conditioned latent-context inference control*: given a source condition and a target query inside a declared BN truthmaker, should the controller continue by exact rewrite, softened

continuation, bounded continuation, context reveal, local exact fallback, or abstention? Topology matters because it tells us which kinds of approximation assumptions are likely to survive transport through the graph and which are likely to fail. Hailfinder is the readable positive lane for this task. Pathfinder is the hostile hidden-context lane.

Hailfinder first: readable exact contraction. Hailfinder is the cleaner forecasting-structured benchmark. The exact variables are meteorological conditions and forecast targets, so the benchmark reads like an actual forecast/update chain rather than an opaque medical codebook. This is the best first example for readers: it teaches what a proof-carrying local contraction looks like before the topology becomes adversarial.

The exact forecast chain

$$P(\texttt{MountainFcst=SIG} \mid \texttt{AMInstabMt=Strong}) = 0.5$$

fires cleanly, and a longer forecast/update path still matches exact BN semantics:

$$P(\texttt{CapChange=Increasing} \mid \texttt{CldShadeOth=Cloudy}) = 0.508284\ldots$$

The negative control remains equally important. In the selected collider motif, the exact semantic answer is $0.23595\ldots$, but the hard gate still blocks unrestricted reuse; bounded mode returns the interval $[0.23595\ldots, 0.27959\ldots]$, and soft mode returns an explicit operational mixture with semantic fallback. So Hailfinder says there is still a real exact forecasting-style chaining fragment, while the blocked cases preserve the "yes, but not everywhere" boundary.

Pathfinder next: the same calculus under hostile topology. Pathfinder is the stronger context-heavy benchmark. In the current BIF release it has 109 nodes and 195 arcs; `Fault` has 63 states and 106 outgoing children, and 56 nodes have multiple parents. So omitted context is not a side issue here. It is the main structural fact of the dataset. In the selected motifs, `Fault` is the latent disease hypothesis and the `F…` variables are benchmark-coded pathology findings from the network file. For exposition, it is often better to read the main fork/chain path as *observation* → *bridge* → *downstream consequence*, with the actual BN names supplied by Table 11.2.

The main Pathfinder stress question is therefore not "does local chaining ever work?" but "what happens to the same local contraction when hidden diagnostic context is omitted, partially restored, or completed?" In the

widened suite the same path family is explicitly tracked across five regime types: *context-missing*, *alt-support-only*, *hub-only*, *context-pair*, and *oracle-context-complete*. That is the second-order object in practice: uncertainty over which context/admissibility regime the step is really in.

Network	Variable	Reader gloss	Role in the selected motifs
Hailfinder	`AMInstabMt`, `InsInMt`, `MountainFcst`	Morning mountain instability, mountain instability, mountain forecast	The clean local forecast-update chain.
Hailfinder	`CldShadeOth`, `AreaMesoALS`, `CompPlFcst`, `CapChange`	Cloud shading, mesoscale-area summary, composite plains forecast, capping change	The longer readable forecast/update path.
Hailfinder	`AMCINInScen`, `CapInScen`	Scenario-specific convective inhibition and cap state	The collider-style negative boundary.
Pathfinder	`Fault`	Latent disease hypothesis / diagnosis class	The hidden diagnostic context that often has to stay in Σ.
Pathfinder	`F3`, `F4`, `F103`	Benchmark-coded observed pathology clues	Upstream evidence nodes in the selected fork and long-path motifs.
Pathfinder	`F97`, `F105`	Benchmark-coded intermediate pathology factors	Intermediate shared nodes through which the admissible contraction travels.
Pathfinder	`F101`, `F102`, `F104`	Benchmark-coded downstream pathology targets	Queried consequences used to test exactness and hidden-context sensitivity.

Table 11.2: Mini glossary for the variables actually used in the current benchmark motifs. Pathfinder keeps the compact benchmark coding from the released network; Hailfinder uses more readable meteorological names.

In the selected fork motif, keeping `Fault` explicit in Σ yields an exact hard-gated contraction:

$$P(F104\texttt{=x_50} \mid F103\texttt{=Almost_all_poor_def}, \texttt{Fault=T_immunob_lrg}) = 0.4.$$

But if the same motif is evaluated in the *context-missing* regime, with `Fault` omitted from Σ, the exact semantic answer drops to $0.14998\ldots$, and the hard gate blocks. The bounded mode then returns the rule estimate $0.31236\ldots$ together with the certified interval $[0.14998\ldots, 0.47474\ldots]$, while the soft-

gated operational layer returns $0.28599\ldots$ with explicit fallback to the exact semantic query. This is the intended result: Pathfinder does not show that "free PLN chaining works" on a large medical BN; it shows that proof-carrying local contraction survives only when the hidden diagnostic context is carried, and that context-erasing contractions must be blocked or degraded gracefully.

Case	Exact BN	Guarded output	Why it matters
Hailfinder exact forecast chain	0.500	Hard fires. Bounded $\delta = 0$. Soft $g = 1$.	There is still a clean exact chaining fragment for forecasting-style paths.
Hailfinder forecast collider negative	0.236	Hard blocks. Bounded 0.258 in $[0.236, 0.280]$. Soft 0.257.	The guarded layer preserves the no-free-abduction boundary even in the forecasting BN.
Pathfinder context-complete fork	0.400	Hard fires. Bounded $\delta = 0$. Soft $g = 1$.	When `Fault` stays in Σ, PLN-style contraction is exact.
Pathfinder context-missing fork	0.150	Hard blocks. Bounded 0.312 in $[0.150, 0.475]$. Soft 0.286.	Omitting `Fault` changes the answer sharply; carried context is necessary.

Table 11.3: Compact guarded-admissibility summary for the current real BN benchmarks. Exact BN semantics remains the truthmaker in every row.

The next question is why adventurous continuation sometimes wins the original broad Pathfinder task and sometimes does not. Here the most useful abstraction is no longer a single guard score. It is a topology-conditioned latent-context controller. Let C denote unresolved hidden context, typically including `Fault` and, in some paths, an auxiliary support variable. Before any reveal action, the target is the broad query $P(Q \mid E)$. After revealing one context value c, the target becomes the refined query $P(Q \mid E, C{=}c)$. These are different tasks. Exact reveal wins the refined task because it conditions on the right branch explicitly. But it need not minimize error against the original broad task, because the broad query marginalizes over the still unresolved context:

$$P(Q \mid E) = \sum_c P(Q \mid E, C{=}c)\, P(C{=}c \mid E).$$

A controlled soft continuation,

$$\hat{p}_{\text{soft}} = g\, p_{\text{rule}} + (1-g)\, p_{\text{fallback}},$$

can therefore be a better *operational* estimator for the original broad query than prematurely collapsing onto a single revealed branch, even though we do not yet claim that the current learned g is itself the semantic posterior. But the widened Pathfinder suite shows that this is not universal. The topology label is precisely what tells us which approximation assumptions are more or less plausible. In hub-dominated cases, omitted-context mass is often concentrated around one dominant diagnostic direction, so direct continuation can stay close to the unresolved broad marginal. In loopy-dense or partial-support cases, several live alternatives can remain active, and then softened or marginalization-aware continuation becomes more attractive. So the current benchmark-side explanation is already informative: topology is not a decorative covariate but a guide to which kinds of assumptions do or do not transport well. The new point is that the finite regime-mixture layer is now theorem-backed in Lean, in `PLNRegimeMixtureTheorems.lean`, `PLNRegimeMixtureBenchmarkBridge.lean`, `PLNWorldModelRegimeAdmissibility.lean`, and `PLNProbHOLPlannerBridge.lean`. If R is a finite regime space, w_r are posterior regime weights, and q_r are branch query values, then the unresolved broad-query predictor is

$$m = \sum_{r \in R} w_r q_r.$$

For any designated branch r_0, the direct-continuation approximation error obeys the residual-mass bound

$$|m - q_{r_0}| \le (1 - w_{r_0}) \cdot \max_{r \neq r_0} |q_r - q_{r_0}|.$$

Under squared loss, the constant-predictor risk decomposes as

$$\sum_{r \in R} w_r (x - q_r)^2 = \underbrace{\sum_{r \in R} w_r (m - q_r)^2}_{\text{mixture variance}} + (x - m)^2,$$

so the unresolved mixture predictor m is optimal among constant predictors, and exact reveal is preferred once

$$\text{gain}_{\text{reveal}} = \text{mixture variance} - c_{\text{reveal}} > 0.$$

This is already enough to turn the higher-order direct/soft/reveal split into a theoremic finite-regime statement: direct continuation is justified when posterior mass concentrates tightly around one branch; soft continuation is justified because the mixture predictor minimizes unrevealed squared-loss risk;

and reveal/context completion is justified once value-of-information beats reveal cost. What remains empirical at present is the *topology-conditioned proxy story*: labels such as "hub-dominated" or "loopy-dense" are still benchmark-side ways of predicting when concentration, spread, and latent support are likely to take those theoremically relevant shapes.

Three higher-order claims. The current benchmark supports three proposition-shaped claims that keep the semantics/control split clean.

1. **Objective mismatch under reveal.** Revealing hidden context changes the target from a broad marginalized query to a refined conditional query, so reveal can improve refined fidelity while worsening broad-query utility.

2. **Topology-conditioned continuation preference.** Hub-dominated slices often reward direct continuation; partial-support and some loopy-dense slices leave more live alternatives and therefore make softened continuation more competitive.

3. **Higher-order flattening at decision time.** Even when admissibility and trust are modeled explicitly, action choice need not recurse through an ever taller explicit tower; it can flatten to expected utility, interval dominance, or abstention at decision time.

We now make that higher-order split explicit. The second-order variable G_{step} means "this local contraction is admissible enough in the current structural regime", so the live second-order object is best read as a regime posterior $\pi(r \mid \text{local state})$ over hidden-context/admissibility regimes rather than as one free-floating confidence scalar. A third-order trust layer then records how much confidence to place in the current model of G_{step} for the present topology/context class. The point is not infinite regress for its own sake, but an explicit separation between object-level belief, admissibility belief, and trust in the admissibility model itself. Fourth-order and higher concerns are then handled, for the benchmark, as flattening or robustness questions rather than as an endlessly explicit tower. In the widened sweep, Hailfinder *context-complete* paths have mean third-order meta-trust 0.819 with reveal pressure 0.147, while Pathfinder has meta-trust 0.563 with reveal pressure 0.761 in the *context-missing* regime, 0.603/0.545 in the *alt-support-only* regime, 0.764/0.184 in the *context-pair* regime, and 0.798/0.159 in the *oracle-context-complete* regime. So the planner is not only asking "can I chain?" but also "how much do I trust my own admissibility estimate

here, in this topology and context class?" In the current architecture these planner-facing second-/third-order quantities are intended as empirical control shadows of the semantic ProbHOL → belief-process → planner bridge, not as replacement semantics.

Regime slice	Meta-trust	Reveal pressure	Best broad-query action	Best refined-query action
Hailfinder forecast chain, context-complete	0.819	0.147	`Regime-mixture` / exact chain, MAE 0.0000.	`Regime-mixture` / exact chain, MAE 0.0000.
Pathfinder hub-dominated, context-missing	0.590	0.769	`Anchored regime-mixture`, MAE 0.0090 (one-shot).	Context-complete / exact fallback, refined MAE 0.0000.
Pathfinder hub-dominated, alt-support-only	0.555	0.596	`Anchored regime-mixture`, MAE 0.2584.	Context-complete / exact fallback, refined MAE 0.0000.
Pathfinder loopy-dense, context-missing	0.548	0.757	`Anchored regime-mixture`, MAE 0.0489 (one-shot).	Context-complete / exact fallback, refined MAE 0.0000.
Pathfinder loopy-dense, oracle-context-complete	0.926	0.074	`Anchored regime-mixture`, MAE 0.2646.	`Rule-only`, refined MAE 0.0087.

Table 11.4: Compact topology-conditioned higher-order split in the widened benchmark sweep. One-shot broad-query winners are often anchored regime-mixture actions, while refined-query winners still mostly come from explicit context completion or exact fallback.

Table 11.4 also shows why the higher-order story is genuinely interesting rather than merely cautious. The current benchmark no longer supports the simpler claim that "soft continuation wins the hardest broad-query cases". Instead, the better statement is now more precise. After anchoring the explicit regime-mixture continuation to the current broad-query branch, the strongest *one-shot* broad-query action is often the anchored regime-mixture policy, including the widened Pathfinder *context-missing* rows. Mechanistically, the hub-dominated slices look like unresolved broad queries whose mass is already concentrated in one main diagnostic direction, so a small anchored mixture smooths the remaining uncertainty without collapsing to one refined branch. The alt-support and loopy slices keep more hidden-context alternatives live, which is why structured regime averaging helps more than blunt `rule-only` continuation at the one-shot level. But the stricter *higher-order chain-composition* benchmark still leaves several multi-step *context-missing* slices to `rule-only` chain continuation. So the present

result is not "the planner is solved", but "the broad/refined objective split is real, the best adventurous one-shot action depends on topology and hidden-context regime, and the best full-chain action can still be different". In this sense, topology is functioning as a compact summary of which kinds of independence, concentration, and branch-mixing assumptions are likely to work in a local region of the graph.

To keep the setup fully auditable, we make the protocol explicit here. The pipeline is: `scripts/wm_pln_pathfinder_context_regimes.py` → `scripts/wm_pln_second_ord` → `scripts/wm_pln_third_order_trust.py` → `scripts/wm_pln_regime_mixture_eval.py` → `scripts/wm_pln_reveal_voi_eval.py` → `scripts/wm_pln_higher_order_chain_composition.p` The corresponding stable artifacts are written under `results/second\_order\_admissibility/`, including `pathfinder_context_regimes.csv`, `second_order_admissibility.csv` `third_order_trust_summary.csv`, `regime_mixture_eval_winners.csv`, `reveal_voi_eval_summary` `multi_support_revision_summary.csv`, and `higher_order_chain_composition_summary.csv`.

The one-shot continuation policy set in this pass is `force_rule_only_continue`, `force_soft_continue`, `marginalization_aware_soft_continue`, `explicit_regime_mixture_conti` `force_reveal_context`, `force_local_exact_fallback`, and `adaptive_topology_broad_planner`. The full-chain policy set is `exact_only_chain`, `rule_only_chain`, `soft_chain`, `explicit_regime_mixture_chain`, `reveal_then_exact`, `fallback_chain`, and `adaptive_chain`.

The scoring function used by the regime-mixture and chain benchmarks is

$$\text{score} = 1 - 2 \cdot \text{primary-error} - 0.08 \cdot \text{plan-cost} - \lambda_{\text{reveal}} \cdot \#\text{reveals},$$

with $\lambda_{\text{reveal}} = 0.12$ for broad-query utility, 0.04 for refined-query fidelity, and 0.08 for the balanced objective. Action-cost parameters are

$$\begin{aligned} \text{hard} &= 0.8 + 0.6 \cdot \text{edges}, \\ \text{guarded} &= 0.6 + 0.4 \cdot \text{edges}, \\ \text{fallback} &= 2.0 + 0.8 \cdot \text{edges}, \\ \text{reveal} &= 0.7 \cdot \#\text{missing-required-context-vars}. \end{aligned}$$

Pathfinder topology labels are assigned directly from local graph statistics in the benchmark scripts: *hub-dominated* if `Fault` is on-path or max node degree ≥ 50; *loopy-dense* if downstream multi-parent count ≥ 2 or mean local branching ≥ 4. The widened Pathfinder suite in this pass is 16 path families (4 anchors + 12 discovered) and 77 regime rows (16 context-missing, 16 hub-only, 16 alt-support-only, 16 oracle-context-complete, 13 context-pair). Importantly, those 12 discovered paths come from a deterministic curated shortlist in `wm_pln_pathfinder_context_regimes.py`; this is

substantially wider than the original four-path slice, but it is still not an exhaustive Pathfinder crawl.

The guard-model sweep now adds broader motif coverage and a calibrated cross-network control layer. The current dataset has 207 evaluated rows over 142 motif groups (62 Hailfinder, 145 Pathfinder). As a coverage result, Pathfinder is exactly where we expected the stress to appear: every Pathfinder row requires extra context support, and 55.2% of rows are context-missing cases where contraction is blocked. For cross-network operation, a conformal shrink layer is now applied to the calibrated guard score before policy decisions; this materially improves calibration in the harder Hailfinder→Pathfinder transfer setting while preserving the semantic contract that exact BN answers remain the truthmaker.

For these real-network cases, the bounded mode is currently *evaluation-certified* from the exact BN truthmaker rather than derived from a general theoremic bound calculus over arbitrary BNs. That limitation should be read as a strength of the presentation rather than a weakness: the semantic anchor stays exact, and the guarded layer is explicit about where it is theorem-backed, where it is certified by comparison against the truthmaker, and where it is merely operational.

Reproducibility block. *Scripts.* `scripts/wm\_pln\_pathfinder\_prob\_guarded.py` `scripts/wm\_pln\_hailfinder\_prob\_guarded.py` `scripts/wm\_pln\_guard\_xgb\_dataset.py` `scripts/wm\_pln\_guard\_xgb\_train.py` `scripts/wm\_pln\_guard\_xgb\_eval.py`

Results. `results/pathfinder/pathfinder\_prob\_guarded.csv` `results/hailfinder/hailfinder\_prob\_guarded.csv` `results/guard\_model/guard\_dataset.csv` `results/guard\_model/metrics.json` `results/guard\_model/motif\_coverage\_by\_dataset.csv` `results/guard\_model/motif\_coverage\_by\_dataset\_motif.csv`

Lean demo. `Mettapedia/Logic/PLNProbGuardedAdmissibilityDemo.lean`

Lean regression. `Mettapedia/Logic/PLNProbGuardedAdmissibilityRegression.lean`

11.5.2 Hetionet / Rephetio: the next flagship benchmark

At this point the project has three ingredients in place. First, the Pathfinder/Hailfinder lane already gives us a real small-to-medium benchmark story for exact contraction, guarded continuation, reveal as value of information, multi-support revision, and higher-order chain composition. Second, Chapter 11 now exposes the semantic ProbHOL → belief-process → planner-

shadow bridge as the canonical waist of the architecture. Third, we now have a concrete large-scale benchmark candidate in hand: the Hetionet / Rephetio drug-repurposing stack.[16, 15] The motive for moving there is simple. Hetionet is large, typed, path-rich, and explanation-oriented, so it is a better stress test of higher-order chaining than another toy relational dataset. It is also already rich enough that there is no single clean leaderboard; instead there is a cluster of benchmark traditions with different task definitions, path semantics, explanation targets, and reproducibility caveats.[24, 19, 14, 26, 5, 27, 21]

The dataset itself also needs to be described clearly, because the benchmark object is not one flat table. Hetionet is the heterogeneous biomedical graph; Rephetio is the benchmark-facing drug-repurposing layer built on top of it. In the current public browser-table bundle we have a typed graph with 47,031 nodes, 2,250,197 relationships, and 24 edge types, together with benchmark-facing tables for 1,538 compounds, 136 diseases, 1,206 metapaths, 1,000 compound-conditioned prediction tables, and 136 disease-conditioned prediction tables.[16, 15] This is exactly the shape we want: small enough to inspect, large enough to matter, and rich enough that typed multi-hop support families are not an afterthought.

The flagship query for this benchmark is

$$\texttt{treats}(\textit{compound}, \textit{disease}).$$

The methods/motives follow directly from the work already in hand. Pathfinder and Hailfinder taught us that the interesting control problem is not merely whether a local step is exact, but how to act when context is incomplete, several support families are live, and reveal can change the target from a broad unresolved query to a refined conditional one. Hetionet / Rephetio is where that question becomes typed and large-scale. For example, metapaths such as `CbGaD` (Compound–binds–Gene–associates–Disease), `CrCtD` (Compound–resembles–Compound–treats–Disease), and `CiPCiCtD` (Compound–includes–Pharmacologic Class–includes–Compound–treats–Disease) already look like natural higher-order support families rather than accidental path fragments.

The planned protocol is therefore:

1. **One-shot regime-mixture continuation.** Treat typed metapath/-support families as second-order regimes and compare `rule-only`, explicit regime-mixture continuation, reveal, and fallback on the broad `treats(c,d)` task.

2. **PLN-style multi-support revision.** Combine several live support

families for the same query with explicit overlap penalties or residual ignorance terms instead of pretending free independence.

3. **Reveal as value of information.** Ask when inspecting an additional mechanism family or context slice improves refined fidelity enough to justify its cost, instead of silently changing the target.

4. **Higher-order chain composition.** Evaluate whether higher-order guarded continuation survives over interpretable compound–gene–disease and compound–compound–disease chains, not just one-shot scoring.

At the semantic level, the split is the same as in the current benchmark lane:

- **1st order**: support for `treats(c,d)`;
- **2nd order**: uncertainty over metapath/context/support regime;
- **3rd order**: trust in the regime estimator;
- **4th and beyond**: flattened to expected utility, interval dominance, or abstention at action time.

This means the next benchmark consumer is already aligned with the Chapter 11 architecture:

truth → hierarchical semantic probability → belief tracking → planner shadow.

What we are *not* claiming yet is equally important. This section does not report finished Hetionet results, and it does not treat the Rephetio browser tables as a semantic oracle by themselves. Rather, it states the next flagship benchmark plan now that the semantic ProbHOL → belief → planner waist is finally in place: the benchmark-side regime and trust fields should become principled projections of that bridge, not one more layer of bespoke scores.

11.6 Bridge Theorems: Naive Bayes and k-NN

The BN world-model framework connects PLN to two standard machine-learning methods.

Theorem 11.4 (Naive Bayes bridge (Core))**.** *For a Naive Bayes model (class variable C, features $F_1, \dots, F_n$ conditionally independent given C), the PLN tensor-strength computation equals the Naive Bayes posterior:*

$$\mathsf{strength}_{\otimes}(e_{F_1 \Rightarrow C}, \dots, e_{F_n \Rightarrow C}) = P(C \mid F_1, \dots, F_n)_{\mathrm{NB}}$$

See `PLN_tensorStrength_eq_nbPosterior` in `Mettapedia/Logic/PLNBayesNetInference.lean`.

Theorem 11.5 (k-NN bridge (Core)). *PLN's evidence-weighted relevance score equals the k-NN relevance function:*

$$\oplus\text{-pos}(e_1, \ldots, e_k) = \text{knnRelevance}(x)$$

where the e_i are evidence values from the k nearest neighbors.

See `PLN_hplusPos_eq_knnRelevance` in `Mettapedia/Logic/PremiseSelectionKNN\_PLNBridge.lean`.

These bridges show that PLN's evidence algebra is not an isolated formalism: it subsumes standard probabilistic classifiers as special cases.

11.7 Factor Graphs: Semantic vs. Operational

PLN has long used factor-graph and message-passing intuition. ξPLN separates two roles:

- **Semantic factor graphs.** Factors *are* the model: the posterior state is represented by a factorization. Variable elimination gives exact answers; complexity is controlled by treewidth. This is a *world-model representation*.
- **Operational factor graphs.** Factors are *rule schemas* (deduction, induction, etc.) whose local "potentials" are truth-value update formulas. This is *inference control*: it proposes which queries to ask, but does not itself constitute the semantics.

The two align when rule factors are *compiled* from a world-model class: each factor corresponds to a conditional in the BN, and message passing implements the correct queries.

Slice	Best broad one-shot policy	Broad MAE	Best full-chain policy	Main takeaway
Hailfinder forecast chain, context-complete	`Explicit regime-mixture`	0.0000	`Rule-only` chain (exact)	In the clean lane the higher-order controller collapses back to the exact fragment.
Pathfinder hub-dominated, context-missing	`Explicit regime-mixture`	0.0090	`Rule-only` chain	Anchored regime averaging helps at the one-shot level, but full multi-step composition still prefers blunt continuation.
Pathfinder loopy-dense, context-missing	`Explicit regime-mixture`	0.0489	`Rule-only` chain	One-shot smoothing helps, but loopy full-chain continuation still resists a universal adventurous policy.
Pathfinder multi-support revision	`Base-plus-support revision`	0.0197 / 0.0854	–	Combining hub/non-hub/pair supports is the clearest PLN-style positive result on the widened suite (hub-dominated / loopy-dense).
Reveal as value of information on Pathfinder context-missing	Continue for the broad query	Δ_{refined} = +0.715 / +0.067	Reveal only for the refined task	Reveal improves refined fidelity, but usually solves the wrong task for the original broad query and incurs large broad-query regret.

Table 11.5: Stabilized higher-order benchmark takeaways after explicit regime-mixture continuation, PLN-style multi-support revision, reveal-as-value-of-information evaluation, and higher-order chain composition. The two numbers in the multi-support and VOI rows correspond to the hub-dominated and loopy-dense Pathfinder slices respectively.

Slice	Rows	Groups	Hard	Blocked	ϵ-exact	ECE iso→conf	Brier iso→conf
Hailfinder coverage	62	62	0.629	0.371	0.758	0.205→0.205	0.188→0.188
Pathfinder coverage	145	80	0.448	0.552	0.634	0.074→0.074	0.064→0.064
Cross H→P guard transfer	145	–	0.448	0.552	0.634	0.323→0.190	0.307→0.237
Cross P→H guard transfer	62	–	0.629	0.371	0.758	0.119→0.110	0.118→0.117

Table 11.6: Motif-coverage and calibrated guard-transfer summary. "Hard" and "Blocked" are structural admissibility rates; ϵ-exact uses $\epsilon = 0.02$. The conformal guard is operational confidence shaping, not posterior semantics.

Paper	Role for this project	What it contributes	Main caution
Rephetio / Hetionet [16]	Canonical benchmark definition	Defines the graph, the `Compound-treats-Disease` target, and the metapath / DWPC setup.	Original setup is not itself a higher-order control benchmark.
PoLo-style logical multi-hop [24]	Closest chaining comparator	Shows that typed logical multi-hop reasoning on biomedical KGs is a real comparison class.	Not the same semantic / reveal / regime split as our WM lane.
XG4Repo [19]	Explainable path-completion comparator	Direct path-based Hetionet drug-repurposing benchmark with rule and RL comparisons.	Reported Hetionet test slices are small, so hard SOTA claims are shaky.
Hetnet connectivity search [14]	Unsupervised path-family discovery	Finds degree-adjusted important path families without a supervised gold label.	Analysis tool, not a full continuation controller.
REx [26]	Explanation-ranking comparator	Benchmarks explanatory path quality using Hetionet-style drug repurposing hypotheses.	Optimizes explanation quality rather than the broad/refined control tradeoff directly.
Bonner et al. [5]	Benchmark-sensitivity warning	Shows that ranking changes materially with split, hyperparameters, and seed.	Raw score-chasing without protocol control is misleading.
Reviews [27, 21]	Broader field orientation	Situate Hetionet among larger drug-repurposing KGs and method families.	Useful context, but not benchmark semantics by themselves.

Table 11.7: Research landscape for the next large-scale benchmark. The right reading is not "one SOTA leaderboard", but a cluster of benchmark traditions around path reasoning, explanation, and evaluation protocol.

Artifact	What it contains	Why it matters for HO chaining
Hetionet graph	47,031 nodes, 2,250,197 relations, 24 edge types	Semantic heterogeneous substrate and typed path space.
Rephetio compounds table	1,538 compounds, with 755 treatments and 390 palliations	Compound-side query inventory and positive label counts.
Rephetio diseases table	136 diseases, with the same treatment/palliation support counts	Disease-side query inventory and target coverage.
Rephetio metapaths table	1,206 metapaths of length 2–4, with ΔAUROC and fitted coefficients	Typed support families and regime candidates.
Per-compound predictions	1,000 tables, each scoring one compound against all diseases	Broad compound-conditioned support surface.
Per-disease predictions	136 tables, each scoring one disease against all compounds	Disease-conditioned scoring and explanation surface.

Table 11.8: What the Hetionet / Rephetio benchmark actually consists of in the current public bundle. The graph is the heterogeneous biomedical substrate; the Rephetio tables provide the benchmark-facing task surface.

Chapter 12

Error, Thresholds, and Transport Discipline

When truth values are transported through chains of inference, errors can accumulate. Chapter 8 of the PLN book discusses "error magnification." In ξPLN, this topic becomes precise through the formalization of threshold transport and view transport.

12.1 View Transport Under Query Equivalence

Definition 12.1 (Query equivalence)**.** Two queries q_1, q_2 are *equivalent in a world model* W when they yield the same evidence from every state:

$$\text{evidence}(W, q_1) = \text{evidence}(W, q_2)$$

Under query equivalence, all views transport:

Theorem 12.2 (View transport (Core))**.** *If* $q_1 \equiv_W q_2$*, then for any state* W*:*

$$\mathsf{strength}(W, q_1) = \mathsf{strength}(W, q_2) \tag{12.1}$$

$$\mathsf{confidence}(W, q_1) = \mathsf{confidence}(W, q_2) \tag{12.2}$$

$$[s_L, s_U]_{q_1} = [s_L, s_U]_{q_2} \tag{12.3}$$

and threshold acceptance transfers: if $\tau \leq \mathsf{confidence}(W, q_1)$*, then* $\tau \leq \mathsf{confidence}(W, q_2)$*.*

See `Mettapedia/Logic/PLNWorldModelCalculus.lean`.

12.2 Threshold Semantics and OSLF Bridge

The OSLF (Open Specification Language Framework) provides a typed truth-value bridge. Threshold acceptance becomes a formal judgment:

Definition 12.3 (Threshold acceptance)**.** For a threshold $\tau \in [0, 1]$, a query q, and a world-model state W with lookahead parameter κ:

$$\text{thresholdAccepted}(\tau, q, W) \iff \tau \leq \frac{n^+(W, q) + n^-(W, q)}{n^+(W, q) + n^-(W, q) + \kappa}$$

The end-to-end pipeline:

$$\text{selector} \rightarrow \text{rewrite} \rightarrow \text{query} \rightarrow \text{threshold} \rightarrow \text{acceptance}$$

is composed in a single theorem, ensuring that no intermediate step loses correctness guarantees.

See `Mettapedia/Logic/PLNErrorMagnificationGrounding.lean` and `Mettapedia/Logic/PLNCanonicalAPI.lean`.

12.3 Threshold Transport Boundaries

Not all threshold operations compose freely.

Theorem 12.4 (Double-damping gap (Boundary))**.** *When confidence passes through two damping stages (e.g., induction followed by threshold), the composed confidence is strictly lower than the correct single-stage value. Formally, there exist $c_1, c_2 \in (0, 1)$ such that:*

$$c_{\text{buggy}}(c_1, c_2) < c_{\text{correct}}(c_1, c_2)$$

This gap creates a threshold region where the double-damped formula rejects (incorrectly) while the correct formula accepts.

Further boundary results:

- **Disjunction threshold counterexample**: threshold acceptance for disjunctions does not follow from acceptance of disjuncts without total-order comparability of evidence values.
- **Diamond threshold failure**: modal diamond operators require finite branching for threshold transport.

See `Mettapedia/Logic/OSLFEvidenceSemantics.lean`.

12.4 Categorical Transport Layer

For advanced applications (governance, modal logic, institutional reasoning), the formalization includes a categorical transport layer: institution satisfaction conditions, hyperdoctrine Beck-Chevalley transport, and unified endpoints that connect Kripke/neighborhood modal semantics to the world-model layer.

See `Mettapedia/Logic/PLNWorldModelInstitution.lean` and `Mettapedia/Logic/PLNWorldModelCategoricalBridge.lean`.

Part V

Main Extensions

Chapter 13

Higher-Order Reduction

This chapter is now grounded in a real higher-order logic layer rather than treating higher-order PLN as only a satisfying-set reduction trick.

The formal architecture is:

$$\text{Set semantics} \longrightarrow \text{HOL} \longleftrightarrow \text{WM}, \qquad \text{FOL} \longrightarrow \text{HOL}, \qquad \text{Set theory} \longrightarrow \text{FOL} \longleftrightarrow \text{WM}.$$

The key point is that these arrows mean different things:

- **Set semantics → HOL** is semantic grounding: set-theoretic structures induce Henkin models for higher-order logic.
- **FOL → HOL** is a language embedding: first-order syntax is translated into the higher-order language.
- **HOL ↔ WM** is the world-model lens: satisfaction, consequence, and rewrite structure are transported into PLN-style evidence extraction over world-model states.

This gives a cleaner and more faithful foundation for higher-order PLN than the older story in which higher-order talk was largely reduced immediately to first-order extensional set manipulations.

13.1 Canonical Higher-Order Layer

The canonical higher-order object language is now Church-style simple type theory:

- base types, proposition type, and arrow types,

- lambda abstraction and application,
- equality at every type,
- universal and existential quantification at every type.

This layer lives in `Mettapedia/Logic/HOL/Syntax/Type.lean`, `Mettapedia/Logic/HOL/Syntax/Term.lean`, `Mettapedia/Logic/HOL/Derivation.lean`, and `Mettapedia/Logic/HOL/Semantics/Henkin.lean`.

The semantics are Henkin rather than full standard higher-order semantics. This matters because Henkin semantics are still first-order-manageable at the metatheoretic level, while preserving the cleaner higher-order rule language. So one can speak directly about

$$\lambda x.\, t, \qquad f = g, \qquad \forall P : \iota \to o,\ \varphi(P),$$

instead of encoding those patterns awkwardly inside a purely first-order surface.

Theorem 13.1 (HOL soundness over Henkin models (Core))**.** *If a closed HOL formula is derivable in the real HOL calculus, then it is satisfied by every Henkin model.*

This is formalized in `Mettapedia/Logic/HOL/Soundness.lean`.

13.2 FOL Embedding into HOL

First-order syntax is embedded into HOL by choosing one distinguished individual base type and interpreting first-order function and relation symbols as typed higher-order constants. This embedding is separate from the set-semantics grounding:

- **FOL → HOL** says how first-order formulas are expressed in the higher-order language.
- **Set semantics → HOL** says how higher-order formulas are interpreted in directly induced Henkin models.

This separation keeps the architecture honest. An embedding is not the same thing as a semantic grounding.

The embedding is formalized in `Mettapedia/Logic/HOL/Embedding/FirstOrder.lean`.

13.3 HOL as a World-Model Lens

The world-model side of the higher-order story is now based on genuine HOL satisfaction, not a placeholder predicate-code proxy.

The state is a multiset of pointed Henkin models, the query is a closed HOL formula, and evidence is extracted by counting satisfiers and non-satisfiers. So for a HOL formula φ and WM state W:

$$\mathrm{strength}(W, \varphi)$$

is the normalized support for φ across the higher-order models in W.

The core bridge says:

- singleton adequacy: $M \vDash \varphi$ iff the singleton state $\{M\}$ gives strength 1 to φ;
- pointwise implication is equivalent to singleton-strength consequence;
- pointwise equivalence is equivalent to WM query equivalence.

This is formalized in `Mettapedia/Logic/HOL/WorldModel.lean`, `Mettapedia/Logic/HOL/WorldModelCompleteness.lean`, and publicly packaged in `Mettapedia/Logic/PLNWorldModelHOL.lean`.

13.4 Direct Set-Semantics Grounding of HOL

The low-level arrow from set semantics to HOL is now explicit.

Given a pointed set-theoretic structure, the formalization induces a genuine Henkin HOL model directly, rather than routing everything through first-order syntax first. This is the semantic arrow

$$\text{Set semantics} \longrightarrow \text{HOL}.$$

The directly grounded set/HOL model then inherits the HOL-to-WM bridge, giving the public route

$$\text{Set semantics} \longrightarrow \text{HOL} \longrightarrow \text{WM}.$$

This is formalized in `Mettapedia/Logic/HOL/Semantics/SetBased.lean` and `Mettapedia/Logic/PLNWorldModelHOLSetBridge.lean`.

Theorem 13.2 (Direct set/HOL singleton adequacy (Core))**.** *For a pointed set structure S and closed HOL query φ, $S \vDash_{\mathrm{HOL}} \varphi$ if and only if the singleton world-model state $\{S\}$ assigns strength 1 to φ.*

On embedded first-order set-theory sentences, the older route

$$\text{Set theory} \longrightarrow \text{FOL} \longrightarrow \text{WM}$$

and the newer route

$$\text{Set semantics} \longrightarrow \text{HOL} \longrightarrow \text{WM}$$

agree exactly on evidence and query strength. Moreover, on theory-model states, mutual implication now yields equality of the direct HOL-routed evidence and query-strength values.

13.5 Higher-Order PLN over HOL and WM

Higher-order PLN now sits *above* both the HOL core and the HOL-to-WM bridge.

The key rule surface is not a second semantics. It is a theorem-backed layer over real HOL derivability:

- proved HOL implications become WM consequence rules,
- proved HOL equivalences become WM rewrite and query-equivalence rules,
- conjunction, disjunction, negation, implication monotonicity, and eta-equality all transport through the higher-order bridge.

This layer is formalized in `Mettapedia/Logic/PLNHigherOrderHOLCore.lean`, `Mettapedia/Logic/PLNHigherOrderHOLRules.lean`, `Mettapedia/Logic/PLNHigherOrderHOLSoundness.lean`, `Mettapedia/Logic/PLNHigherOrderHOLConsequence.lean`, and `Mettapedia/Logic/PLNHigherOrderHOLLinkBridge.lean`.

The direct set/HOL/WM route is already exercised by genuinely higher-order positive and negative fixtures:

- higher-order reflexivity and eta-equality, both at the object-function level and at the predicate-function level,
- quantified predicate laws such as $\forall P : \iota \to o. \forall x : \iota. P(x) \to P(x)$,
- conjunction/disjunction commutativity as WM query equivalence,
- negation transport,
- multiset consequence transport through higher-order rules,

- concrete countermodels on small carriers, including the failure of $\forall P : \iota \to o. \forall x : \iota. P(x)$ on a two-point carrier.

In other words, the direct route is no longer limited to endofunction-shaped examples. It already supports genuinely quantified predicate-level higher-order queries, predicate-level eta-equality, and WM-side transport of higher-order rules whose principal formulas quantify over predicates.

See `Mettapedia/Logic/PLNWorldModelHOLSetBridgeRegression.lean` and `Mettapedia/Logic/PLNHigherOrderHOLRegression.lean`.

13.6 Where Satisfying-Set Reduction Still Fits

The older satisfying-set reduction remains mathematically useful, but it is now best understood as a *finite extensional fragment* of the higher-order story rather than as the foundational higher-order layer.

Definition 13.3 (Satisfying set)**.** Given a predicate P over a finite universe U, its satisfying set is $\mathrm{Sat}(P) = \{x \in U \mid P(x)\}$.

Definition 13.4 (Weakness-based inheritance)**.** For satisfying sets $A, B \subseteq U$ and a weight function $\mu : U \to \mathbb{R}_{\geq 0}$, the inheritance proxy is:

$$\mathrm{Inheritance}(A, B, \mu) = \frac{\mathrm{weakness}(A \cap B, \mu)}{\mathrm{weakness}(A, \mu)}.$$

This finite extensional layer still proves useful structural facts such as pointwise implication collapse and corrected inheritance/similarity transport, but it should not be mistaken for the entire higher-order semantics.

Those finite extensional results live in `Mettapedia/Logic/HigherOrder/HigherOrderReduction.lean` and `Mettapedia/Logic/HigherOrder/Basic.lean`.

13.7 Higher-Order Probability: Now Semantically Grounded

The codebase already contains real higher-order probability theory and a real PLN-to-Kyburg bridge:

- Kyburg flattening and expectation preservation,
- Giry-style monadic laws for the flattening,

- posterior-mean identities linking PLN evidence updates to higher-order probability.

See `Mettapedia/ProbabilityTheory/HigherOrderProbability/KyburgFlattening.lean`, `Mettapedia/ProbabilityTheory/HigherOrderProbability/GiryMonad.lean`, and `Mettapedia/Logic/PLNKyburgReduction.lean`.

What was still missing at the previous checkpoint was the fully semantic object

"probability of higher-order formulas over a measure on Henkin models."

The repository now contains an infinitary-first semantic version of that construction. The new subtree `Mettapedia/Logic/HOL/Probabilistic/` defines:

- measurable indexed spaces of pointed Henkin models in `Mettapedia/Logic/HOL/Probabilistic/ModelSpace.lean`,
- sentence probabilities for closed HOL formulas in `Mettapedia/Logic/HOL/Probabilistic/Semantics.lean`,
- the thin probabilistic WM-facing lens in `Mettapedia/Logic/HOL/Probabilistic/WorldModelBridge.lean`, and
- the theorem that the existing empirical HOL↔WM multiset semantics is a special case in `Mettapedia/Logic/HOL/Probabilistic/EmpiricalSpecialCase.lean`.

So the higher-order picture is no longer merely "real HOL here, real higher-order probability there." It now includes a semantic probability layer for closed HOL formulas. What remains missing is the *full dynamic fusion*: beliefs over higher-order formulas that simultaneously interact with logical-induction-style deductive dynamics and this semantic model-measure layer.

13.7.1 Hierarchical and Infinite-Order Probabilistic HOL

The semantic probabilistic HOL layer now also supports a clean infinite-order reading: uncertainty over *measures on indexed model spaces*, rather than only one flat measure or an explicit syntax of nested "probability of probability of probability." This is the right semantic form for the higher-order probability literature ([22, 8, 2]) and for the current guarded higher-order benchmarks.

The key new files are:

- `Mettapedia/Logic/HOL/Probabilistic/HierarchicalState.lean`, which defines hierarchical uncertainty as a probability distribution over probability measures on a fixed indexed model space;
- `Mettapedia/Logic/HOL/Probabilistic/Flattening.lean`, which proves that hierarchical sentence probabilities are computed by flattening the hierarchy to the predictive marginal;
- `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBridge.lean`, which interprets the current guarded higher-order benchmark payloads as one concrete finite hierarchical instance; and
- `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBeliefBridge.lean`, which derives a benchmark-facing LI-style belief day and process from that semantic benchmark marginal, while proving that this adapter is query-focused rather than a replacement for the benchmark semantics; and
- `Mettapedia/Logic/PLNProbHOLPlannerBridge.lean`, which packages the carried guarded query together with that theorem-backed belief shadow, so planner/control layers can consume semantically justified prices and tracking results without becoming the canonical semantics; and
- `Mettapedia/Logic/PLNProbHOLPlannerBridgeRegression.lean`, which records positive and negative planner-facing bridge fixtures for the concrete higher-order leaky benchmark step; and
- `Mettapedia/Logic/HOL/Probabilistic/HierarchicalRegression.lean`, which packages positive and negative canaries for the infinitary, empirical-special-case, and guarded-benchmark routes.

The semantic picture is:

$$\begin{array}{c}
\text{latent regime } \theta \\
\downarrow_{\text{kernel } K(\theta,-) \text{ on model indices}} \\
\text{measure on pointed Henkin-model indices} \\
\downarrow_{\text{flatten / predictive marginal}} \\
\text{sentence probability of } \varphi \\
\downarrow_{\text{thin WM lens}} \\
\text{probabilistic query strength / evidence}
\end{array}$$

This is elegant for two reasons. First, it keeps semantic truth in pointed Henkin models primary. Second, it lets finite and empirical cases become

theorem-proved special cases rather than the canonical meaning of higher-order uncertainty.

The current benchmark bridge is intentionally modest: it interprets the existing guarded three-regime payload as the first concrete hierarchical seed. That is already enough to show how branch-level admissibility values become component sentence probabilities, and how the benchmark's flattened value is the predictive marginal of that hierarchy. The broader Pathfinder-style hierarchy over hidden context, admissibility, trust, and topology is the natural next semantic refinement, not an ad hoc tower. For the benchmark itself, this means that second-order uncertainty can live over latent regimes, third-order uncertainty can live over trust in the regime model, and higher orders beyond that should usually be read through flattening, robustness, or abstention criteria rather than through an endlessly explicit public syntax.

The planner-facing consequence is now explicit. A mixed-mode higher-order guarded step can be consumed as a *belief price* precisely because the repository proves that the carried value agrees with the benchmark belief shadow derived from semantic ProbHOL. In Lean this is exposed by `Mettapedia/Logic/PLNProbHOLPlannerBridge.lean`, including theorems `leakyHigherOrderPlan_C_current_value_eq_benchmarkBeliefPrice` and `leakyHigherOrderPlan_C_current_gateConfidence_eq_higherOrderGuardConfidence`. So the control layer is no longer merely carrying a higher-order-looking score: it is carrying a value with an explicit semantic provenance, together with a separate belief-process shadow that tracks the same benchmark query on the designated sample. The paired regression surface in `Mettapedia/Logic/PLNProbHOLPlannerBridgeRegression.lean` then records both the positive tracking fixture and the negative expanded-sample guardrail.

13.8 Logical-Induction-Ready Dynamic Belief Infrastructure

Following Logical Induction [9], the next higher-order uncertainty layer should not be hard-coded as one static measure or one specific prover. Instead it should be parameterized by:

- a deductive process over coded closed HOL formulas,
- time-indexed belief states over those formulas,
- theory-extension/conditioning operators, and

- calibration and timely-learning criteria that remain distinct from semantic truth in Henkin models.

This interface layer is now explicit in the codebase. The new subtree `Mettapedia/Logic/HOL/LogicalInduction/` contains:

- canonical coding of closed HOL formulas in `Mettapedia/Logic/HOL/LogicalInduction/Code.lean`,
- deductive-process and theory-extension interfaces in `Mettapedia/Logic/HOL/LogicalInduction/DeductiveProcess.lean` and `Mettapedia/Logic/HOL/LogicalInduction/Conditioning.lean`,
- day-indexed market/belief vocabulary plus trader and exploitation criteria in `Mettapedia/Logic/HOL/LogicalInduction/Market.lean` and `Mettapedia/Logic/HOL/LogicalInduction/Criterion.lean`,
- calibration and timely-learning specification hooks in `Mettapedia/Logic/HOL/LogicalInduction/Calibration.lean`,
- the current artifact-only HOL→Pure compatibility layer in `Mettapedia/Logic/HOL/LogicalInduction/PureBridge.lean`, and
- a thin WM-facing day interface together with the theorem that the existing multiset-counting HOL WM semantics is its empirical special case in `Mettapedia/Logic/HOL/LogicalInduction/WorldModelBridge.lean` and `Mettapedia/Logic/HOL/LogicalInduction/EmpiricalSpecialCase.lean`.

The corresponding lightweight top-level wrapper is now `Mettapedia/Logic/ProbHOL.lean`. It re-exports both:

- the new semantic probabilistic HOL layer, and
- the logical-induction-ready dynamic belief layer.

It still explicitly avoids the stronger claim that the repository already contains the fully fused dynamic theory of logical-induction belief processes over semantic probabilities of Henkin-model events.

There is now also one precise benchmark-facing bridge back from semantic probability to LI-style pricing. The file `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBeliefBridge.lean` constructs a benchmark belief day whose visible price on the benchmark query is the carried guarded

higher-order value *because* that value is already proved equal to the hierarchical semantic benchmark marginal. The key tracking theorems are `benchmarkBeliefDay_tracks_benchmarkHierarchicalProbOn` and `benchmarkBeliefProcess_event` Just as importantly, the negative theorem `benchmarkBeliefDay_not_tracks_benchmarkHierarchical` shows that this adapter is intentionally narrow: it prices the benchmark query correctly, but it is not presented as a global belief oracle for unrelated HOL sentences.

The generic comparison waist is now explicit in `Mettapedia/Logic/HOL/Probabilistic/BeliefBridge.lean`. Its central predicates are `BeliefDayTracksHierarchicalProbO` and `BeliefProcessEventuallyTracksHierarchicalProbOn`: these say exactly when a day-level price assignment or a time-indexed belief process matches the semantic hierarchical ProbHOL query strength on a chosen finite sample of coded HOL formulas. The benchmark layer then instantiates those generic predicates, rather than inventing a second benchmark-only notion of tracking. In Lean this is visible through theorems such as `benchmarkBeliefDay_tracks_benchmarkLatentHierarchicalProbOn` in `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBeliefBridge.lean` and `benchmarkPlannerShadow_process_` in `Mettapedia/Logic/PLNProbHOLPlannerBridge.lean`. So the intended four-layer example is now theorem-traceable:

HOL truth for $\varphi \implies$ hierarchical semantic probability of $\varphi \implies$ belief-process tracking on the bench

Here the richer benchmark latent coordinates for hidden context, trust, and topology live explicitly in `Mettapedia/Logic/HOL/Probabilistic/BenchmarkBridge.lean`; the bridge to planner control remains downstream of the generic semantic-vs-belief layer rather than becoming a new semantics in its own right.

truth side
closed HOL formulas
$\downarrow$Henkin semantics
truth in pointed Henkin models
$\downarrow$measurable indexed model spaces
semantic sentence probability $\mu\{i \mid M_i \vDash \varphi\}$
$\downarrow$empirical multiset special case
static HOL $\leftrightarrow$ WM strength

belief side
coded closed HOL formulas
$\downarrow$deductive process D_n
belief days $P_n(\varphi)$
$\downarrow$day-level WM-facing strength
belief-day strength/evidence
dynamic belief/evidence interface

The important architectural point is that this does *not* replace the existing HOL truth layer. Henkin semantics in `Mettapedia/Logic/HOL/Semantics/Henkin.lean` still determines semantic truth, and `Mettapedia/Logic/HOL/`

`WorldModel.lean` still provides the static world-model lens. The new logical-induction-ready layer is an overlay for future deductively limited belief dynamics about closed HOL formulas.

So Chapter 13 now has three clearly separated higher-order levels:

1. semantic truth in Henkin HOL,
2. semantic probability over measurable indexed model spaces of pointed Henkin models,
3. the empirical HOL↔WM strength layer as a theorem-proved special case, and
4. dynamic belief-process scaffolding for future logical-induction-aware probabilistic HOL.

This is the right place to stop for now. It is already enough to keep future ProbHOL and higher-order uncertain reasoning from being forced through a hard-coded prover or a single empirical model sample, but it does *not* yet claim a full logical-inductor construction or the full dynamic fusion of belief processes with semantic probabilities over Henkin models.

Chapter 14

Quantifiers and the Fuzzy/QFM Layer

PLN extends beyond propositional inference to quantified reasoning. This chapter covers the stable finite-domain quantifier bundle, fuzzy quantifier algebra, and their ITV-coordinate transport. The codebase also contains an arbitrary-domain infinitary semantic layer in `Mettapedia/Logic/PLNFirstOrder/Infinite.lean`; what remains finite-domain here is the reviewed soundness/canary package for this chapter, not the whole WM calculus.

14.1 Finite-Model Quantifier Semantics

In a finite domain $U = \{u_1, \dots, u_m\}$ with weight function $\mu : U \to \mathbb{R}_{\geq 0}$, quantifiers receive a weakness-based semantics:

Definition 14.1 (Universal quantifier evaluation (Core))**.** For a predicate set $S \subseteq U$:

$$\forall_\mu(S) = \text{weakness}\big(\mu,\ \text{diag}(S)\big) = \sum_{u,v \in S} \mu(u) \cdot \mu(v)$$

where $\text{diag}(S) = \{(u, v) \mid u \in S \wedge v \in S\}$. Under uniform μ, this reduces to $|S|^2/|U|^2$, the squared fraction of the domain satisfying S.

Definition 14.2 (Existential quantifier evaluation (Core))**.** By de Morgan reduction:

$$\exists_\mu(S) = \text{compl}\big(\forall_\mu(\neg S)\big)$$

Theorem 14.3 (Quantifier ordering (CORE)). *For any predicate set S and weight μ:*

$$\forall_\mu(S) \leq \exists_\mu(S)$$

Extensional quantifiers use lattice operations: $\forall^{\text{ext}}(S) = \bigwedge_u S(u)$ and $\exists^{\text{ext}}(S) = \bigvee_u S(u)$.

See `Mettapedia/Logic/PLNFirstOrder/QuantifierSemantics.lean` and `Mettapedia/Logic/PLNFirstOrder/Soundness.lean`.

14.2 Fuzzy Quantifier Semantics

Beyond crisp quantifiers, PLN supports fuzzy quantifiers ("most," "few," "about half") via a QFM (Quantitative Fuzzy Modifier) approach.

Definition 14.4 (Fuzzy quantifier parameters). A fuzzy quantifier specification $\mathcal{P}$ consists of:

$$\varepsilon \in [0,1] \qquad \text{proxy tolerance around } 0/1 \tag{14.1}$$

$$L, U \in [0,1],\ L \leq U \qquad \text{lower/upper proxy confidence} \tag{14.2}$$

$$\theta \in [0,1] \qquad \text{proxy confidence threshold} \tag{14.3}$$

The "near-one" and "near-zero" predicates are:

$$\text{nearOne}_\varepsilon(x) \iff 1 - \varepsilon \leq x \leq 1, \qquad \text{nearZero}_\varepsilon(x) \iff 0 \leq x \leq \varepsilon$$

Definition 14.5 (Witness fraction). For a predicate ϕ on U:

$$\text{witFrac}(\phi) = \frac{|\{u \in U \mid \phi(u)\}|}{|U|}$$

The fuzzy-$\forall$ holds when the near-one witness fraction exceeds the threshold: $\theta \leq \text{witFrac}(\text{nearOne}_\varepsilon \circ f)$ where $f : U \to [0,1]$ is the membership profile.

QFM composition operators: multiplicative ($x \cdot y$), minimum ($\min(x,y)$), Łukasiewicz ($\max(0, x+y-1)$), and probabilistic sum ($x + y - xy$) provide graded conjunction/disjunction with ITV-coordinate transport.

See `Mettapedia/Logic/PLNFirstOrder/FuzzyQuantifierSemantics.lean` and `Mettapedia/Logic/PLNFirstOrder/FuzzyITVBridge.lean`.

Arbitrary-domain graded fuzzy layer. The repository now also exposes a reusable arbitrary-domain graded specialization layer. Instead of treating the Sugeno/proxy-cut and Choquet branches as unrelated surfaces, Lean factors out their honest common spine via `Mettapedia/Logic/PLNFirstOrder/GradedQuantifierSpecialization.lean`. The generic interface `GradedQuantifierSemantics` packages:

- score-based interval and universal predicates,
- complement-dual existential predicates,
- fuzzy-domain restriction via `domainRestrict`,
- and equality / monotonicity transport on the restricted domain.

The current Sugeno and Choquet branches are then explicit instances (`sugenoGradedQuantifierSemant` `choquetGradedQuantifierSemantics`), while the fuzzy-domain layer is a further specialization of that shared graded interface rather than a duplicated theory. Direct graded canaries now live in `Mettapedia/Logic/PLNFirstOrder/GradedQuantifierCanary.lean`, so the shared graded spine is itself exercised by positive and negative examples, not only through the specialized fuzzy-domain wrappers.

14.3 Scope Note: Stable Finite-Domain Bundle vs. Infinitary Semantics

The theorem-complete quantifier bundle emphasized in this chapter is established for finite domains. However, the repository already contains an arbitrary-domain infinitary semantic layer with the basic definitions and laws (`WeightFunctionInf`, `SatisfyingSetInf`, `forAllEvalInf`, `thereExistsEvalInf`, `deMorgan_inf`, `weaknessInf_mono`) in `Mettapedia/Logic/PLNFirstOrder/Infinite.lean`.

So the honest limitation is narrower: the reviewed Chapter 14 front door is finite-domain, while arbitrary-domain quantifier semantics already exist in Lean. What remains future work is full parity between the finite-domain chapter bundle and the infinitary layer: stronger metatheory, canary/regression coverage, and any measure-theoretic extensions needed for probabilistic applications on genuinely infinite spaces.

Chapter 15

Intensional Inheritance

Intensional inheritance is PLN's mechanism for reasoning about relationships based on shared properties (intension) rather than shared instances (extension). This is one of the most ambitious parts of the PLN book, and one where the formalization required the most careful treatment.

15.1 What Intensional Inheritance Captures

Two concepts A and B have high intensional similarity when they share many properties—when things that are true of A tend also to be true of B. The original PLN book defines intensional inheritance as a mixture of extensional and intensional components, with the mixture controlled by a parameter.

15.2 Typed WM Grounding

The ξPLN approach grounds intensional inheritance in the world-model framework through a typed query family:

Definition 15.1 (Inheritance sort)**.** Queries are indexed by their semantic channel:

InheritanceSort = extensional | intensionalAssoc | intensionalPAT | mixed

Each sort has its own evidence extraction and strength computation.

The ASSOC channel computes association strength via mutual information. Given features F and a world-model state W:

$$P_{\text{ASSOC}}(W \mid F) = P(W) \cdot 2^{I(F;W)}$$

where $I(F;W) = \sum_{f,w} p(f,w) \log \frac{p(f,w)}{p(f)\cdot p(w)}$ is the Shannon mutual information between the feature profile and the world state. High ASSOC inheritance between A and B means that knowing which properties A has gives strong predictive power for which properties B has.

The PAT (pattern) channel provides a complementary structural route based on property-pattern proximity: two concepts A, B have high PAT similarity when they share structural patterns beyond what mutual-information association captures.

See `Mettapedia/Logic/PLNIntensionalWorldModel.lean`.

15.3 Solomonoff Bridge

The Solomonoff bridge connects intensional inheritance to algorithmic information theory:

- The exact formal bridge is the universal-mixture log-ratio theorem: $\mathrm{InhInt}_\xi(W,F,x) = \log_2\big(P_\xi(W \mid F,x)/P_\xi(W \mid x)\big)$ when both conditionals are positive.
- Specialized `xiSemimeasure` and geometric-mixture versions are proved as explicit theorem endpoints.
- This supports the algorithmic-information interpretation of intensional inheritance, but the primary theorem statement is the log-ratio bridge rather than a standalone normalized-information-distance identity.

See `Mettapedia/Logic/IntensionalInheritanceSolomonoffBridge.lean`.

15.4 Mixed Semantic Packaging

The `AssocPatSemanticModel` packages ASSOC and PAT channels into a unified semantic model with selector-specialized endpoints. The original PLN book proposes a simple mixture:

$$\mathrm{InhInt}(A,B) = \alpha \cdot \mathrm{Ext}(A,B) + (1-\alpha) \cdot \mathrm{IntAssoc}(A,B)$$

with a single mixing parameter α. The formalization explicitly refutes this as a general law:

Theorem 15.2 (No binary mixed policy (BOUNDARY))**.** *There exists a concrete state* W_{demo} *with evidence scores* $(3, 2, 1)$ *for (extensional, ASSOC, PAT) channels such that the PAT channel contributes nontrivially (*pat_channel_nontrivial*) and the mixed result cannot be expressed as any function of only the extensional and ASSOC values (*mixed_not_assoc_only*).*

This means the book's binary mixture is not merely incomplete but false as a general chapter-level law: the full three-channel model (extensional, ASSOC, PAT) is needed. The corrected theoremic surface is therefore a typed three-channel mixed rule, not a repaired version of the old binary extensional+ASSOC law.

See `Mettapedia/Logic/PLNIntensionalCanary.lean`, `Mettapedia/Logic/PLNIntensionalAssocPatClosure.lean`, and `Mettapedia/Logic/PLNIntensionalRegression.lean`.

Chapter 16

Temporal and Causal Inference

Temporal and causal inference in PLN addresses reasoning about events ordered in time and about cause-effect relationships.

16.1 Temporal Operators

Temporal predicates live in the type $\text{TemporalPred}(D, T) = D \to T \to \text{Prop}$, where D is the domain of entities and T is the time type (typically $\mathbb{N}$ or $\mathbb{R}$).

Definition 16.1 (Simultaneous operators (Core))**.**

$$\text{SimAND}(P, Q)(x, t) := P(x, t) \wedge Q(x, t) \tag{16.1}$$

$$\text{SimOR}(P, Q)(x, t) := P(x, t) \vee Q(x, t) \tag{16.2}$$

Definition 16.2 (Sequential operators (Core))**.**

$$\text{SeqAND}(P, Q)(x, t) := P(x, t) \wedge \exists \Delta > 0.\ Q(x, t + \Delta) \tag{16.3}$$

$$\text{SeqAND}_f(P, Q)(x, t) := P(x, t) \wedge \exists \Delta.\ f(\Delta) \wedge Q(x, t + \Delta) \tag{16.4}$$

where f is a "future" predicate constraining the delay (e.g., $\Delta \in [1, T_{\max}]$).

Definition 16.3 (Predictive implication (Core))**.**

$$\text{PredImpl}(P, Q)(x, t) := P(x, t) \Rightarrow \exists \Delta > 0.\ Q(x, t + \Delta)$$

The 3-node predictive chain composes: given temporal bounds T_1, T_2,

$$\text{PredChain}_3(T_1, T_2, A, B, C)(x, t) := A(x, t) \wedge B(x, t + T_1) \Rightarrow C(x, t + T_2)$$

16.2 Causal Profile

Causal inference is grounded through a probabilistic event calculus:

- **Event predicates**: Happens(e, t), Initiates(e, f, t), Terminates(e, f, t), HoldsAt(f, t).
- **Causal profile**: for a pair (A, B), the causal profile is the collection of predictive implication strengths across temporal lags: CausalProfile$(A, B) = \{\mathsf{strength}(\text{PredImpl}_\Delta(A, B)) \mid \Delta \in [1, T_{\max}]\}$.
- **Distinguishing causation from correlation**: Granger-style temporal precedence plus the event calculus distinguish $A \to B$ (causal) from $A \leftarrow C \to B$ (spurious correlation) within the world-model framework.

See `Mettapedia/Logic/PLNTemporalCausalInference.lean` and `Mettapedia/Logic/PLNProbabilisticEventCalculus.lean`.

16.3 Temporal as Another WM Instance

The key design decision: temporal and causal inference is *not* a separate ontology. It is another instantiation of the world-model framework. The world-model state includes temporal structure (event histories), the query language includes temporal operators (SimAND, SeqAND, PredImpl), and the evidence extraction/revision machinery operates unchanged. This keeps the foundational theory unified: a single kernel, many instantiations.

Part VI

Large-Scale Inference, Control, and Boundaries

Chapter 17

Large-Scale Inference at Theorem Level

Chapter 9 of the PLN book discusses large-scale inference strategies: batch inference, multideduction, BN augmentation, trail-free protocols. This is the most complex chapter to formalize, and the most instructive about the gap between theoretical correctness and operational feasibility.

17.1 Multideduction and Identifiability

Multideduction—applying multiple deduction steps and combining results—runs into a fundamental identifiability problem.

Theorem 17.1 (Inclusion-exclusion identifiability no-go (Boundary))**.** *Two-term inclusion-exclusion statistics are not sufficient to identify the union mass. There exist distinct probability distributions agreeing on all first-two-term statistics but disagreeing on the union probability.*

The corrected positive result:

Theorem 17.2 (Residual decomposition (Conditional))**.** *The union cardinality decomposes as the two-term estimate plus a residual. If the residual is known (or bounded), the corrected estimate agrees with the true value.*

See `Mettapedia/Logic/PLNMultideductionResidual.lean` and `Mettapedia/Logic/PLNInclusionExclusionIdentifiability.lean`.

17.2 Trail-Free Protocol Dynamics

The trail-free deterministic update protocol can fail to stabilize:

Theorem 17.3 (Trail-free non-stabilization (BOUNDARY))**.** *There exists a two-state system where the trail-free deterministic update enters a 2-cycle and never reaches a fixed point.*

The positive result under damping:

Theorem 17.4 (Damped convergence (CONDITIONAL))**.** *Under damping and eventual fresh evidence, the SP/SPN protocol eventually stabilizes. Under bounded-gap fairness, stabilization occurs within an explicit bound.*

See `Mettapedia/Logic/PLNTrailFreeDynamicsCounterexample.lean` and `Mettapedia/Logic/PLNTrailFreeDampedConvergence.lean`.

17.3 Pooling and Fusion

Evidence pooling axioms are insufficient to characterize additive fusion:

Theorem 17.5 (Pooling non-uniqueness (BOUNDARY))**.** *The max-pooling operator satisfies the pooling axioms but differs from additive fusion.*

The corrected uniqueness result:

Theorem 17.6 (Pooling uniqueness (CONDITIONAL))**.** *External Bayesianity plus total additivity uniquely recovers additive evidence pooling.*

See `Mettapedia/Logic/PremiseSelectionExternalBayesianity.lean`.

17.4 Evidence-Conservation and Order-Cost Governance

The non-additive perimeter is now covered by a theorem pack rather than ad hoc prose. The key move is to separate three layers:

- **Conservation under forgetting**: anti-hallucination and outside-scope leakage bounds when exact inverse forgetting holds.
- **Order-cost bounds**: schedule-level error bounded by pairwise swap-anomaly budgets.

- **Runtime certification**: executable pairwise checks imply the proved schedule/policy bounds.

Theorem 17.7 (Evidence-conservation pack from forgetting (CONDITIONAL))**.** *If a forgetting operator is an exact inverse for a scoped revision, then outside that scope:*

1. *no new evidence can appear (anti-hallucination),*
2. *query evidence is conserved,*
3. *leakage count/budget is zero, hence any nonnegative leakage budget threshold is satisfied.*

Theorem 17.8 (Order-cost budget bound (CONDITIONAL))**.** *If each adjacent swap in a schedule is bounded by a per-step anomaly budget, then total schedule error is bounded by the sum of those budgets. In particular, commutative merge-evidence implies zero swap anomaly and zero schedule error.*

Theorem 17.9 (Runtime audit certificate soundness (CONDITIONAL))**.** *If runtime pairwise-order checks pass with budget vector B, then the formal schedule-error bound and budget-threshold policy conclusions follow in Lean. For two-step schedules, a single swap-step check certifies the two-step policy threshold.*

Lean endpoints:

- `evidenceConservationPack_of_forgetting` in `Mettapedia/Logic/PLNWorldModelConservationPack.lean`
- `scheduleErrorBound_of_pairwise_bounds` and `scheduleErrorBound_twoStep_of_swapStepB` in `Mettapedia/Logic/PLNWorldModelOrderCostBounds.lean`
- `runtimePairwiseOrderCheck_certifies_scheduleErrorBound` and `runtimePairwiseOrderCheck_certifies_policyThreshold` in `Mettapedia/Logic/PLNWorldModelOrderCostAuditCertificate.lean`

Concrete demos. The weighted numeric lane proves nontrivial non-commutative behavior (`weightedScheduleErrorBound_twoStep_not_zero_of_pos_weight`), and the gas governance lane proves certified zero-budget pass on batch swap under additive merge (`gasOrderBudgetPolicy_batchSwap_zero_via_runtimeCertificate`). See `Mettapedia/Logic/PLNWorldModelOrderCostWeightedDemo.lean` and `Mettapedia/Logic/PLNWorldModelOrderCostGasPolicyDemo.lean`.

17.5 BRGI and Feasible Sets

The Balanced Relevance-Guided Inference (BRGI) framework provides an optimization-theoretic perspective:

- Finite feasible sets with policy-sensitive optimum characterization.
- Graph-level BRGI objects with optimality-preserving refinement.

See `Mettapedia/Logic/PremiseSelectionBRGI.lean`.

17.6 Stochastic Experiment Layer

Blackwell factorization, weighted source policies, and trusted-gate transfer provide a formal framework for reasoning about the value of experimental evidence:

- Expected utility equals the Blackwell factor.
- Optimal value is monotone under Blackwell domination.
- Trusted-gate transfer preserves optimality ordering.

See `Mettapedia/Logic/PLNWorldModelExperimentStochastic.lean`.

Chapter 18

Inference Control

Inference control addresses the meta-level problem: given a large knowledge base and many possible inference steps, which steps should be taken?

18.1 Selector Specifications

A *selector* is a function that chooses premises for inference. The formalization provides:

- Formal selector specifications with typed interfaces.
- Ranking transfer: if a selector produces a good ranking on one distribution, it produces a good ranking on similar distributions.
- Ranking stability: small perturbations in the knowledge base produce small perturbations in the ranking.

See `Mettapedia/Logic/PremiseSelectionSelectorSpec.lean` and `Mettapedia/Logic/PremiseSelectionRankingStability.lean`.

18.2 Coverage and Submodularity

Let D be the set of true dependencies and S the selected premise set. The coverage objective is:

$$f(S) = |S \cap D|$$

Theorem 18.1 (Coverage submodularity (CORE))**.** *The function f is monotone ($S \subseteq T \Rightarrow f(S) \leq f(T)$) and submodular: for $S \subseteq T$ and any element $x \notin T$:*

$$f(T \cup \{x\}) - f(T) \;\leq\; f(S \cup \{x\}) - f(S)$$

By the classical result of Nemhauser, Wolsey, and Fisher, the greedy algorithm (iteratively adding the element with maximum marginal gain) achieves a $(1 - 1/e) \approx 0.632$ approximation ratio for monotone submodular maximization under cardinality constraints. This guarantee is formalized via an explicit `GreedyChain` inductive construction.

See `Mettapedia/Logic/PremiseSelectionCoverage.lean`.

18.3 Coverage Surrogate Boundaries

Theorem 18.2 (Coverage/cardinality surrogate no-go (BOUNDARY))**.** *Coverage and cardinality together are insufficient for risk-sensitive selection: there exist premise sets with identical coverage and cardinality but different false-positive risk.*

See `Mettapedia/Logic/PremiseSelectionCoverageCounterexamples.lean`.

18.4 Composed Core Endpoints

The one-call selector → rewrite → threshold composition:

```
ch9_selector_rewrite_threshold_end_to_end_of_interval
```

composes the entire pipeline into a single theorem: given a selector that produces a ranked premise set, a rewrite that applies a PLN rule under BN side conditions, and a threshold that accepts the result, the composed pipeline is correct.

See `Mettapedia/Logic/PLNCanonicalAPI.lean`.

Part VII

Artifact, Scope, and Relation to PLN

Chapter 19

What ξPLN Proves

This chapter provides a compact, family-level summary of the formalization.

19.1 The Kernel

- World-model interface with evidence compositionality.
- Dirichlet reference instance with query extraction.
- Sequent-style calculus with admissible query-revision.
- Local-completeness no-go theorem.

19.2 Views and Evidence

- Evidence quantale and Heyting algebra.
- STV, ITV, distributional views as typed projections.
- Explicit separation of Walley predictive from Bayesian credible intervals.
- De Finetti and Solomonoff convergence bridges.
- Knuth–Skilling valuation-algebra bridge.

19.3 Compiled Inference

- Corrected extensional-rule exactness for chain, fork, collider motifs under screening-off / positivity conditions.
- Class-packaged d-separation discharge.
- Xi rule registry with typed BN constructor lifts.
- One-call selector → rewrite → threshold composition.
- Threshold transport with explicit boundaries.

19.4 Extensions

- Higher-order reduction with corrected side conditions.
- Finite-model quantifier and fuzzy quantifier semantics.
- Intensional inheritance with ASSOC/PAT typed WM grounding.
- Temporal/causal inference as WM instance.

19.5 Large-Scale Inference and Control

- Multideduction residual decomposition with identifiability no-go.
- Trail-free non-stabilization and damped convergence.
- Pooling uniqueness under external Bayesianity.
- BRGI optimization with feasible-set characterization.
- Premise-selection coverage with submodularity and greedy guarantee.
- Coverage surrogate no-go.

19.6 Cross-Cutting

- **PLN ↔ NARS:** confidence transforms, rule correspondences, revision coherence, informativeness adjunction.
- **MeTTa parity:** executable formula parity between Lean theorems and MeTTa truth functions.

- **Naive Bayes bridge** (Theorem 11.4) and **k-NN bridge** (Theorem 11.5).
- **ProbLog bridge:** the world-model infrastructure subsumes ProbLog distribution semantics exactly:

$$\mathsf{strength}(\text{probLogToJointEvidence}(p), q) = \text{probLogQueryProb}(p, q)$$

  with a critical distinction: $T_{P,K,\mathrm{LP}}$ at $K = \mathbb{R}^{\infty}_{\geq 0}$ computes *bag* semantics, not distribution semantics. The correct path is: total-choice → residual KB → least Herbrand model.
- **MLN bridge (finite-support countable fragment):** the world-model infrastructure now also subsumes an infinite-first MLN semantics, specialized under locality/finite-support witnesses:

$$\text{queryStrength}(\{\text{compiledMassSemantics}(M, h_s)\}, q) = \text{queryProb}_M(q)$$

  where the equality factors through a finite restriction theorem and an exact factor-graph compilation. This is a semantic import theorem for the countable finite-support fragment, *not* a claim that MLN KB union is identical to WM additive revision.
- **Noisy-OR:** $s_{\text{combined}} = 1 - \prod_i (1 - s_i)$ for independent causes, with delta-method variance propagation and **posterior mean scoring**: score $= S \cdot C + (1 - C) \cdot \mu_0$ as the Bayesian point estimate (see `Mettapedia/Logic/PLNNoisyOr.lean`).

Chapter 20

What ξPLN Does Not Prove

Honest scope is as important as proud achievements.

20.1 Operational Runtimes

The formalization provides mathematical specifications and correctness theorems. It does not provide:

- An executable PLN inference engine.
- Performance benchmarks on large knowledge bases.
- Integration with MeTTa or Hyperon runtimes.

These are operational concerns that depend on runtime infrastructure not yet available. The theorem-level results constrain what any correct implementation must satisfy, but they do not constitute an implementation.

20.2 Infinite-Model Quantifiers

The reviewed quantifier package in Chapter 14 is finite-domain, but this is not a global restriction on WM-PLN. The codebase already exports an arbitrary-domain infinitary semantic layer for quantifiers; what remains future work is bringing that layer up to full chapter-level parity and adding any measure-theoretic strengthening needed for infinite probabilistic spaces.

20.3 Chapter 12 Settlement

The intensional inheritance formalization now settles the chapter in corrected typed form: typed WM grounding for extensional, ASSOC, and PAT channels; the Solomonoff log-ratio bridge as the semantic anchor; selector-specialized ASSOC/PAT endpoints; and a constructive no-go against collapsing the three-channel mixed rule back to the old binary law. No formula/claim statement currently appearing in this chapter remains open: each is explicitly classifiable as proved, strengthened to the corrected typed statement, or ruled out.

20.4 Universal Prediction

The Universal Prediction module (21 files, work-in-progress) explores connections between PLN and algorithmic information theory but is not part of the stable core narrative.

20.5 Global PLN Reorganization

This document presents the formalization as it stands. A comprehensive refactoring of the Lean codebase into a "mathlibified" library structure is a worthy future goal but is explicitly not attempted here.

20.6 MLN Import and Subsumption

The Lean development now contains an *infinite-first* MLN import layer. The formal route is:

1. abstract mass semantics for MLN-style world weights and query mass;
2. countable semantics via $\sum$ over worlds;
3. a finite-support restriction theorem under locality witnesses;
4. exact specialization of that finite restriction to a factor graph;
5. transfer of the compiled factor graph into a singleton WM state.

The terminal theorem says that if a countable MLN semantics M admits a finite-support witness h_s, then the compiled singleton world-model state satisfies

$$\text{queryStrength}(\{\text{compiledMassSemantics}(M, h_s)\}, q) = \text{queryProb}_M(q).$$

So the finite factor graph is not the foundation; it is the first exact computable shadow of the general semantics.

This result is intentionally scoped. It covers the countable, finite-support fragment needed for a precise subsumption theorem. It does *not* yet claim:

- lifted MLN inference;
- approximate inference;
- open-universe MLNs;
- or that MLN knowledge-base union is the same operation as additive WM revision.

A benchmark such as UW-CSE [28] remains the right next empirical canary, but it is now downstream of an exact semantic bridge rather than the starting point.

20.7 DeFi Hybrid Risk Monitoring

The same hybrid WM evidence machinery should extend naturally to DeFi protocol risk monitoring, combining continuous market and reserve variables (TVL, utilization, volatility) with discrete governance, oracle-type, and incident propositions. The per-source Normal-Gamma conjugate framework already handles heterogeneous evidence fusion; the missing piece is an operational data pipeline and a curated fixture. We leave this as future empirical work.

Chapter 21

Conclusion

ξPLN is not a replacement for PLN. It is the mathematical foundation that PLN always needed.

The central insight is that PLN's original instincts were largely correct: evidence counts as the fundamental representation, revision as the primary operation, and link-level rules as the practical inference mechanism. What was missing was the explicit separation of layers and the honest statement of assumptions.

The refoundation gives us:

- **One kernel**: the world-model posterior-state calculus, with evidence compositionality as the fundamental law.
- **Many views**: strength, confidence, intervals, distributions, all as typed projections from the kernel.
- **Explicit side conditions**: every compiled rule carries its assumptions, with counterexamples showing what happens when they are violated.
- **Cleaner boundaries**: the no-go theorem, the identifiability failures, the pooling non-uniqueness, the coverage surrogate limitations— all stated precisely and proved formally.

The result is a PLN that knows what it knows, states what it assumes, and marks what it cannot do. That is not a weakness. That is the beginning of a trustworthy reasoner.

> *Probability is what you query. Evidence is what you carry. And the boundaries you prove are worth as much as the theorems you prove.*

Appendices

Appendix A

Advanced-AI Outcomes (WM Scaffold)

This appendix records the current WM-PLN analysis status for *advanced-AI outcomes* without over-claiming posterior precision. The objective is to keep one auditable pipeline from evidence ingestion to predicate- and lens-level query outputs.

A.1 WM Query Form

The current data bundle does *not* justify a single exclusive global terminal-outcome atom. Different regions can instantiate different governance and control regimes simultaneously, and benefit, lock-in, mixed, and catastrophic pressures can co-occur across jurisdictions and time horizons. The appendix therefore treats the world-model state as a richer situation state

$$S = (g, t, G, A, I, U, F),$$

where g is geography/region, t is horizon, G is governance/auditability structure, A is authoritarian/lock-in pressure, I is the incident/harm surface, U is the benefit-side capability surface, and F is the forecast-target/reliability layer. Given prior world-model state W_0, evidence bundle E, and a declared scenario-regime scaffold $\mathcal{R}$, the revised state is

$$W_E := \text{revise}(W_0, E; \mathcal{R}),$$

and the basic query form is predicate-valued:

$$q_\varphi := P(\varphi(S) \mid W_E).$$

The familiar labels `beneficial_pluralistic_transition`, `mixed_reversible_world`, `authoritarian_lock_in`, `catastrophic_disempowerment`, and `extinction` are used here only as *coarse reporting lenses* over S, not as primitive exhaustive atoms.

In the current appendix build, coarse-lens queries are reported in an *abstention-first interval form*

$$q_\varphi \in [L_\varphi, U_\varphi],$$

with explicit status labels (theorem-certified exact, evaluation-certified bound, or abstain/operational-only).

A.2 What the Current Data Actually Supports

The normalized sources support factor-level analysis much more directly than terminal-world labels. In particular:

- the forecast layer supports question families by target tag, horizon, source platform, and resolution status;
- the governance layer supports organization-, commitment-, and capability-domain-conditioned queries;
- the incident layer supports geography-, domain-, harm-, and severity-conditioned pressure queries;
- the authoritarian and benefit proxy layers support thin but explicit directional pressure predicates.

For external review, the primary empirical surface is the normalized table bundle itself. That bundle is intentionally small enough to audit: each table has a clear granularity, a short list of key axes, a declared WM role, and a visible caveat.

The six normalized tables should be read as follows:

- `forecast_questions.csv`: target-family and horizon evidence, plus calibration substrate;
- `source_reliability_candidates.csv`: source trust layer;
- `preparedness_governance_events.csv`: governance and auditability events;

Normalized table	Rows	Granularity	Key axes	WM role	Key caveat
forecast_questions.csv	1195	one row per forecast question / questio...	source_platform, category_target, horizon, resolved_status	direct target-family evidence plus calibration...	Mixes direct target questions with calibration rows and snapshot/open items...
source_reliability_candidates.csv	13	one row per source or source family	source_type, domain, calibration_window, resolved_items_count	reliability / trust layer for forecast sources	Thin table; reliability signals are heterogeneous and partly report-level r...
preparedness_governance_events.csv	14	one row per document-level governance/p...	org, date, event_type, capability_domain, auditability_level	governance and auditability factor surface	Currently normalized at document-event granularity, not full claim/event ex...
incident_signals.csv	1392	one row per incident	date, geography, domain, harm_type, severity_proxy, actor_type	harm/misuse/incident pressure surface	Domain, harm, severity, actor, and geography are heuristic normalizations f...
authoritarian_risk_proxies.csv	5	one row per proxy statistic	source, geography, proxy_type, interpretation_direction	lock-in / authoritarian pressure proxies	Very thin proxy layer; useful for directional pressure, not a calibrated po...
benefit_side_proxies.csv	10	one row per upside-related capability p...	source, date, proxy_type, unit	benefit-side capability and upside predicates	Capability proxies are not the same thing as welfare or pluralistic social...

Table A.1: Normalized appendix datasets for review: what each table is, what it supports, and its main limitation.

Factor family	Rows	Supported axes	Example WM query family
Forecast target family	1195	source_platform, category_target, horizon, resolved_status	Question- and horizon-conditioned outcome lenses, especially mixed_outcomes versus extinction/lock_in-/disempowerment...
Governance and auditability	14	org, date, event_type, capability_domain, auditability_level	Preparedness and evaluation-governance predicates conditioned on organization and capability domain.
Incident and harm surface	1392	date, geography, domain, harm_type, severity_proxy, actor_type	Region- and domain-conditioned misuse/harm pressure, especially surveillance, generative-model, and safety incident s...
Authoritarian pressure proxies	5	source, geography, proxy_type, interpretation_direction	Lock-in pressure queries by geography/sample and proxy direction.
Benefit-side capability proxies	10	source, date, proxy_type, unit	Benefit-side capability and science/medicine upside predicates, not direct welfare posteriors.

Table A.2: Factor families directly supported by the current normalized advanced-AI appendix bundle.

- `incident_signals.csv`: harm/misuse/incident surface;
- `authoritarian_risk_proxies.csv`: thin lock-in pressure proxies;
- `benefit_side_proxies.csv`: thin upside capability proxies.

A.3 Derived Outcome Queries

We therefore treat the familiar outcome names as shorthand query families, not as globally exclusive state atoms:

$$\begin{aligned} p_{\text{doom}}(t) &:= P(\varphi_{\text{global_disemp}}(t) \vee \varphi_{\text{global_ext}}(t) \mid W_E), \\ p_{1984}(g,t) &:= P(\varphi_{\text{lock_in}}(g,t) \mid W_E), \\ p_{\text{BGI}}(g,t) &:= P(\varphi_{\text{beneficial_pluralistic}}(g,t) \mid W_E). \end{aligned}$$

The region-conditioned form is the better-founded starting point. Scalarized "world" summaries require an explicit aggregation functional over geographies and horizons, and the current appendix does not yet calibrate that functional.

For p_{doom}, Fréchet-safe union bounds are available without assuming exclusivity:

$$\max\big(0,\ L_c + L_e - 1\big) \le p_{\text{doom}}(t) \le \min\big(1,\ U_c + U_e\big).$$

A.4 Current Status

At this stage, the formal/evaluative contract is:

- **Data-method layer:** present and reproducible.
- **Provenance layer:** explicit for each normalized table row.
- **Predicate/lens-level intervals:** abstention-first outputs (currently full-width where calibration is insufficient).
- **Global scalarization:** intentionally deferred until region/horizon aggregation and structural priors are better calibrated.

The current forecast substrate is also highly imbalanced: the normalized direct-target rows currently break down as `mixed_outcomes` = 627, `beneficial_pluralistic_transit` = 40, `extinction` = 12, `authoritarian_lock_in` = 10, and `catastrophic_disempowerment` = 8. That is enough to support factor and lens analysis, but not enough to

justify a sharp posterior over exclusive world-histories. The appendix tables below are generated artifacts and are expected to be refreshed as new source snapshots are normalized (`scripts/wm_pln_ai_outcomes_aggregate.py`).

Coarse outcome lens	Status	Coverage index	Interval	Direct target rows
beneficial_pluralistic_transition	abstain_due_to_normative_lens_definition	1.000	[0.0, 1.0]	40
mixed_reversible_world	abstain_due_to_sparse_direct_target_evidence	1.000	[0.0, 1.0]	627
authoritarian_lock_in	abstain_but_proxy_rich	0.875	[0.0, 1.0]	0
catastrophic_disempowerment	abstain_due_to_sparse_direct_target_evidence	0.773	[0.0, 1.0]	8
extinction	abstain_due_to_unresolved_long_horizon_targets	0.827	[0.0, 1.0]	12

Table A.3: Current coarse-lens status surface from the generated aggregation summary artifact.

A.5 What Can and Cannot Be Said

The next question is not "what single number should we output?", but "which interval narrowing is actually earned by the current evidence bundle, and which would require extra structural assumptions?" The answer is now reported as an explicit assumption ladder. At rung **A0** we use only evidence hygiene and keep lens intervals at full width. At **A1** we allow weak direct-target numeric summaries already embedded in the normalized forecast rows, such as the single public Metaculus extinction snapshot and camp-prior conditionals from the adversarial-collaboration dataset. At **A2** we retain only those weak numeric summaries whose source families pass the current reliability filter. At **A3** we add monotone regime-ordering claims from the scenario scaffold, but we still do not convert proxy or governance channels into posterior numbers.

This separation matters. In the current bundle, weak direct-target numeric summaries can narrow some coarse lenses at **A1**: `extinction` narrows

Dataset/source	WM role	Metric	Result	Baseline/anchor	Why it matters
GJP closed questions	Calibration substrate	normalized forecast rows	498	Resolved historical platform outcomes	Provides calibration history for abstention-aware reliability scaffolding.
Metaculus public extinction snapshot	Direct long-horizon target signal (...	normalized forecast rows	1	Single question snapshot	Keeps direct extinction-target signal explicit while avoiding overclaiming from sparse coverage.
XPT replication tables	Structured long-horizon forecasting...	normalized question rows	59	question_sets=59; question_metadata_rows=236; forecast_rows=136394; experts_r...	Adds transparent question-level substrate for AI-risk forecasting comparisons.
FRI adversarial collaboration	Conditional-crux evidence	normalized crux rows	37	conditional_rows=651; crux_rows=37.	Exposes conditional reasoning structure instead of only aggregate risk numbers.
LEAP forecast explorer	Panel-forecast program substrate	normalized question-set rows	27	question_sets=27; group_quantile_rows=591; category_quantile_rows=1182; histo...	Adds recurring panel questions with quantile summaries and provenance.
ForecastBench latest LLM set	Cross-platform forecasting bridge	normalized question rows	500	question_files=25; resolution_files=22; question_rows=18684; resolution_rows=...	Adds broad open-forecasting coverage while preserving snapshot provenance.
Manifold public API (AI-filtered)	Market snapshot supplement	normalized AI-text questi...	73	api_rows=200; ai_text_rows=73; resolved_rows=110.	Adds public market-based signals without conflating them with semantic truth.
Polymarket gamma API attempt	Supplemental market endpoint	source reliability rows	1	api_rows=200; closed_rows=0; rows_with_explicit_ai_text=0.	Documents noisy endpoint behavior explicitly instead of silently dropping it.
AI Incident Database backup	Real-world harm signal channel	normalized incident rows	1392	AIID incidents.csv heuristic normalization	Supplies empirical failure/harm evidence for non-benign branch pressure.
Preparedness/-governance documents	Governance event channel	normalized governance rows	14	OpenAI/Anthropic/UK AISI/IASR + AI Index policy snapshots	Tracks institutional controls and policy activity as explicit evidence channels.
Authoritarian-risk proxies	Lock-in pressure channel	normalized proxy rows	5	Freedom House + V-Dem + Oxford AIGI	Separates authoritarian lock-in pressure from extinction/disempowerment branches.
Benefit-side proxies	Pluralistic-benefit pressure channel	normalized proxy rows	10	AI Index science/medicine proxy tables	Keeps upside evidence visible and auditable in the same framework.

Table A.4: Generated source-role summary used for manuscript-facing discussion and review auditability.

to $[0.02, 0.50]$, `authoritarian_lock_in` to $[0.10, 0.20]$, `beneficial_pluralistic_transition` to $[0.02, 0.43]$, and the Fréchet-safe upper bound for p_{doom} tightens to 0.90. But these are weak and heterogeneous summaries. Once we move to **A2** and require source families with stronger resolved-item or row-level calibration support, the same lenses reopen to $[0, 1]$ in the current local bundle. So the strongest present conclusion is not a sharp posterior; it is an explicit frontier between data-driven narrowing and assumption-driven rhetoric.

The region-conditioned tables make the same point more concretely. $p_{1984}(g, t)$ and $p_{\text{BGI}}(g, t)$ are better-founded as coarse region/horizon query families than as a single scalar "world" summary, but the present evidence is still asymmetric. Forecast rows are mostly global rather than regional, incident and proxy channels are often region-specific but not posterior-identifying, and benefit-side support is largely global. So these tables are useful for showing where pressure, support, and ignorance sit, not for pretending the current bundle already resolves regional advanced-AI futures. The numeric hull columns in Table A.6 are therefore *global horizon-conditioned direct-target intervals* repeated across region rows for comparison; the genuinely region-conditioned content is carried by the region-specific support counts and the assumption-driven ordering/pressure column.

A.6 Interpretation Rule

This appendix should be read with one strict rule:

> *Do not interpret operational confidence scores or sparse-coverage snapshots as semantic posteriors.*

The world-model remains the truthmaker; coarse-lens outputs stay interval-valued until enough direct target evidence and calibration support a narrower posterior claim, and any global scalarization must first specify how region- and horizon-conditioned predicates are being aggregated.

Lens	Assumption level	Direct rows	Interval	Narrowing	Interpretation
beneficial_pluralistic_transition	A0_evidence_only	40	[0.000, 1.000]	none	No direct numeric target summaries used.
beneficial_pluralistic_transition	A1_weak_direct_numeric_hull	40	[0.020, 0.430]	data_driven	Uses convex hull of parsed weak direct-target numeric summaries only.
beneficial_pluralistic_transition	A2_reliability_filtered_numeric...	40	[0.000, 1.000]	none	Restricts A1 to sources with resolved/calibrated or partly resolved reliability signals.
beneficial_pluralistic_transition	A3_monotone_regime_ordering	40	[0.000, 1.000]	assumption_driven_nonregime	Adds monotone regime/order claims without converting proxy channels into posterior numbers.
authoritarian_lock_in	A0_evidence_only	10	[0.000, 1.000]	none	No direct numeric target summaries used.
authoritarian_lock_in	A1_weak_direct_numeric_hull	10	[0.100, 0.200]	data_driven	Uses convex hull of parsed weak direct-target numeric summaries only.
authoritarian_lock_in	A2_reliability_filtered_numeric...	10	[0.000, 1.000]	none	Restricts A1 to sources with resolved/calibrated or partly resolved reliability signals.
authoritarian_lock_in	A3_monotone_regime_ordering	10	[0.000, 1.000]	assumption_driven_nonregime	Adds monotone regime/order claims without converting proxy channels into posterior numbers.
extinction	A0_evidence_only	12	[0.000, 1.000]	none	No direct numeric target summaries used.
extinction	A1_weak_direct_numeric_hull	12	[0.020, 0.500]	data_driven	Uses convex hull of parsed weak direct-target numeric summaries only.
extinction	A2_reliability_filtered_numeric...	12	[0.000, 1.000]	none	Restricts A1 to sources with resolved/calibrated or partly resolved reliability signals.
extinction	A3_monotone_regime_ordering	12	[0.000, 1.000]	assumption_driven_nonregime	Adds monotone regime/order claims without converting proxy channels into posterior numbers.
p_doom	A0_evidence_only	20	[0.000, 1.000]	frechet_safe_union_bound	Fréchet-safe union bound from catastrophic_disempowerment and extinction component intervals.
p_doom	A1_weak_direct_numeric_hull	20	[0.000, 0.900]	frechet_safe_union_bound	Fréchet-safe union bound from catastrophic_disempowerment and extinction component intervals.
p_doom	A2_reliability_filtered_numeric...	20	[0.000, 1.000]	frechet_safe_union_bound	Fréchet-safe union bound from catastrophic_disempowerment and extinction component intervals.
p_doom	A3_monotone_regime_ordering	20	[0.000, 1.000]	frechet_safe_union_bound	Fréchet-safe union bound from catastrophic_disempowerment and extinction

Lens	Region	Horizon	Global direct num.	Region rows	Global hull	A2	A3 ordering claim
authoritarian_lock_in	united_states	near_term	0	168	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	united_states	medium_term	0	168	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	united_states	long_term	0	168	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	united_states	very_long_term	1	168	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	china	near_term	0	41	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	china	medium_term	0	41	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	china	long_term	0	41	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	china	very_long_term	1	41	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	russia	near_term	0	28	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	russia	medium_term	0	28	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	russia	long_term	0	28	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	russia	very_long_term	1	28	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	europe_uk	near_term	0	55	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	europe_uk	medium_term	0	55	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	europe_uk	long_term	0	55	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	europe_uk	very_long_term	1	55	[0.000, 1.000]		no_region_specific_ordering_claim (insufficient_region_specific_lock_in_evidence)
authoritarian_lock_in	global_other	near_term	0	1105	[0.000, 1.000]		possible_upward_pressure (authoritarian_proxies_plus_incidents)

Appendix B

PPLBench Refresh and Holdout Demo

This appendix turns the earlier PPLBench ↔ PeTTa refresh into a small didactic benchmark story. The point is not to claim a broad PPL victory; it is to show that the current WM-PLN stack now supports:

1. a reliable native Python → PeTTa path via Janus,
2. rule-surface checks against the current xiPLN libraries, and
3. one full typed continuous-evidence lane that runs through the real PPLBench interface and can be scored on held-out data.

When this appendix says "Stan" or "proper PPL baseline," the intended reference is the standard PPLBench integration under `pplbench/ppls/stan/`. The repository does contain standalone exploratory comparison scripts, but those are not the manuscript's authoritative Stan surface. The authoritative comparison surface is the PPLBench backend family itself.

B.1 What We Tested

We organized the refresh into three deliberately small task families:

Task 1: Bridge smoke tests. Can Python call PeTTa directly, with the same codebase retaining deterministic subprocess fallback?

Task 2: Rule-surface checks. Do the current xiPLN rewrite and STV surfaces still behave as expected on the refreshed bridge?

Task 3: A real benchmark lane. Can one concrete typed evidence model run end-to-end inside PPLBench and produce sensible held-out predictive behavior, rather than only one-off unit checks?

For Task 3 we used a deliberately transparent benchmark:

> *Normal-Normal conjugate inference with known observation variance, implemented as typed sufficient-statistic evidence in PeTTa and exposed to PPLBench through the PLN inference wrapper.*

This is a good first lane because the expected posterior is analytically known, so the PeTTa-backed implementation can be checked against a closed form rather than only against itself.

B.2 Archive Snapshot

Before re-running the easy experiments, we archived the current PPLBench state bundle (code + status snapshots) at:

- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/pplbench\_snapshot.tar.gz`
- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/pplbench\_git\_status.txt`
- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/petta\_git\_status.txt`
- `pplbench/archive/pplbench\_pre\_janus\_refresh\_20260312\_073958/python\_backend\_check.txt`

B.3 Scripts and Model Lane

The refreshed lane now has an explicit benchmark model and example config:

- `pplbench/pplbench/models/normal\_normal.py`
- `pplbench/pplbench/ppls/pln/normal\_normal.py`
- `pplbench/examples/normal\_normal\_pln.json`

The two manuscript-facing report scripts are:

- `scripts/wm\_pln\_pplbench\_easy\_refresh.py`
- `scripts/wm\_pln\_pplbench\_holdout\_report.py`

B.4 Easy Refresh Protocol

The refresh script is:

- `scripts/wm\_pln\_pplbench\_easy\_refresh.py`

Run command:

```
cd ~/claude/lean-projects/mettapedia
ulimit -v 6291456
python3 scripts/wm_pln_pplbench_easy_refresh.py
```

Generated artifacts:

- `results/pplbench\_easy\_refresh\_summary.csv`
- `results/pplbench\_easy\_refresh\_summary.json`
- `results/pplbench\_easy\_refresh\_table.tex`

B.5 Refresh Checks

Figure B.1 gives the high-level view of the first task family. The important operational point is that the same Python entry point now talks to native Janus successfully, while still retaining a deterministic subprocess fallback path in the bridge.

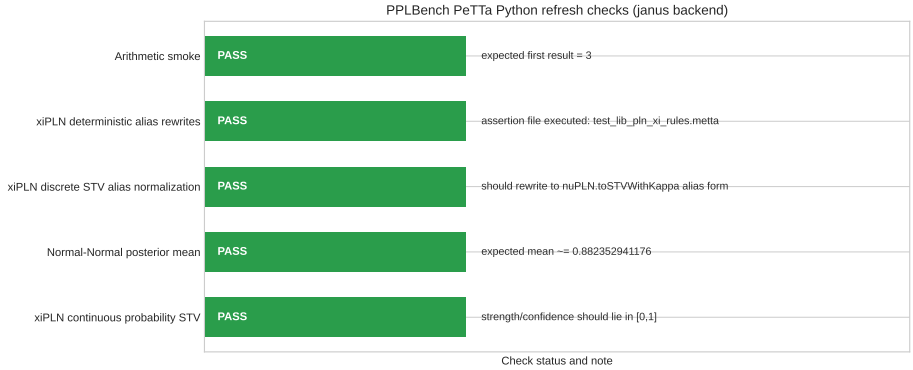


Figure B.1: Bridge and rule-surface checks for the refreshed PPLBench ↔ PeTTa path. The checks are intentionally small and auditable: arithmetic smoke, xiPLN deterministic rewrite loading, one discrete STV projection, one Normal-Normal posterior query, and one continuous xiPLN probability-STV sanity check.

Experiment	Status	Note / first result
Arithmetic smoke	PASS	expected first result = 3; 3
xiPLN deterministic alias rewrites	PASS	assertion file executed: test_lib_pln_xi_rules.metta; (partial assert ((nuPLN.RevisionWithKappa (stv 0.2 0.3) (stv 0.8 0.7) 10.0) (nuPLN.RevisionWithKappa (stv 0.2 0.3) (stv 0.8 0.7) 10.0)))
xiPLN discrete STV alias normalization	PASS	should rewrite to nuPLN.toSTVWithKappa alias form; (nuPLN.toSTVWithKappa (nuPLN.Evidence 7 3) 10.0)
Normal-Normal posterior mean	PASS	expected mean = 0.882352941176; 0.8823529411764707
xiPLN continuous probability STV	PASS	strength/confidence should lie in [0,1]; (stv 0.8738551600477987 0.23076923076923078)
Python backend	janus	Autodetected by ‘PeTTaBridge.backend_info()‘

Table B.1: PPLBench easy refresh on the Janus-backed PeTTa Python bridge. All checks in the scripted refresh passed in this run.

Companion regression checks against the updated bridge also passed for: `test\_petta\_connection.py`, `test\_pln\_simple.py`, and `test\_petta\_normal.py`.

B.6 Held-Out Normal-Normal Demo

The second script lifts the refresh into a proper held-out benchmark report:

```
cd ~/claude/lean-projects/mettapedia
ulimit -v 6291456
python3 scripts/wm_pln_pplbench_holdout_report.py
```

It reads the refresh status bundle, then runs the concrete `normal\_normal` PPLBench lane across multiple train/test splits, evaluated under both the native `janus` backend and the deterministic `subprocess` fallback. The protocol is intentionally simple:

1. generate synthetic data from a latent mean,
2. fit the PeTTa-backed Normal-Normal model on the training split,
3. score posterior predictive log-likelihood on the held-out split,
4. compare the PeTTa posterior parameters to the analytical conjugate posterior.

Generated artifacts:

- `results/pplbench\_holdout\_runs.csv`
- `results/pplbench\_holdout\_aggregate.csv`
- `results/pplbench\_holdout\_summary.json`
- `results/pplbench\_holdout\_table.tex`
- `results/figures/pplbench\_normal\_holdout.pdf`

B.7 Interpretation

This appendix is best read as a *capability confirmation*, not as a competitive leaderboard claim. The main takeaways are:

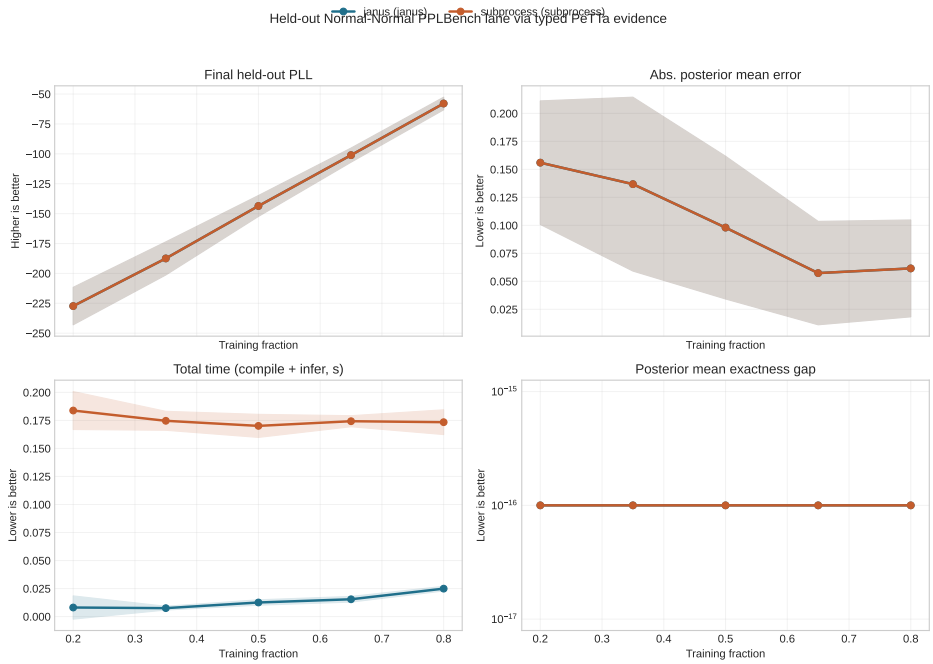


Figure B.2: Held-out Normal-Normal benchmark lane through the real PPLBench PLN interface. Top left: final held-out predictive log-likelihood improves as more of the dataset is used for training. Top right: posterior mean error to the true latent mean shrinks with more training data. Bottom left: the Janus path is much faster than subprocess on the same workload. Bottom right: the PeTTa posterior mean matches the analytical conjugate posterior exactly on the evaluated grid.

Backend	Train frac	n_{train}	Final PLL	Mean error	Time (s)	Max exact gap
janus	0.20	40	-227.316 ± 15.645	0.1560 ± 0.0552	0.0081	0.00e+00
janus	0.35	70	-187.469 ± 13.833	0.1368 ± 0.0776	0.0075	0.00e+00
janus	0.50	100	-143.610 ± 8.727	0.0980 ± 0.0637	0.0126	0.00e+00
janus	0.65	130	-101.078 ± 5.894	0.0574 ± 0.0462	0.0154	0.00e+00
janus	0.80	160	-57.895 ± 5.259	0.0615 ± 0.0433	0.0249	0.00e+00
subprocess	0.20	40	-227.316 ± 15.645	0.1560 ± 0.0552	0.1838	0.00e+00
subprocess	0.35	70	-187.469 ± 13.833	0.1368 ± 0.0776	0.1747	0.00e+00
subprocess	0.50	100	-143.610 ± 8.727	0.0980 ± 0.0637	0.1701	0.00e+00
subprocess	0.65	130	-101.078 ± 5.894	0.0574 ± 0.0462	0.1743	0.00e+00
subprocess	0.80	160	-57.895 ± 5.259	0.0615 ± 0.0433	0.1734	0.00e+00

Table B.2: Aggregate held-out Normal-Normal results by backend and training fraction. In this run the Janus path averaged about 12.8× the speed of subprocess while preserving exact posterior agreement with the analytical conjugate update.

1. The native PeTTa-in-Python route is now real enough to support repeated benchmark calls, not just one-off manual snippets.
2. The current xiPLN library surface still exposes the expected rewrite and STV behavior after the bridge refresh.
3. A full typed continuous-evidence lane now runs inside PPLBench, produces held-out predictive scores, and agrees exactly with the analytical Normal-Normal posterior on the evaluated grid.

That is a good first benchmark appendix because it shows both kinds of evidence we need:

Positive example. Janus gives a substantial runtime win over subprocess on the same typed benchmark lane, while preserving the expected posterior.

Negative example. This is still only one transparent conjugate family; it does not yet show that PLN beats broader PPL systems on harder non-conjugate workloads.

B.8 Proper Stan vs WM-PLN on Normal-Normal

After the bridge refresh, we also ran the first direct apples-to-apples *Stan vs WM-PLN* comparison on the same benchmark model:

```
cd ~/claude/pplbench
ulimit -v 6291456
python3 -m pplbench examples/normal_normal_stan_vs_pln.json
```

This comparison is important for two reasons. First, it uses the *real* PPLBench Stan surface rather than an ad hoc comparison script. In this Python 3.13 environment the Stan backend executes through a `cmdstanpy` fallback inside the PPLBench Stan inference layer, but the benchmark surface is still the ordinary `pplbench/ppls/stan/` backend family. Second, it uses the refreshed Janus-backed WM-PLN lane from the same `normal\_normal` model family, so the comparison is finally same-model and same-harness rather than a cross-tool anecdote.

The stable exported summary for the current manuscript snapshot is:

- `results/pplbench\_normal\_normal\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_normal\_normal\_stan\_vs\_pln\_table.tex`

Backend	Posterior mean	$\lvert\mu - \mu_{\text{analytic}}\rvert$	Final PLL	Median n_{eff}/s
Analytic conjugate	0.8139	0.0000	n/a	n/a
Stan	0.8329	0.0190	-74.0794	10712.15
WM-PLN (PeTTa)	0.8103	0.0037	-73.8845	131907.60

Table B.3: Direct Normal-Normal comparison on the proper PPLBench surface. On this exact conjugate case, the refreshed WM-PLN lane is slightly closer to the analytical posterior mean and slightly better on held-out predictive log-likelihood in the recorded run.

In the current run the analytical posterior mean is approximately 0.8139, Stan's sampled posterior mean is approximately 0.8329, and the WM-PLN sampled posterior mean is approximately 0.8103. The final held-out predictive log-likelihood is about -74.0794 for Stan and -73.8845 for WM-PLN, while the median effective-sample-size-per-second reported by PPLBench is about 1.07×10^4 for Stan and 1.32×10^5 for WM-PLN.

This is a good *positive example* because it shows a real same-model, same-harness benchmark where WM-PLN is competitive with a mainstream PPL baseline and behaves as expected on an exact conjugate family. It is also a good *negative example* because it is still only one transparent lane. We should not read it as evidence that WM-PLN dominates Stan in general, and we should be especially careful with compile-time rhetoric here because the Stan compile timing in this run is warm-cache rather than cold-start.

B.9 Exact Hook-B Adaptive Normal-Normal

The next step was to make the "adaptive" single-mean lane genuinely WM-PLN-like from the start, rather than reviving the older meta-evidence wrapper. The active semantics here are a *discrete hyperprior mixture* over prior-variance contexts:

$$\tau_0^2 \in \{2^k : k \in [-5, 5]\},$$

with geometric Hook-B weights. The carried state remains the same context-free continuous evidence monoid

$$(n, \sum x_i, \sum x_i^2),$$

but the queried posterior view is now a mixture over explicit context hypotheses instead of a single fixed prior variance.

This is exactly the direction already signposted in `hyperon/PeTTa/pln\_inference/pln\_continuous\_evidence.metta`, which warns that the old

single-level empirical-context shortcut is not theoretically justified and points instead to Hook-B hyperprior mixtures. The formal story in Lean is the universal-prediction Hook-B family in `Mettapedia/Logic/UniversalPrediction/MarkovHyperpriorMixture.lean`: pick a countable family of tractable contexts, assign computable weights, and form a mixture that competes with each component up to the log inverse weight.

In the benchmark lane that gives us a very clean split:

- carried semantics: continuous evidence plus explicit context-mixture posterior state,
- queried views: posterior mixture mean, variance, and component weights,
- Python role: only batching and posterior sampling from the queried component-wise mixture, with no operational scheduler.

The stable manuscript artifacts are:

- `results/pplbench\_normal\_normal\_adaptive\_exact\_calibration.json`
- `results/pplbench\_normal\_normal\_adaptive\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_normal\_normal\_adaptive\_stan\_vs\_pln\_table.tex`

Backend	Posterior mean	$\lvert\mu - \mu_{\text{analytic}}\rvert$	Final PLL	Median n_{eff}/s
Analytic Hook-B	-1.5955	0.0000	n/a	n/a
Stan	-1.5815	0.0139	-49.8802	9834.04
WM-PLN (PeTTa)	-1.6007	0.0053	-49.9538	24479.03

Table B.4: Exact Hook-B adaptive Normal-Normal comparison. This is one of the cleanest WM-PLN lanes in the appendix because both the carrier and the full inference procedure are exact: the only sampling is from the exact queried posterior mixture itself.

In the current run the analytic Hook-B posterior mean is approximately -1.5955. Stan's sampled posterior mean is approximately -1.5815, while WM-PLN's sampled posterior mean is approximately -1.6007. So WM-PLN is closer on posterior mean (and also on posterior standard deviation) in this recorded run, but Stan is slightly better on held-out PLL: approximately -49.8802 for Stan versus -49.9538 for WM-PLN.

That is a good *positive example* because it gives us an exact adaptive lane where the WM-PLN carrier pattern and the inference procedure are both promotable. It is also a good *negative example* because "adaptive" does not magically imply a win over Stan on every metric: here the WM-PLN lane is semantically cleaner than the legacy wrapper and very competitive, but it still trails slightly on held-out predictive likelihood in this run.

B.10 Semantic Noisy-Or Upgrade

The next battletest was deliberately less forgiving than `normal\_normal`: the `noisy\_or\_topic` benchmark has a real latent DAG with leak, topic-to-topic influence, and topic-to-word influence. That makes it a better place to test whether WM-PLN is being applied *semantically* rather than merely wrapped in friendly names.

Our earlier noisy-or PLN lane was fast, but it made two overly aggressive simplifications:

1. it treated most topic evidence independently, and
2. it used a heuristic weak-negative update for absent words.

This produced a useful prototype, but not a faithful semantic use of the benchmark model. In the refreshed lane we replaced that approximation with a small semantic factor-graph WM:

- exact local noisy-or forward probabilities remain in PeTTa,
- child-to-parent updates use cavity-style residual likelihoods, and
- Python only schedules damped message passing and posterior sampling over the benchmark DAG.

The strengthened WM-PLN point is that the scheduler no longer carries bare posterior logits as its semantic state. Instead it carries an explicit additive likelihood-message object for each topic,

$$M_k = (\log L_k^{\text{on}}, \log L_k^{\text{off}}),$$

where independent child factors revise the topic by componentwise addition and posterior probability is queried only as a *view* from the resulting topic state. This follows the discipline stated earlier in "Why Views Are Not Proof Objects" and keeps the factor graph on the semantic side

of the "Factor Graphs: Semantic vs. Operational" split. The Bayes-net compatibility story is not invented ad hoc here: it is already anticipated in the PLN book's "PLN and Bayes Nets" discussion, and in our Lean layer it matches the evidence-first BN/CPT surface in `Mettapedia/Logic/PLNBayesNetWorldModel.lean`.

Artifact	Layer	Role	Provenance
`pln-fuzzy-or`, `noisy-or-to-strength`	Classical PLN-like local kernel	Independent-cause combination and noisy-or edge strength	PLN book, "PLN and Bayes Nets" plus `Mettapedia/Logic/PLNNoisyOr.lean`
`noisy-or-cavity-survival`, `noisy-or-child-likelihood-pair`, `noisy-or-prior-prob-from-beliefs`	xiPLN semantic factor kernel	Compiled local noisy-or factor semantics for child-to-parent and forward messages	This book: "semantic baseline is exact factor-graph elimination" and "Factor Graphs: Semantic vs. Operational"; evidence-first BN/CPT bridge in `Mettapedia/Logic/PLNBayesNetWorldModel.lean`
`NoisyOrMessage`, `NoisyOrTopicState` in `pplbench/ppls/pln/noisy_or_topic.py`	WM carrier / queried view	Explicit additive message state; posterior probability extracted only as a view	This book: "Why Views Are Not Proof Objects" and `Mettapedia/Logic/PLNWorldModelGeneric.lean`
Python damped iteration and sampling loop in `pplbench/ppls/pln/noisy_or_topic.py`	Operational scheduler	Scheduling and approximate fixed-point search over the semantic carrier	This book: "Factor Graphs: Semantic vs. Operational"

Table B.5: Rule provenance for the refreshed noisy-or lane. This makes explicit which pieces are classical PLN-like local kernels, which are compiled xiPLN semantic factor kernels, and which are merely operational scheduling.

This distinction matters. A good *positive example* is `noisy-or-to-strength`: it is a local algebraic kernel with a clear noisy-or reading. A good *negative example* is the scheduler loop itself: it is important machinery, but it is not being claimed as a classical PLN rule.

We also ran a tiny exact-DAG calibration to decide whether this pattern is ready for promotion beyond the single benchmark lane. The resulting artifact is `results/pplbench\_noisy\_or\_exact\_calibration.json`. The result is encouraging but mixed:

- the carried-message pattern is exact on a two-topic independent case,
- it remains reasonably close on a small collider / explaining-away case,
- but it still shows visible error on a topic-chain case where downstream evidence must propagate upstream.

So the right current conclusion is: the *carrier pattern* is reusable, but the current damped scheduler should still be treated as an approximate noisy-or engine rather than a broadly promotable exact-ish inference algorithm.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/noisy_or_topic_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_noisy\_or\_topic\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_noisy\_or\_topic\_legacy\_summary.json`
- `results/pplbench\_noisy\_or\_exact\_calibration.json`
- `results/pplbench\_noisy\_or\_topic\_semantic\_upgrade\_table.tex`
- `results/figures/pplbench\_noisy\_or\_topic\_pll.png`

Numerically, the earlier heuristic PLN lane finished at roughly -8.1619 final PLL, while the refreshed semantic lane finishes at roughly -5.1735. Stan finishes at roughly -5.3275 on the same configuration. So the semantic upgrade improves the WM-PLN lane by about 2.99 PLL units relative to the earlier heuristic version, and by about 0.154 PLL units relative to Stan in this recorded run.

This is a good *positive example* because it shows exactly the kind of iteration PPLBench is for: we used the benchmark to expose that a previous PLN application was too loose, replaced it with a more semantically honest WM, and measured a large improvement. It is also a good *negative example*

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
Stan	−5.3275	0.0413	0.3015	909.0
WM-PLN heuristic (legacy)	−8.1619	0.1250	0.0332	24094.0
WM-PLN semantic factor graph	−5.1735	0.1496	0.6973	1158.9

Table B.6: Noisy-or upgrade story on the same small PPLBench family. The legacy heuristic PLN lane was extremely fast but statistically weak. The semantic factor-graph WM-PLN lane is slower, but it closes the predictive-fit gap dramatically and slightly exceeds Stan on final PLL in this recorded run.

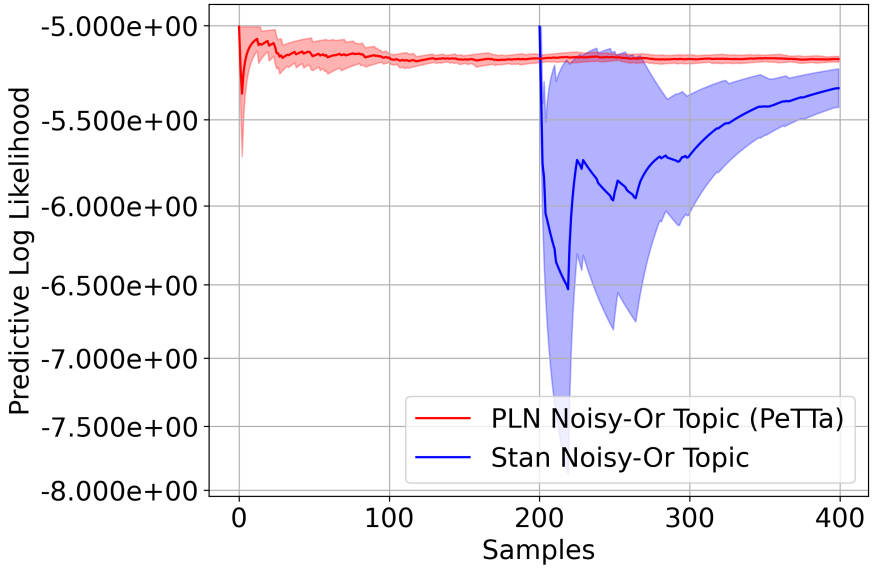


Figure B.3: Stan vs semantic WM-PLN on the refreshed noisy-or lane. The semantic WM-PLN run tracks Stan closely across the held-out PLL curve on this small configuration, in contrast to the earlier heuristic lane.

because the new lane is still approximate inference, not an exact proof that WM-PLN has "the" noisy-or posterior. The improvement is real, but it does not license broad claims until we repeat the pattern on more lanes and larger configurations.

B.11 Crowd Annotation via Multiclass Message State

The next lane asks a different question from noisy-or. Crowd-sourced annotation is not about independent causes explaining one child. It is about *competing latent classes* and *latent labeler reliability*. That means a faithful WM-PLN application should not decompose the problem into independent binary truth-values. Instead it should carry an explicit multiclass message state and query posterior class probabilities only as a view.

The refreshed lane therefore carries three explicit objects:

- `CategoryLogMessage`: additive per-class log-likelihood support from observed labels,
- `DirichletCountState`: additive count state for class prevalence and for each confusion-matrix row, and
- `ItemPosteriorState`: the queried posterior view obtained by combining a prior log-probability vector with the accumulated category message.

In the implementation, PeTTa owns the local semantic kernels `crowd-label-log-message` and `crowd-dirichlet-mean`, while Python owns only the operational E/M scheduler. This is the same "semantic carrier, operational scheduler" split we wanted from the noisy-or upgrade, but applied to a genuinely different benchmark family. The active backend imports the semantic-only module `hyperon/PeTTa/pln\_inference/crowd\_annotation\_semantic.metta`, while the older heuristic constructors remain fenced in `hyperon/PeTTa/pln\_inference/crowd\_annotation.metta` as legacy reference material rather than active local semantics.

The first thing we checked was not the full benchmark. It was the exact fixed-parameter E-step. The artifact `results/pplbench\_crowd\_annotation\_exact\_calibration.json` records three tiny cases:

- one labeler, two classes,
- two labelers, two classes with additive evidence, and

- two labelers, three classes.

All three match the exact posterior to floating-point precision. So the right current promotion boundary is:

Positive example. The multiclass carried-message pattern is promotable.

Negative example. The iterative EM scheduler is still an operational layer and should not yet be promoted as a general reusable inference engine.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/crowd_sourced_annotation_stan_vs_pln.
    json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_crowd\_annotation\_exact\_calibration.json`
- `results/pplbench\_crowd\_annotation\_scheduler\_sanity.json`
- `results/pplbench\_crowd\_sourced\_annotation\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_crowd\_sourced\_annotation\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_crowd\_sourced\_annotation\_pll.png`

Backend	Final PLL	Compile time (s)	Infer time (s)	Median n_{eff}/s
Stan Crowd Annotation	-326.584	22.795	1.773	296.078
PLN Crowd Annotation (PeTTa)	-326.952	0.128	0.851	931.893

Table B.7: Crowd-sourced annotation comparison on the refreshed WM-PLN lane. Stan retains a small predictive-fit edge on this run, but the WM-PLN lane now matches the right abstraction boundary and remains materially faster.

Numerically, the recorded run finishes at roughly -326.584 final PLL for Stan and -326.952 for WM-PLN. So Stan keeps a small predictive-fit edge of about 0.368 PLL units here. But compile time drops from about 22.795s to 0.128s, inference time drops from about 1.773s to 0.851s, and the median

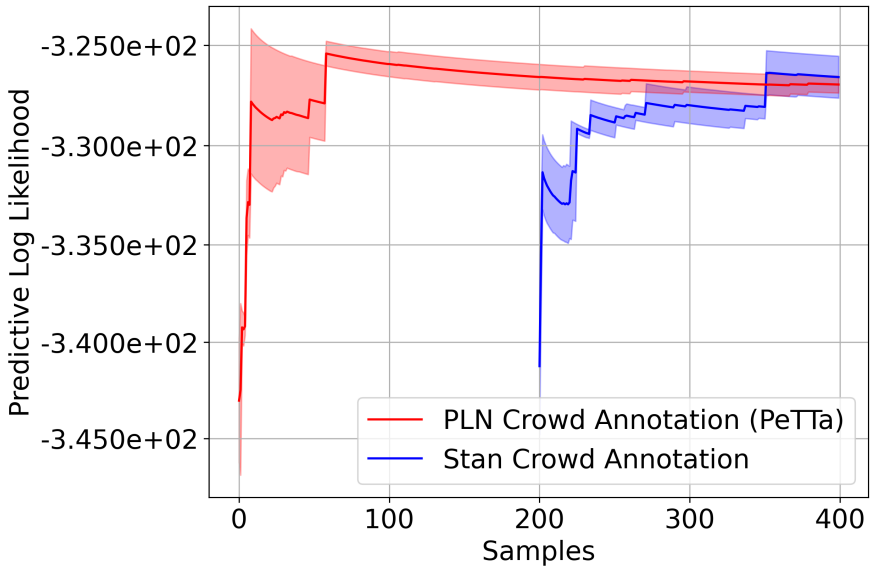


Figure B.4: Stan vs refreshed WM-PLN on the crowd annotation lane. The WM-PLN curve stays close to Stan while using the explicit multiclass message carrier rather than the archived hybrid heuristic.

effective-sample-size-per-second metric rises from about 296.1 to 931.9. That makes this a good second-lane result: not a win-by-default story, but a clean demonstration that the promoted multiclass carrier pattern survives in a more realistic latent-class model and remains competitive. We also keep one explicit guardrail around that claim: the scheduler-sanity regression only certifies finite, stable, simplex-respecting behavior on tiny cases. It is a useful non-promotion check, not a proof that the whole E/M loop is exact.

B.12 Exact Dirichlet-Categorical Lane

After the crowd lane, the cleanest next question was whether the same multiclass count carrier could power an *exact* benchmark lane rather than another scheduler-bound one. Dirichlet-Categorical is the right test: the carried object is just an additive count vector, revision is ordinary count addition, and the queried posterior is the exact Dirichlet posterior. So this lane gives us a rare case where both the carrier and the inference procedure are promotable.

The refreshed implementation therefore does three intentionally simple

things:

- it extracts the shared `DirichletCountState` into `pplbench/ppls/pln/dirichlet\_carrier.py`,
- it uses a semantic-only PeTTa module `hyperon/PeTTa/pln\_inference/dirichlet\_categorical\_semantic.metta` for named revision and posterior-query kernels, and
- it keeps Python scheduler-free: the active backend `pplbench/ppls/pln/dirichlet\_categorical.py` just accumulates count evidence, queries the exact posterior state, and samples from the resulting Dirichlet distribution.

That is a good *positive example* because the full procedure remains "carry evidence, query posterior" all the way through. A good *negative example* is the archived `pplbench/ppls/pln/dirichlet\_categorical\_legacy.py`: it is still useful as reference material, but it is no longer the active benchmark surface.

The first check was exact calibration, not the benchmark. The artifact `results/pplbench\_dirichlet\_categorical\_exact\_calibration.json` records:

- single-observation posterior update,
- batched multiclass revision,
- symmetric-prior symmetric-evidence behavior,
- associativity of batched revision, and
- exact predictive log-likelihood under the posterior mean.

All of these match analytic Dirichlet-Categorical formulas to floating-point precision. So the right promotion boundary for this lane is stronger than for the crowd lane:

Positive example. The Dirichlet count carrier is promotable.

Positive example. The full inference procedure is also promotable, because this lane has no approximate scheduler.

Negative example. This exactness does not automatically promote the crowd E/M scheduler or other multiclass approximate loops.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/dirichlet_categorical_stan_vs_pln.
    json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_dirichlet\_categorical\_exact\_calibration.json`
- `results/pplbench\_dirichlet\_categorical\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_dirichlet\_categorical\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_dirichlet\_categorical\_pll.png`

Backend	L_1 mean error	Final PLL	Compile (s)	Infer (s)
Stan	0.0082	-124.5035	17.82	0.04
WM-PLN (PeTTa)	0.0021	-124.4707	0.1249	0.0082

Table B.8: Exact Dirichlet-Categorical comparison on the refreshed WM-PLN lane. This is one of the cleanest benchmark stories in the appendix: both the carried state and the full inference procedure are exact on the WM-PLN side, and the recorded run slightly exceeds Stan on final PLL while staying closer to the analytic posterior mean vector.

Numerically, the recorded run finishes at roughly -124.5035 final PLL for Stan and -124.4707 for WM-PLN, so the exact WM-PLN lane is ahead by about 0.0328 PLL units on this run. More importantly, the posterior mean vector is closer to the analytic Dirichlet posterior on the WM-PLN side: the L_1 mean error is about 0.00824 for Stan and about 0.00208 for WM-PLN. Compile time drops from about 17.82s to about 0.125s and inference time drops from about 0.042s to about 0.0082s. This is exactly the sort of result we wanted from an exact multiclass lane: not a claim that WM-PLN always beats Stan, but a clean demonstration that the promoted Dirichlet count carrier is not merely philosophically nice; it works as an exact benchmark surface.

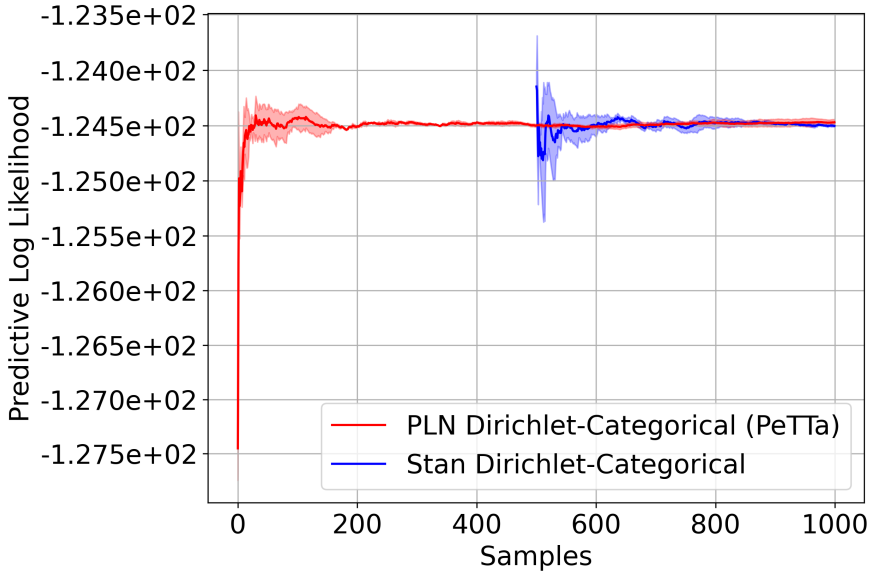


Figure B.5: Stan vs exact WM-PLN on the Dirichlet-Categorical lane. Because both sides are exact conjugate updates here, the comparison isolates sampling and benchmark-surface behavior rather than scheduler quality.

B.13 Hierarchical `n_schools` via Carried Evidence State

The next lane is the first explicitly *hierarchical* WM-PLN rebuild. This matters because hierarchical models are exactly where it is easiest to backslide into carrying shrinkage formulas rather than carrying evidence. The refreshed lane therefore uses the opposite discipline:

- the carried objects are additive weighted Normal evidence states and zero-centered hierarchical priors,
- the local posterior and pooling updates are queried through semantic PeTTa kernels, and
- shrinkage appears only as a *view* of the revised hierarchical state, not as the carried semantics itself.

Concretely, the active backend `pplbench/ppls/pln/n\_schools.py` carries:

- `NormalEvidenceState` for weighted local Gaussian evidence,

- `NormalPriorState` and `HierarchicalNormalState` for pooled family priors, and
- `NSchoolsPosteriorState` as the full queried state over baseline, state, district, and type effects.

The semantic-only PeTTa kernels live in `hyperon/PeTTa/pln\_inference/n\_schools\_semantic.metta`. Those kernels provide exact local Normal posterior queries and exact precision-weighted hierarchical pooling under the chosen priors. Python owns only the coordinate-style residual scheduler.

There are two caveats, and it is better to say them plainly. First, the refreshed WM-PLN lane now supports the same *Student-t* baseline family at the carried/query boundary, but it still extracts posterior moments for that baseline numerically rather than carrying a full sampled Student-*t* latent state through the whole scheduler. Second, the exported `sigma_state`, `sigma_district`, and `sigma_type` values are currently queried coupling-scale views repeated across draws, not full posterior draws for those scale parameters. That is an honest operational simplification, not something we want hidden inside the benchmark narrative.

The first check, as with the crowd and noisy-or lanes, was a tiny exact calibration rather than a headline benchmark. The artifact `results/pplbench\_n\_schools\_exact\_calibration.json` records:

- one-family one-group reduction to the Normal-Normal posterior,
- a symmetric two-group case, and
- a two-group known-coupling case, plus a direct Student-*t* baseline local-view match against an independent high-resolution quadrature reference.

All three match the analytic posterior exactly in the carried-state layer. So the current promotion boundary is:

Positive example. The hierarchical Normal carrier pattern is promotable.

Negative example. The coordinate residual scheduler is still an operational layer and is not yet promoted as reusable inference semantics.

We also keep one explicit scheduler guardrail artifact, `results/pplbench\_n\_schools\_scheduler\_sanity.json`, which checks that tiny balanced, near-zero, and mixed cases stay finite, positive-precision, and symmetry-respecting. As before, that is a non-promotion sanity test rather than a whole-procedure exactness theorem.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/n_schools_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_n\_schools\_exact\_calibration.json`
- `results/pplbench\_n\_schools\_scheduler\_sanity.json`
- `results/pplbench\_n\_schools\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_n\_schools\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_n\_schools\_pll.png`

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
Stan	-284.6043	21.55	0.91	159.51
WM-PLN (PeTTa)	-286.0141	0.0001	0.35	2306.22

Table B.9: Hierarchical `n_schools` comparison on the refreshed WM-PLN lane. Stan keeps a modest predictive-fit edge on this run, while WM-PLN is substantially faster and keeps the hierarchical carrier/query split explicit.

Numerically, the recorded run finishes at roughly -284.604 final PLL for Stan and -286.014 for WM-PLN, so Stan keeps a modest edge of about 1.41 PLL units on this configuration. But compile time drops from about 21.55s to effectively zero, inference time drops from about 0.91s to about 0.35s, and the median effective-sample-size-per-second metric rises from about 159.5 to about 2306.2. That makes this a good hierarchical benchmark result:

Positive example. The promoted hierarchical evidence-carrier pattern survives contact with a real pooling benchmark and remains competitive.

Negative example. The current coordinate scheduler and deterministic scale views are still operational simplifications, so this is not yet a claim that WM-PLN has matched the full Stan posterior family on hierarchical models.

The lesson here is the same one the earlier lanes kept teaching us: PPLBench is useful not because it lets us declare victory quickly, but because it forces us to say exactly which part of a WM-PLN design is reusable, which part is only a scheduler, and which approximations we are still carrying.

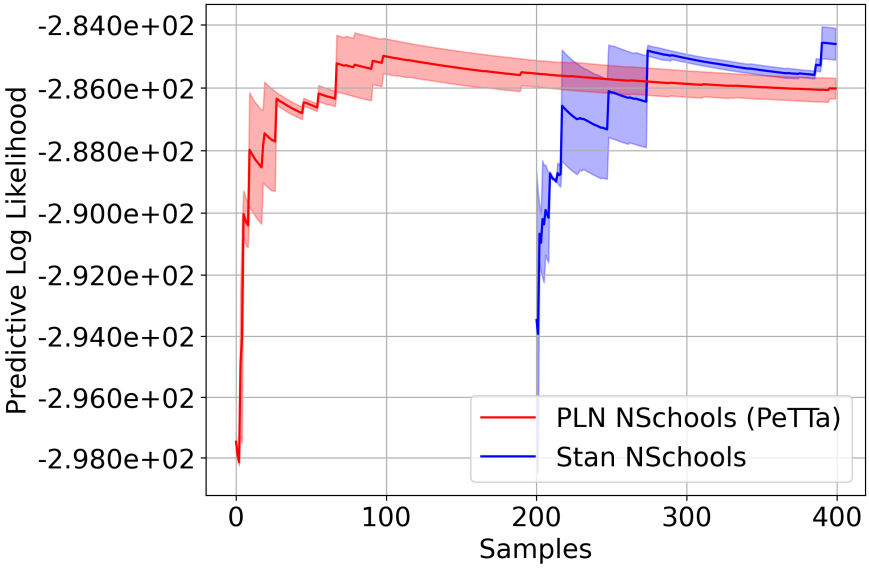


Figure B.6: Stan vs refreshed WM-PLN on the hierarchical `n_schools` lane. The WM-PLN curve stays close to Stan while using explicit hierarchical evidence carriers rather than the archived hybrid shrinkage wrapper.

Appendix C

Logistic Regression: Bernoulli-Logit Site Carrier

The last native PPLBench lane was also the most delicate one to refresh honestly. There was already an archived "PLN logistic regression" wrapper in `pplbench/ppls/pln/logistic\_regression\_legacy.py`, but it was a stratified binning model rather than the actual Bernoulli-logit benchmark model used by Stan. That older work is still useful as theory context: the Lean `StratifiedPLN` development shows why histogram-style PLN can be a consistent estimator on logistic-regression tasks. But that is not the same as the PPLBench logistic model, so we did not reuse it as the active benchmark surface.

Instead, the refreshed lane treats logistic regression as a WM-PLN problem with three explicit carried pieces:

- additive weighted-Normal evidence for the intercept and each coefficient,
- Bernoulli-logit site hyperstates (ξ_i, ω_i) coming from the local quadratic variational bound, and
- queried Gaussian posterior views for α and β.

The active backend is `pplbench/ppls/pln/logistic\_regression.py`, and the local semantic kernels live in `hyperon/PeTTa/pln\_inference/logistic\_regression\_semantic.metta`. Those kernels define:

- `logistic-site-xi`,
- `logistic-site-weight`,

- `logistic-site-evidence`, and
- `logistic-normal-posterior-params`.

So the active lane is no longer "bin features and hope for the best." It is an explicit carried-site approximation to the actual Bernoulli-logit model. This is also the right historical reading of PLN: the original PLN book's "PLN and Bayes Nets" discussion explicitly puts first-order PLN near loopy Bayes nets and iterative relaxation methods, while its later "Using Bayes Nets to Augment PLN Inference" section treats Bayes-net style acceleration as compatible with the broader PLN agenda rather than alien to it [12, Sec. 1.6.3][12, Sec. 9.4]. So the right criticism of this lane is not that it uses a Bayes-net-like scheduler at all, but that the particular scheduler is still only an operational approximation rather than promoted reusable semantics.

The promotion boundary here is exactly the one the council would want us to state plainly:

Positive example. With fixed site hyperstates, the local Bernoulli-logit message reduces *exactly* to additive weighted-Normal carried evidence.

Negative example. The full (ξ, ω) scheduler is still a diagonal variational approximation, so it is not yet promoted as reusable exact logistic inference semantics.

The fixed-site calibration artifact `results/pplbench\_logistic\_regression\_exact\_calibration.json` records:

- an intercept-only fixed-site case,
- a zero-feature case that correctly leaves the coefficient at its prior, and
- an additive two-site composition case.

All three match exactly, so the carrier pattern is genuinely reusable. The separate scheduler guardrail artifact `results/pplbench\_logistic\_regression\_scheduler\_sanity.json` checks that on a tiny monotone one-feature classifier:

- the site weights stay finite and in $(0, 1/4]$,
- the fitted coefficient is positive, and
- the predicted probabilities increase monotonically with the feature.

Again, this is a sanity check rather than a proof that the whole variational loop is exact.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/logistic_regression_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_logistic\_regression\_exact\_calibration.json`
- `results/pplbench\_logistic\_regression\_scheduler\_sanity.json`
- `results/pplbench\_logistic\_regression\_query\_profile.json`
- `results/pplbench\_logistic\_regression\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_logistic\_regression\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_logistic\_regression\_pll.png`

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
Stan	-9.1522	0.06	0.30	986.50
WM-PLN (PeTTa)	-10.1150	0.1636	0.37	2070.85

Table C.1: Logistic-regression comparison on the refreshed WM-PLN lane. The new lane finally matches the benchmark model family rather than a stratified proxy, but the current variational scheduler still lags Stan on both fit and runtime.

Numerically, the refreshed batched run finishes at roughly -9.152 final PLL for Stan and -10.115 for WM-PLN, so WM-PLN is about 0.96 PLL units behind on this configuration. Compile time remains low on the WM-PLN side, changing from about 0.06s for Stan to about 0.16s for WM-PLN, while inference time now rises only from about 0.30s to about 0.37s. That is still slower than Stan, but it is far better than the earlier unbatched WM-PLN logistic run. The median effective-sample-size-per-second metric now moves from about 986.5 for Stan to about 2070.8 for WM-PLN. The separate query-profile artifact confirms why: after introducing batched Prolog-side

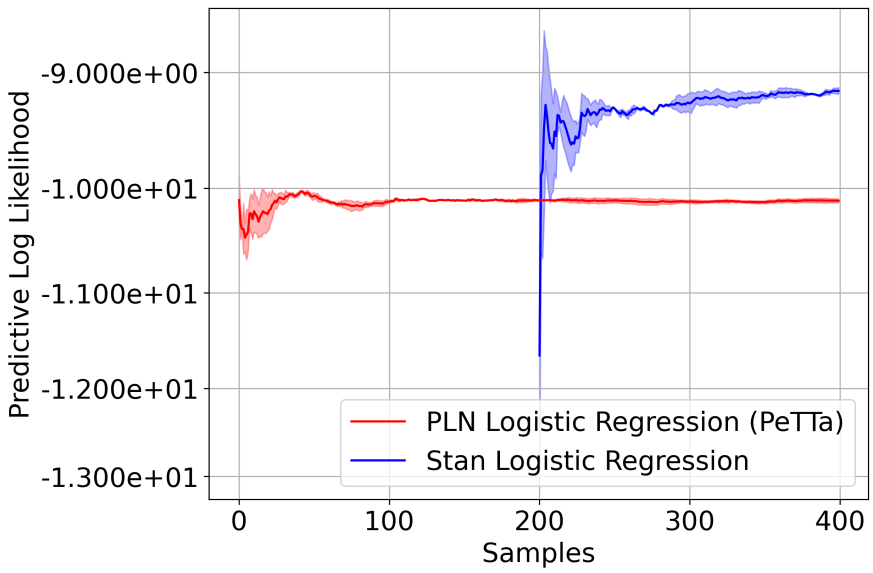


Figure C.1: Stan vs refreshed WM-PLN on the logistic-regression lane. The WM-PLN curve is stable and model-matched, but the diagonal variational scheduler keeps a clear gap to Stan on this first slice.

hyperstate, site-evidence, and posterior helpers, the fit loop drops to roughly 225 bridge calls and about 225 evaluated expressions over 25 iterations, down from roughly 350 calls and 40,150 expressions before batching. The same profile also shows that the remaining cost is dense numeric reduction rather than keyed lookup, so `MapSpace`/`SetSpace` remain a later option rather than the next optimization move. This is therefore an honest boundary result with real runtime progress:

Positive example. The weighted-Normal carrier plus explicit site hyperstate gives us a crisp WM-PLN logistic lane that matches the benchmark model family instead of falling back to stratified bins.

Negative example. The current diagonal variational scheduler is still too slow and too weak relative to Stan, so it is not yet a promotable reusable logistic inference engine.

That is still useful progress. It means the remaining work on logistic is now squarely about the scheduler, not about whether we are carrying the wrong semantic object.

Appendix D

Robust Regression: Continuous Carrier Under Heavy-Tailed Noise

The next lane keeps the same carrier-first discipline, but under a harder robustness requirement. The benchmark model `pplbench/models/robust\_regression.py` is linear regression with Student-t noise. A bad way to port that into WM-PLN would be to hide a closed-form or matrix solve in Python and merely expose its outputs through a PLN-looking wrapper. We deliberately did not do that.

Instead, the refreshed lane carries:

- additive weighted Gaussian evidence for the intercept and each coefficient,
- an explicit residual-scale hyperstate, and
- a queried tail-shape view.

The active semantics live in `hyperon/PeTTa/pln\_inference/robust\_regression\_semantic.metta`, while the operational scheduler lives in `pplbench/ppls/pln/robust\_regression.py`. The split is:

Promotable. The weighted continuous carrier pattern: explicit additive evidence state, queried posterior view.

Not yet promotable. The current iterative Student-t-style reweighting scheduler.

This is an honest partial match to the Stan model rather than a fake full posterior-family claim. In particular:

- `alpha` and `beta` are sampled from the queried Gaussian views of the carried evidence states;
- `sigma` is a queried robust residual-scale view repeated across draws; and
- `nu` is a fixed queried tail-shape view in this v1 lane, not a full WM-carried posterior random variable.

That limitation is visible by design.

The first check, before any benchmark headline, was the tiny exact calibration artifact `results/pplbench\_robust\_regression\_exact\_calibration.json`. It records:

- a one-feature Gaussian reduction case,
- a symmetric centered-feature case, and
- a direct residual-weight comparison showing that larger residuals receive strictly smaller Student-t-style weights.

The carried continuous state matches the analytic Gaussian posterior exactly on the first two cases, and the weight ordering behaves as intended on the third. So the current promotion boundary is:

Positive example. The weighted continuous carrier pattern is promotable.

Negative example. The iterative robust scheduler is still operational and is not yet promoted as reusable robust inference semantics.

We also keep one explicit scheduler guardrail artifact, `results/pplbench\_robust\_regression\_scheduler\_sanity.json`. It checks that the refreshed lane stays finite on clean and contaminated toy data, and that it resists a single large outlier better than plain unweighted OLS. Again, this is a non-promotion sanity test rather than a theorem that the full robust scheduler is exact.

The benchmark command is:

```
cd ~/claude/pplbench
ulimit -v 10485760
python3 -m pplbench examples/robust_regression_stan_vs_pln.json
```

The stable exported manuscript artifacts are:

- `results/pplbench\_robust\_regression\_exact\_calibration.json`
- `results/pplbench\_robust\_regression\_scheduler\_sanity.json`
- `results/pplbench\_robust\_regression\_stan\_vs\_pln\_summary.json`
- `results/pplbench\_robust\_regression\_stan\_vs\_pln\_table.tex`
- `results/figures/pplbench\_robust\_regression\_pll.png`

Backend	Final PLL	Compile (s)	Infer (s)	Median n_{eff}/s
Stan	-778.3170	22.28	0.11	5673.62
WM-PLN (PeTTa)	-781.5752	0.0001	1.29	622.24

Table D.1: Robust-regression comparison on the refreshed WM-PLN lane. The continuous evidence carrier remains honest and stable, but Stan keeps both the predictive-fit and runtime edge on this first heavy-tailed benchmark slice.

Numerically, the recorded run finishes at roughly -778.317 final PLL for Stan and -781.575 for WM-PLN, so WM-PLN is about 3.26 PLL units behind on this configuration. Compile time drops from about 22.28s to effectively zero, but inference time rises from about 0.11s to about 1.29s, and the median effective-sample-size-per-second metric falls from about 5673.6 to about 622.2. This is therefore a useful negative boundary result:

Positive example. The continuous evidence carrier pattern survives a heavy-tailed regression benchmark without collapsing back into disguised coefficient fitting.

Negative example. The current robust scheduler is slower and less predictively accurate than Stan on this first slice, so it is not yet a promotable reusable inference engine.

That is still good PPLBench progress. It tells us the next optimization target is no longer the carrier boundary; it is the robust scheduler itself.

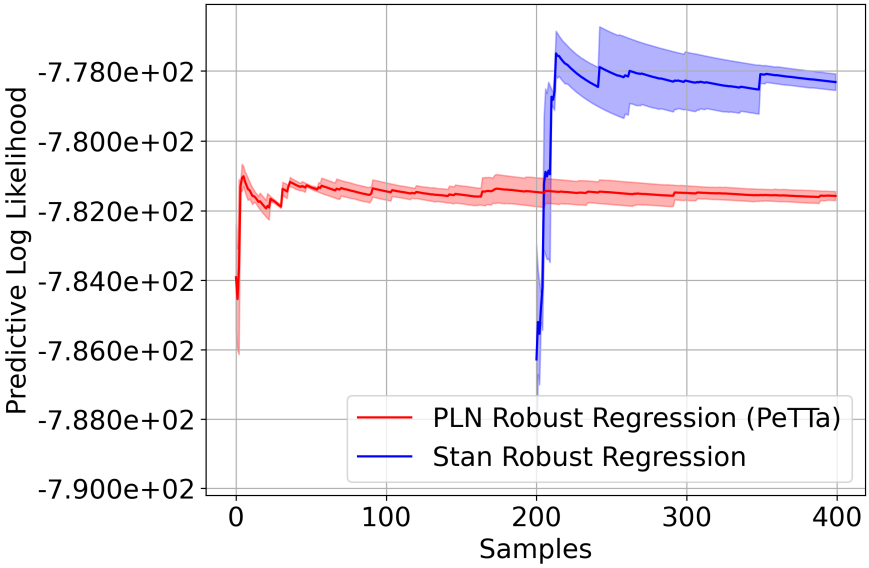


Figure D.1: Stan vs refreshed WM-PLN on the robust-regression lane. The WM-PLN curve is stable, but the current robust scheduler pays both a predictive-fit and runtime penalty relative to Stan on this first run.

Appendix E

Notation and Symbol Glossary

Symbol	Meaning	Lean name / location	
`Evidence`	Evidence type (n^+, n^-)	`Evidence` `EvidenceClass.lean`	in
`JointEvidence`	Dirichlet world-table	`JointEvidence` `PLNJointEvidence.lean`	in
`WorldModel`	World-model typeclass	`WorldModel` `PLNWorldModel.lean`	in
$\oplus$	Evidence revision (addition)	`Evidence.add`	
$\otimes$	Quantale tensor	`Evidence.tensor` `EvidenceQuantale.lean`	in
$\langle s, c \rangle$	Simple truth value	Strength/confidence pair	
Σ	Side-condition context	`WorldModelSigma.sigma`	
strength	Strength view	`queryStrength`	
confidence	Confidence view	`queryConfidence`	
n_{eff}	Effective sample size	`effectiveSampleSize` `PLNNoisyOr.lean`	in

Appendix F

Compact Theorem Map

Theorem family	Key result	Module	Label
Evidence compositionality	`evidence_add`	PLNWorldModel	CORE
Evidence conservation pack	`evidenceConservationPack_of_forgetting`	PLNWorldModelConservationPack	CONDITIONAL
Order-cost schedule bound	`scheduleErrorBound_of_pairwise_bounds`	PLNWorldModelOrderCostBounds	CONDITIONAL
Runtime audit certificate	`runtimePairwiseOrderCheck_certifies_policyThreshold`	PLNWorldModelOrderCostAuditCertificate	CONDITIONAL
Weighted non-commutative witness	`weightedScheduleErrorBound_two_step_not_zero_of_pos_weight`	PLNWorldModelOrderCostWeightedDemo	CONDITIONAL
Gas policy certificate	`gasOrderBudgetPolicy_batchSwap_zero_via_runtimeCertificate`	PLNWorldModelOrderCostGasPolicyDemo	CONDITIONAL
Local-completeness no-go	Thm 6.1	PLNJointEvidenceNoGo	BOUNDARY
BN chain exactness	`chainBN_plnDeductionStrength_exact`	PLNBayesNetFastRules	CONDITIONAL

BN fork exactness	`forkBN_plnDeductionStrengthBayesNet`	PLNBayesNetFastRules	CONDITIONAL
Quantale transitivity	tensor transitivity	EvidenceQuantale	CORE
Fréchet bounds	consistency iff	PLNFrechetBounds	CORE
De Finetti collapse	exchangeable → counts	DeFinetti	CORE
Solomonoff collapse	universal → counts	SolomonoffExchangeable	CORE
NB bridge	`PLN_tensorStrength_eq_mlnBayesNet`	PLNBayesNetInference	CORE
k-NN bridge	`PLN_hplusPos_eq_knnRelevance`	PremiseSelectionKNN_PLNBridge	CORE
Kyburg flattening	expectation consistency	PLNKyburgReduction	CORE
IE identifiability nogo	`same_first_two_terms_different_union`	PLNIntensionExclusionIdentifiability	BOUNDARY
Trail-free non-stabilization	`orbit_not_eventually_constant`	PLNTrailFreeDynamicsCounterexample	BOUNDARY
Damped convergence	`damped_sp_spn_eventually_constant`	PLNTrailFreeDampedConvergence	CONDITIONAL
Pooling uniqueness	external Bayes + total add	PremiseSelectionExternalBayesianity	CONDITIONAL
Coverage submodularity	greedy $(1 - 1/e)$ guarantee	PremiseSelectionCoverage	CORE
ProbLog bridge	`queryStrength = probLogQueryProb`	ProbLogDistributionSemantics	CORE
Noisy-OR	$1 - \prod(1 - s_i)$	PLNNoisyOr	CORE
Posterior mean	$S \cdot C + (1 - C) \cdot \mu_0$	PLNNoisyOr	CORE
PLN ↔ NARS	correspondence bundle	PLNNARSRuleCorrespondence	CORE

Appendix G

Code Navigation Guide

G.1 Entry Points

- `Mettapedia/Logic/PLNCore.lean` — umbrella module importing all core PLN infrastructure (153 lines).
- `Mettapedia/Logic/PLNCanonicalAPI.lean` — primary public API with stable, semantically grounded endpoints (3806 lines).
- `Mettapedia/Logic/README.md` — status table, dependency graph, and theorem index.

G.2 Applied Gaussian Demo Suite

Three end-to-end examples exercising continuous world models with increasing complexity:

1. **Broken Sensor** (§G, 360 lines, 0 sorry). Hybrid discrete/continuous state: Normal-Gamma temperature evidence + binary coolant-failure flag. Sleep consolidation, exceedance specification, component independence.
   `Mettapedia/Logic/PLNBrokenSensorDemo.lean`
2. **Widget Factory** (420 lines, 0 sorry). Per-source Gaussian mixtures: two machines with independent Normal-Gamma posteriors and source-attribution evidence. Mixture exceedance validity, mixture bounds ($\min \leq p_{\text{mix}} \leq \max$).
   `Mettapedia/Logic/PLNWidgetFactoryDemo.lean`

3. **Kalman Wake/Sleep** (353 lines, 0 sorry). 1D Kalman filter with temporal predict-update cycles. Kalman gain bounded ($0 < K < 1$), variance reduction, and the flagship *sleep* = *wake* theorem: batch replay yields the same posterior as sequential online filtering.
   `Mettapedia/Logic/PLNKalmanSleepDemo.lean`

4. **Gas Sensor Array Drift** (578 lines, 0 sorry). Real UCI data (DOI 10.24432/C5RP6W): 3 gas types (ethanol, ammonia, toluene) across 10 temporal batches spanning 36 months. Ternary source-conditional Gaussians, cross-gas independence, multi-batch sleep consolidation, and 3-way source weight normalization. Demonstrates sensor drift: posteriors from batch 1 (2007) vs. batch 10 (2011) capture genuine distributional shift.
   `Mettapedia/Logic/PLNGasSensorDriftDemo.lean`

5. **Steel Plates Faults** (488 lines, 0 sorry). Real UCI data (current UCI DOI 10.24432/C5J88N): 3 fault types (Pastry, K_Scratch, Bumps) classified by LogOfAreas. Per-fault Normal-Gamma posteriors, 3-way mixture exceedance validity, cross-fault independence, and source weight normalization. Manufacturing quality-control application of the WM calculus.
   `Mettapedia/Logic/PLNSteelFaultDemo.lean`

6. **NAB Machine Temperature Anomaly** (327 lines, 0 sorry). Real NAB benchmark data (MIT License): machine temperature sensor with 22,695 readings and labeled anomaly windows. Three operating regimes (calm ∼85°F, hot ∼101°F, anomaly ∼30°F). Kalman wake/sleep on real time series, multi-regime composition, anomaly exceedance ordering.
   `Mettapedia/Logic/PLNNABSleepDemo.lean`

Build all six:

```
lake build Mettapedia.Logic.PLNBrokenSensorDemo \
           Mettapedia.Logic.PLNWidgetFactoryDemo \
           Mettapedia.Logic.PLNKalmanSleepDemo \
           Mettapedia.Logic.PLNGasSensorDriftDemo \
           Mettapedia.Logic.PLNSteelFaultDemo \
           Mettapedia.Logic.PLNNABSleepDemo
```

G.3 Regression Targets

- Chapter 8: `lake build Mettapedia.Logic.PLNWorldModelNeighborhoodConsequence Mettapedia.Logic.PLNWorldModelCategoricalRegression`
- Chapter 9: `lake build Mettapedia.Logic.PLNSelectorRewriteThresholdRegression`
- Chapter 11: `lake build Mettapedia.Logic.PLNFirstOrder.QuantifierRegression`
- Chapter 12: `lake build Mettapedia.Logic.PLNIntensionalRegression`
- Chapter 13: `lake build Mettapedia.Logic.PLNInferenceControlRegression`

G.4 Build

```
cd ~/claude/lean-projects/mettapedia
export LAKE_JOBS=3
nice -n 19 lake build          # full build
lake exe cache get             # cache after update
```

Appendix H

PLN Book Coverage Map

Chapter	Core content	Lean coverage	Status
Ch. 1	Introduction	Semantic stack split (this document; PLNCore)	Done
Ch. 2	Knowledge repr.	PLNQuery, WorldModel, WorldModelSigma	Done
Ch. 3	Experiential sem.	EvidenceClass, EvidenceQuantale, PLNJointEvidence	Done
Ch. 4	Indefinite TVs	PLNIndefiniteTruth, PLNWorldModelITV, PLNWorldModelITVHypercube	Done
Ch. 5	FO ext. inference	PLNDeduction, PLNDerivation, PLNBayesNetFastRules	Done
Ch. 6	FO ext. + ITV	PLNWMOSLFBridgeITV, PLNCanonicalAPI	Done
Ch. 7	Distributional TVs	EvidenceBeta, PLNKyburgReduction, PLNDistributional; *Demos:* BrokenSensor, WidgetFactory, KalmanSleep, GasSensorDrift, SteelFault, NABSleep	Done
Ch. 8	Error magnification	PLNErrorMagnificationGrounding, PLNCanonicalAPI	Done
Ch. 9	Large-scale inf.	Counterexample-first core; operational pending	Partial
Ch. 10	Higher-order ext.	HigherOrder/HigherOrderReduction	Done

Ch. 11	Quantifiers + ITV	PLNFirstOrder (core + fuzzy/QFM + regressions)	Done
Ch. 12	Intensional inf.	PLNIntensionalWorldModel, SolomonoffBridge	Done
Ch. 13	Inference control	PLNInferenceControlCore, PremiseSelectionCoverage	Done
Ch. 14	Temporal/causal	PLNTemporalCausalInference, ProbabilisticEventCalculus	Done
App. A	PLN vs. NARS	PLNNARSRuleCorrespondence	Done

Appendix I

Advanced WM Wrappers

For readers interested in the categorical and modal-logic layers:

- **Kripke WM**: `Mettapedia/Logic/PLNWorldModelKripke.lean`— modal Kripke semantics instantiated over PLN world models, with sound/-complete implication bridges for K, KT, KD, K4 logics.
- **Neighborhood WM**: `Mettapedia/Logic/PLNWorldModelNeighborhood.lean`— neighborhood-frame semantics with E, EMN, EMT, ED logics and canonical Kripke→neighborhood representation.
- **Institutional WM**: `Mettapedia/Logic/PLNWorldModelInstitution.lean`— satisfaction conditions, Beck-Chevalley transport, and governance-facing mapped consequences.
- **HOL/FOL completeness wrappers**: `Mettapedia/Logic/PLNWorldModelHOLCompleteness.lean`, `Mettapedia/Logic/PLNWorldModelFOLCompleteness.lean`— consequence-transfer/closure into WM consequence rules.
- **Governance transport**: `Mettapedia/Logic/PLNWorldModelNeighborhoodConsequence.lean`— state-indexed governance formula translation preservation.

These layers are powerful but specialized. Most readers will not need them for understanding the core ξPLN story.

Bibliography

[1] Bruce Abramson, John Brown, Ward Edwards, Allan Murphy, and Robert L. Winkler. Hailfinder: A bayesian system for forecasting severe weather. *International Journal of Forecasting*, 12(1):57–71, 1996. Hailfinder is a Bayesian network for severe-weather forecasting in northeastern Colorado.

[2] David Atkinson and Jeanne Peijnenburg. A consistent set of infinite-order probabilities, 2013. Infinite-order probability reference used for the hierarchical ProbHOL layer.

[3] Jonathan Baron, Barbara A. Mellers, Philip E. Tetlock, Eric Stone, and Lyle H. Ungar. Two reasons to make aggregated probability forecasts more extreme. *Decision Analysis*, 11(2):133–145, 2014.

[4] José M. Bernardo and Adrian F. M. Smith. *Bayesian Theory*. Wiley Series in Probability and Statistics. John Wiley & Sons, 2nd edition, 2000.

[5] Stephen Bonner, Ian P. Barrett, Cheng Ye, Rowan Swiers, Ola Engkvist, Charles Tapley Hoyt, and William L. Hamilton. Understanding the performance of knowledge graph embeddings in drug discovery. *Artificial Intelligence in the Life Sciences*, 2:100036, 2022.

[6] Massimo Buscema, Semeion Research Center, Stefano Terzi, and William Tastle. A new meta-classifier. In *Annual Meeting of the North American Fuzzy Information Processing Society (NAFIPS)*, Toronto, Canada, 2010. Introduces the Steel Plates Faults dataset. The current UCI repository citation lists DOI 10.24432/C5J88N; older project drafts used 10.24432/C53P4W.

[7] Bruno de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Annales de l'Institut Henri Poincaré*, 7(1):1–68, 1937.

[8] Haim Gaifman. A theory of higher order probabilities, 1986. Higher-order probability reference used for the hierarchical ProbHOL layer.

[9] Scott Garrabrant, Tsvi Benson-Tilsen, Andrew Critch, Nate Soares, and Jessica Taylor. Logical induction, 2020. arXiv:1609.03543v5.

[10] Nil Geisweiller and Claude. Towards a complete formalization of pln. Internal draft (nuPLN v1), 2026. Local file: `/home/zar/claude/lean-projects/mettapedia/papers/nuPLN_v1.pdf`.

[11] Bita Ghasemkhani, Reyat Yilmaz, Derya Birant, and Recep Alp Kut. Logistic model tree forest for steel plates faults prediction. *Machines*, 11(7):679, 2023.

[12] Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. Springer, New York, 2009.

[13] David E. Heckerman, Eric J. Horvitz, and Bharat N. Nathwani. Toward normative expert systems: Part i. the Pathfinder project. *Methods of Information in Medicine*, 31(2):90–105, 1992. Pathfinder is a Bayesian expert system for diagnosing lymph-node diseases.

[14] Daniel S. Himmelstein, Michael Zietz, Vincent Rubinetti, Kyle Kloster, Benjamin J. Heil, and Casey S. Greene. Hetnet connectivity search provides rapid insights into how biomedical entities are related. *GigaScience*, 12, 2022.

[15] Daniel Scott Himmelstein. het.io-rep-data. `https://github.com/dhimmel/het.io-rep-data/tree/1a960f0e353586f8fe9f61b569919f24603d4344`, 2026. Public browser-table bundle used by the Rephetio frontend, accessed March 12, 2026.

[16] Daniel Scott Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L. Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, Catrina J. Lee, and Allan J. Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *eLife*, 6, 2017.

[17] IARPA. Iarpa announces publication of data from the good judgment project. `https://www.iarpa.gov/newsroom/article/iarpa-announces-publication-of-data-from-the-good-judgment-project`,

2024. Harvard Dataverse release of ACE / Good Judgment Project forecasting data, DOI 10.7910/DVN/BPCDH5.

[18] Zongze Jiang, Peng Xu, Yongbin Du, Feng Yuan, and Kai Song. Balanced distribution adaptation for metal oxide semiconductor gas sensor array drift compensation. *Sensors*, 21(10):3403, 2021.

[19] Ana Jiménez, María José Merino, Juan Parras, and Santiago Zazo. Explainable drug repurposing via path based knowledge graph completion. *Scientific Reports*, 14(1), 2024.

[20] Kevin H. Knuth and John Skilling. Foundations of inference. *Axioms*, 1(1):38–73, 2012. arXiv:1008.4831.

[21] A. Arun Kumar, Samarth Bhandary, Swathi Gopal Hegde, and Jhinuk Chatterjee. Knowledge graph applications and multi-relation learning for drug repurposing: A scoping review. *Computational Biology and Chemistry*, 115:108364, 2025.

[22] Henry E. Kyburg. Higher order probabilities and intervals. *International Journal of Approximate Reasoning*, 2(3):195–209, 1988.

[23] Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the Numenta anomaly benchmark. In *14th International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2015. NAB v1.0 scores: HTM 70.5, Windowed Gaussian 39.6.

[24] Yushan Liu, Marcel Hildebrandt, Mitchell Joblin, Martin Ringsquandl, Rime Raissouni, and Volker Tresp. Neural multi-hop reasoning with logical rules on biomedical knowledge graphs. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science and Demo Track*, Lecture Notes in Computer Science, pages 375–391. Springer International Publishing, 2021.

[25] Barbara Mellers, Lyle Ungar, Jonathan Baron, Jaime Ramos, Burcu Gürcay, Katrina Fincher, Sydney E. Scott, Don Moore, Pavel Atanasov, Samuel A. Swift, Terry Murray, Eric Stone, and Philip E. Tetlock. Psychological strategies for winning a geopolitical forecasting tournament. *Psychological Science*, 25(5):1106–1115, 2014.

[26] Susana Nunes, Samy Badreddine, and Catia Pesquita. Rewarding explainability in drug repurposing with knowledge graphs. In *Proceedings*

of the Thirty-Third International Joint Conference on Artificial Intelligence, pages 4624–4632. International Joint Conferences on Artificial Intelligence Organization, 2024.

[27] Pablo Perdomo-Quinteiro and Alberto Belmonte-Hernández. Knowledge graphs for drug repurposing: a review of databases and methods. *Briefings in Bioinformatics*, 25(6), 2024.

[28] Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1–2):107–136, 2006.

[29] Ville A. Satopää, Jonathan Baron, Dean P. Foster, Barbara A. Mellers, Philip E. Tetlock, and Lyle H. Ungar. Combining multiple probability predictions using a simple logit model. *International Journal of Forecasting*, 30(2):344–356, 2014.

[30] Marco Scutari. bnlearn bayesian network repository. `https://www.bnlearn.com/bnrepository/`, 2026. Public source for the canonical Pathfinder and Hailfinder BIF files used in our experiments; accessed March 12, 2026.

[31] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A. Ryan, Margie L. Homer, and Ramón Huerta. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, 166–167:320–329, 2012.

[32] Peter Walley. *Statistical Reasoning with Imprecise Probabilities*. Chapman and Hall, London, 1991.

[33] Peter Walley. Inferences from multinomial data: Learning about a bag of marbles. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):3–34, 1996. with discussion pages 34–57.

[34] Zarko Zaremba and Oruži. A formal review of Probabilistic Logic Networks. Draft, Mettapedia project, 2026.